

**LAPORAN AKHIR PRAKTIKUM  
ALGORITMA & STRUKTUR DATA**



**Oleh:**

**Nuzulia Sekti Mahanania**

**NIM. 2510817220006**

**PROGRAM STUDI TEKNOLOGI INFORMASI  
FAKULTAS TEKNIK  
UNIVERSITAS LAMBUNG MANGKURAT  
2026**

**LEMBAR PENGESAHAN**  
**LAPORAN AKHIR PRAKTIKUM ALGORITMA & STRUKTUR**  
**DATA**

Laporan Praktikum Algoritma & Struktur Data

Modul 1 : Struct dan Pointer

Modul 2 : Stack dan Queue

Modul 3 : Linked List

Modul 4 : Sorting

Modul 5 : Searching

Modul 6 : Graph dan Tree

ini disusun sebagai syarat lulus mata kuliah Praktikum Praktikum Algoritma & Struktur Data. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Nuzulia Sekti Mahanania

NIM : 2510817220006

Menyetujui,  
Asisten Praktikum

Mengetahui,  
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani  
NIM. 2310817310009

Muti'a Maulida, S.Kom., M.TI.  
NIP. 198810272019032013

## DAFTAR ISI

|                                   |      |
|-----------------------------------|------|
| LEMBAR PENGESAHAN.....            | ii   |
| DAFTAR ISI .....                  | iii  |
| DAFTAR GAMBAR.....                | vii  |
| DAFTAR TABEL .....                | viii |
| MODUL 1: STRUCT DAN POINTER ..... | 1    |
| SOAL 1.....                       | 1    |
| A. Source Code .....              | 1    |
| B. Output.....                    | 3    |
| C. Pembahasan.....                | 3    |
| SOAL 2.....                       | 5    |
| A. Source Code .....              | 5    |
| B. Output.....                    | 7    |
| C. Pembahasan.....                | 7    |
| MODUL 2: STACK DAN QUEUE .....    | 9    |
| SOAL 1.....                       | 9    |
| A. Source Code .....              | 9    |
| B. Pembahasan.....                | 10   |
| SOAL 2.....                       | 12   |
| A. Source Code .....              | 13   |
| B. Output.....                    | 15   |
| C. Pembahasan.....                | 15   |
| SOAL 3.....                       | 17   |
| A. Source Code .....              | 17   |
| B. Pembahasan.....                | 19   |
| SOAL 4.....                       | 21   |
| A. Source Code .....              | 22   |
| B. Output.....                    | 23   |
| C. Pembahasan.....                | 24   |
| MODUL 3: LINKED LIST .....        | 26   |

|                        |    |
|------------------------|----|
| SOAL 1 .....           | 26 |
| A. Source Code .....   | 26 |
| B. Pembahasan .....    | 31 |
| SOAL 2 .....           | 34 |
| A. Source Code .....   | 35 |
| B. Output .....        | 37 |
| C. Pembahasan .....    | 37 |
| SOAL 3 .....           | 41 |
| A. Source Code .....   | 41 |
| B. Pembahasan .....    | 47 |
| SOAL 4 .....           | 49 |
| A. Source Code .....   | 50 |
| B. Output .....        | 53 |
| C. Pembahasan .....    | 53 |
| MODUL 4: SORTING ..... | 57 |
| SOAL 1 .....           | 57 |
| A. Source Code .....   | 57 |
| B. Output .....        | 58 |
| C. Pembahasan .....    | 58 |
| SOAL 2 .....           | 62 |
| A. Source Code .....   | 62 |
| B. Output .....        | 63 |
| C. Pembahasan .....    | 63 |
| SOAL 3 .....           | 67 |
| A. Source Code .....   | 67 |
| B. Output .....        | 68 |
| C. Pembahasan .....    | 68 |
| SOAL 4 .....           | 73 |
| A. Source Code .....   | 73 |
| B. Output .....        | 75 |
| C. Pembahasan .....    | 75 |
| SOAL 5 .....           | 80 |

|                               |     |
|-------------------------------|-----|
| A. Source Code .....          | 80  |
| B. Output.....                | 81  |
| C. Pembahasan.....            | 82  |
| SOAL 6.....                   | 86  |
| A. Source Code .....          | 86  |
| B. Output.....                | 88  |
| C. Pembahasan.....            | 88  |
| MODUL 5: SEARCHING .....      | 93  |
| SOAL 1.....                   | 93  |
| A. Source Code .....          | 93  |
| B. Output.....                | 94  |
| C. Pembahasan.....            | 94  |
| SOAL 2.....                   | 96  |
| A. Source Code .....          | 96  |
| B. Output.....                | 97  |
| C. Pembahasan.....            | 97  |
| MODUL 6: GRAPH AND TREE ..... | 100 |
| Soal 1 .....                  | 100 |
| A. Source Code .....          | 100 |
| B. Output.....                | 102 |
| C. Pembahasan.....            | 102 |
| Soal 2 .....                  | 104 |
| A. Source Code .....          | 104 |
| B. Output.....                | 106 |
| C. Pembahasan.....            | 106 |
| Soal 3 .....                  | 108 |
| A. Source Code .....          | 108 |
| B. Output.....                | 110 |
| C. Pembahasan.....            | 111 |
| Soal 4 .....                  | 113 |
| A. Source Code .....          | 113 |
| B. Output.....                | 115 |

|                      |     |
|----------------------|-----|
| C. Pembahasan.....   | 115 |
| Soal 5 .....         | 118 |
| A. Source Code ..... | 118 |
| B. Output.....       | 121 |
| C. Pembahasan.....   | 121 |
| TAUTAN GIT .....     | 124 |

## DAFTAR GAMBAR

|                                                          |     |
|----------------------------------------------------------|-----|
| Gambar 1. Screenshot Hasil Jawaban Soal 1 Modul 1 .....  | 3   |
| Gambar 2. Screenshot Hasil Jawaban Soal 2 Modul 1 .....  | 7   |
| Gambar 3. Screenshot Hasil Jawaban Soal 2 Modul 2 .....  | 15  |
| Gambar 4. Screenshot Hasil Jawaban Soal 4 Modul 2 .....  | 23  |
| Gambar 5. Screenshot Hasil Jawaban Soal 2 Modul 3 .....  | 37  |
| Gambar 6. Screenshot Hasil Jawaban Soal 4 Modul 3 .....  | 53  |
| Gambar 7. Screenshot Hasil Jawaban Soal 1 Modul 4 .....  | 58  |
| Gambar 8. Screenshot Hasil Jawaban Soal 2 Modul 4 .....  | 63  |
| Gambar 9. Screenshot Hasil Jawaban Soal 3 Modul 4 .....  | 68  |
| Gambar 10. Screenshot Hasil Jawaban Soal 4 Modul 4 ..... | 75  |
| Gambar 11. Screenshot Hasil Jawaban Soal 5 Modul 4 ..... | 81  |
| Gambar 12. Screenshot Hasil Jawaban Soal 6 Modul 4 ..... | 88  |
| Gambar 13. Screenshot Hasil Jawaban Soal 1 modul 5 ..... | 94  |
| Gambar 14. Screenshot Hasil Jawaban Soal 2 Modul 5 ..... | 97  |
| Gambar 15. Screenshot Hasil Jawaban Soal 1 Modul 6 ..... | 102 |
| Gambar 16. Screenshot Hasil Jawaban Soal 2 Modul 6 ..... | 106 |
| Gambar 17. Screenshot Hasil Jawaban Soal 3 Modul 6 ..... | 110 |
| Gambar 18. Screenshot Hasil Jawaban Soal 4 Modul 6 ..... | 115 |
| Gambar 19. Screenshot Hasil Jawaban Soal 5 Modul 6 ..... | 121 |

## DAFTAR TABEL

|                                           |     |
|-------------------------------------------|-----|
| Tabel 1. Soal 1 Modul 1 .....             | 1   |
| Tabel 2. Source Code Soal 1 Modul 1 ..... | 1   |
| Tabel 3 Source Code Soal 1 Modul 1 .....  | 2   |
| Tabel 4. Soal 2 Modul 1 .....             | 5   |
| Tabel 5. Source Code Soal 2 Modul 1 ..... | 5   |
| Tabel 6 Source Code Soal 2 Modul 1 .....  | 6   |
| Tabel 7. Source Code Soal 1 Modul 2.....  | 9   |
| Tabel 8. Soal 2 Modul 2 .....             | 13  |
| Tabel 9. Source Code Soal 2 Modul 2.....  | 13  |
| Tabel 10. Source Code Soal 3 Modul 2..... | 17  |
| Tabel 11. Soal 4 Modul 2 .....            | 22  |
| Tabel 12. Source Code Soal 4 Modul 2..... | 22  |
| Tabel 13. Source Code Soal 1 Modul 3..... | 26  |
| Tabel 14. Soal 2 Modul 3 .....            | 35  |
| Tabel 15. Source Code Soal 2 Modul 3..... | 35  |
| Tabel 16. Source Code Soal 3 Modul 3..... | 41  |
| Tabel 17. Soal 4 Modul 3 .....            | 50  |
| Tabel 18. Source Code Soal 4 Modul 3..... | 50  |
| Tabel 19. Source Code Soal 1 Modul 4..... | 57  |
| Tabel 20. Source Code Soal 2 Modul 4..... | 62  |
| Tabel 21. Source Code Soal 3 Modul 4..... | 67  |
| Tabel 22. Source Code Soal 4 Modul 4..... | 73  |
| Tabel 23. Source Code Soal 5 Modul 4..... | 80  |
| Tabel 24. Source Code Soal 6 Modul 4..... | 86  |
| Tabel 25. Source Code Soal 1 Modul 5..... | 93  |
| Tabel 26. Source Code Soal 2 Modul 5..... | 96  |
| Tabel 27. Source Code Soal 1 Modul 6..... | 100 |
| Tabel 28. Source Code Soal 2 Modul 6..... | 104 |
| Tabel 29. Source Code Soal 3 Modul 6..... | 108 |



|                                           |     |
|-------------------------------------------|-----|
| Tabel 30. Source Code Soal 4 Modul 6..... | 113 |
| Tabel 31. Source Code Soal 5 Modul 6..... | 118 |

# MODUL 1: STRUCT DAN POINTER

## SOAL 1

Diberikan sebuah garis  $l$  yang direpresentasikan dalam bentuk persamaan umum:

$$ax + by + c = 0$$

dan sebuah titik  $P = (x_p, y_p)$ . Tentukan posisi titik  $P$  terhadap garis  $l$  dengan cara mengimplementasikan fungsi `gradient` dan `CheckPointPosition` pada file header yang telah diberikan.

*Tabel 1. Soal 1 Modul 1*

| Input       | Output  |
|-------------|---------|
| 1 0 -3 5 0  | Left    |
| 1 0 -3 1 7  | Right   |
| 2 -1 -1 1 1 | On Line |

### A. Source Code

*Tabel 2. Source Code Soal 1 Modul 1*

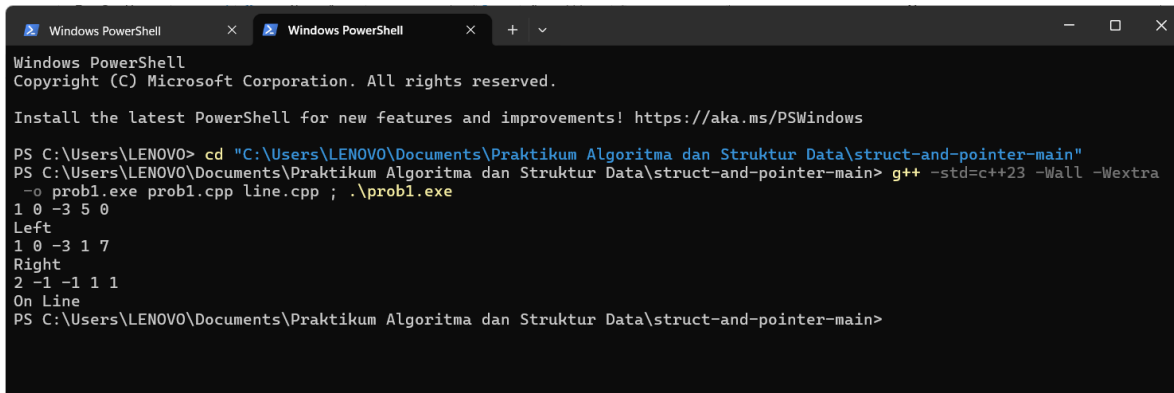
|    |                                                                                                 |
|----|-------------------------------------------------------------------------------------------------|
| 1  | <code>#include &lt;iostream&gt;</code>                                                          |
| 2  | <code>#include "line.h"</code>                                                                  |
| 3  | <code>#include "point.h"</code>                                                                 |
| 4  |                                                                                                 |
| 5  | <code>using namespace std;</code>                                                               |
| 6  |                                                                                                 |
| 7  | <code>int main() {</code>                                                                       |
| 8  | <code>    Line l;</code>                                                                        |
| 9  | <code>    Point p;</code>                                                                       |
| 10 |                                                                                                 |
| 11 | <code>    while (cin &gt;&gt; l.a &gt;&gt; l.b &gt;&gt; l.c &gt;&gt; p.x &gt;&gt; p.y) {</code> |

|    |                                        |   |
|----|----------------------------------------|---|
| 12 | double hasil_rumus = gradient(&l, &p); |   |
| 13 | string hasil_posisi                    | = |
|    | CheckPointPosition(hasil_rumus);       |   |
| 14 | cout << hasil_posisi << endl;          |   |
| 15 | }                                      |   |
| 16 |                                        |   |
| 17 | return 0;                              |   |
| 18 | }                                      |   |

*Tabel 3 Source Code Soal 1 Modul 1*

|    |                                                    |
|----|----------------------------------------------------|
| 1  | #include "line.h"                                  |
| 2  | #include <cmath>                                   |
| 3  |                                                    |
| 4  | double gradient(const Line * l, const Point * p) { |
| 5  | return (l->a * p->x) + (l->b * p->y) + (l->c);     |
| 6  | }                                                  |
| 7  | std::string CheckPointPosition(double val) {       |
| 8  | if (std::abs(val) < EPSILON) {                     |
| 9  | return "On Line";                                  |
| 10 | } else if (val > 0) {                              |
| 11 | return "Left";                                     |
| 12 | } else {                                           |
| 13 | return "Right";                                    |
| 14 | }                                                  |
| 15 | }                                                  |

## B. Output



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO> cd "C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\struct-and-pointer-main"
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\struct-and-pointer-main> g++ -std=c++23 -Wall -Wextra
-o probl.exe probl.cpp line.cpp ; .\probl.exe
1 0 -3 5 0
Left
1 0 -3 1 7
Right
2 -1 -1 1 1
On Line
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\struct-and-pointer-main>
```

Gambar 1. Screenshot Hasil Jawaban Soal 1 Modul 1

## C. Pembahasan

Alur program pada `probl.cpp` dan `line.cpp` bekerja dengan membaca input secara berulang melalui perintah `while (cin >> ...)`. Dalam proses ini, nilai koordinat titik ( $x, y$ ) dan koefisien garis ( $a, b, c$ ) yang telah dimasukkan akan dikirim ke fungsi `gradient(&l, &p)`. Fungsi tersebut pada dasarnya digunakan untuk menghitung nilai substitusi titik ke dalam persamaan garis menggunakan rumus  $ax + by + c$ . Setelah itu, hasil perhitungan akan disimpan pada variabel `hasil_rumus` dan langsung diteruskan ke fungsi `CheckPointPosition`. Pada proses pengecekan ini, disisipkan konstanta `EPSILON` untuk menangani masalah presisi pada tipe data desimal (*floating-point*). Apabila nilai dari hasil substitusi sangat mendekati nol, program secara otomatis akan menetapkan statusnya sebagai "On Line". Namun, apabila nilainya positif, fungsi akan mengembalikan nilai "Left", dan apabila nilainya negatif akan mengembalikan nilai "Right".

- **Kompleksitas Waktu**

Karena algoritma ini hanya melakukan operasi aritmatika dasar (perkalian dan penjumlahan) serta percabangan `if-else` standar pada setiap titik yang diuji, maka kompleksitas waktu yang dibutuhkan untuk tiap kasus uji adalah  $O(1)$ . Apabila terdapat  $N$  baris kasus uji saat perulangan berjalan, total kompleksitas waktunya adalah  $O(N)$ .

- **Kompleksitas Ruang**

Memori yang dialokasikan pada program ini hanya digunakan untuk menyimpan struktur data berukuran kecil, yaitu `Line l` dan `Point p`, serta beberapa variabel lokal bertipe `double` dan `string` untuk menampung output sementara. Oleh karena itu, kompleksitas ruangnya adalah  $O(1)$  atau bersifat konstan.

Berdasarkan hasil pengujian menggunakan contoh kasus uji, logika program ini telah terbukti benar dalam menentukan posisi titik terhadap sebuah garis. Sebagai pembuktian, misalnya pada input `1 0 -3 5 0` (merekpresentasikan garis  $1x + 0y - 3 = 0$  dan titik  $(5, 0)$ ), program berhasil memproses hasil substitusinya menjadi `2` (bernilai positif) dan mencetak output `Left`. Program ini juga terbukti berhasil menangani kondisi ketika suatu titik berada tepat di atas garis, contohnya pada input `2 -1 -1 1 1`. Pada hasil akhir, baris tersebut sukses dicetak sebagai `On Line` karena hasil perhitungannya adalah `0`, tanpa terdeteksi sebagai `Left` maupun `Right`.

## SOAL 2

Diberikan sebuah lingkaran C dengan pusat  $(c_x, c_y)$  dan jari-jari  $r > 0$ , serta sebuah titik  $P = (p_x, p_y)$ . Tentukan posisi titik P terhadap lingkaran C dan mengimplementasikan fungsi distance dan CheckPointInCircle pada file header yang diberikan.

*Tabel 4. Soal 2 Modul 1*

| Input        | Output    |
|--------------|-----------|
| 0 0 5 3 4    | On Circle |
| 0 0 5 1 1    | Inside    |
| 0 0 5 4 4    | Outside   |
| -2 -2 4 2 -2 | On Circle |

### A. Source Code

*Tabel 5. Source Code Soal 2 Modul 1*

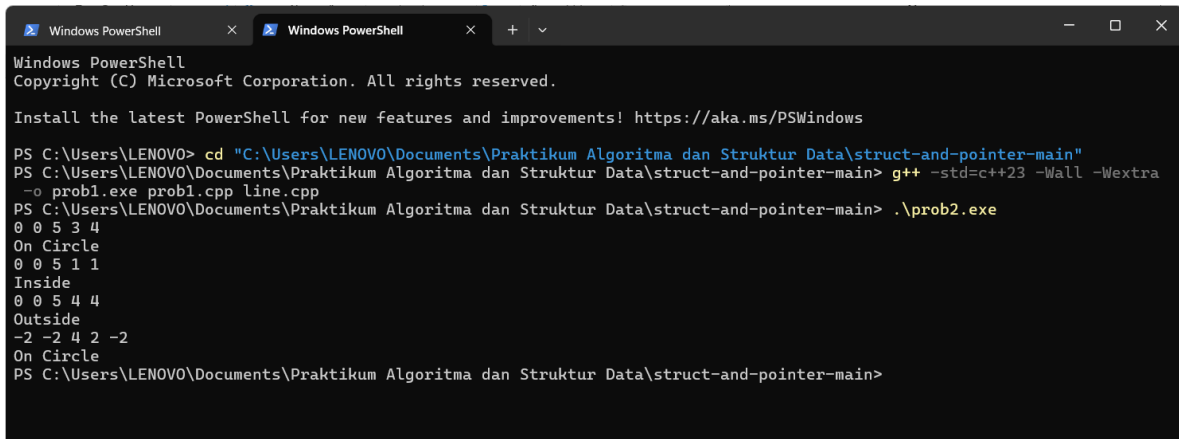
|    |                                                                     |
|----|---------------------------------------------------------------------|
| 1  | #include <iostream>                                                 |
| 2  | #include "circle.h"                                                 |
| 3  | #include "point.h"                                                  |
| 4  |                                                                     |
| 5  | using namespace std;                                                |
| 6  |                                                                     |
| 7  | int main() {                                                        |
| 8  | Circle c;                                                           |
| 9  | Point p;                                                            |
| 10 | while (cin >> c.centre.x >> c.centre.y >> c.radius >> p.x >> p.y) { |
| 11 | double hasil_hitung = distance(&c, &p);                             |
| 12 | string status = CheckPointInCircle(hasil_hitung);                   |
| 13 | cout << status << endl;                                             |

|    |           |
|----|-----------|
| 14 | }         |
| 15 | return 0; |
| 16 | }         |

*Tabel 6 Source Code Soal 2 Modul 1*

|    |                                                           |
|----|-----------------------------------------------------------|
| 1  | #include "circle.h"                                       |
| 2  | #include <cmath>                                          |
| 3  | double distance(const Circle * c, const Point * p) {      |
| 4  | double dx = p->x - c->centre.x;                           |
| 5  | double dy = p->y - c->centre.y;                           |
| 6  | double jarak_ke_pusat = std::sqrt((dx * dx) + (dy * dy)); |
| 7  | return jarak_ke_pusat - c->radius;                        |
| 8  | }                                                         |
| 9  | std::string CheckPointInCircle(double selisih) {          |
| 10 | if (std::abs(selisih) < EPSILON) {                        |
| 11 | return "On Circle";                                       |
| 12 | }                                                         |
| 13 | else if (selisih < 0) {                                   |
| 14 | return "Inside";                                          |
| 15 | }                                                         |
| 16 | else {                                                    |
| 17 | return "Outside";                                         |
| 18 | }                                                         |
| 19 | }                                                         |

## B. Output



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO> cd "C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\struct-and-pointer-main"
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\struct-and-pointer-main> g++ -std=c++23 -Wall -Wextra
-o prob1.exe prob1.cpp line.cpp
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\struct-and-pointer-main> .\prob2.exe
0 0 5 3 4
On Circle
0 0 5 1 1
Inside
0 0 5 4 4
Outside
-2 -2 4 2 -2
On Circle
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\struct-and-pointer-main>
```

Gambar 2. Screenshot Hasil Jawaban Soal 2 Modul 1

## C. Pembahasan

Alur program pada prob2.cpp dan circle.cpp bekerja dengan membaca input secara berulang melalui perintah `while (cin >> ...)`. Di proses ini, nilai koordinat pusat lingkaran (`c_x`, `c_y`), panjang jari-jari (`r`), serta koordinat titik yang akan diuji (`p_x`, `p_y`) yang dimasukkan akan dikirim ke fungsi `distance(&c, &p)`. Fungsi ini digunakan untuk menghitung jarak dari titik ke titik pusat lingkaran. Rumus yang dipakai cukup sederhana, yaitu mencari akar dari penjumlahan kuadrat selisih koordinat `x` dan `y`, lalu hasilnya dikurangi dengan panjang jari-jari lingkaran. Setelah itu, hasil perhitungannya akan disimpan pada variabel `hasil_hitung` dan diteruskan ke fungsi `CheckPointInCircle`. Pada proses pengecekan ini, digunakan bantuan konstanta `EPSILON` untuk menangani masalah ketepatan angka desimal pada program. Apabila nilai dari selisihnya sangat mendekati nol, program otomatis menetapkan statusnya sebagai "On Circle". Namun, apabila selisihnya bernilai negatif (kurang dari 0), fungsi akan menghasilkan nilai "Inside", dan apabila nilainya positif akan menghasilkan nilai "Outside".



- **Kompleksitas Waktu**

Karena program ini hanya menjalankan operasi hitungan matematika dasar serta pengecekan `if-else` pada setiap titik, maka kompleksitas waktu untuk tiap kasus ujinya adalah  $O(1)$ . Jika ada  $N$  baris kasus uji saat perulangan berjalan, total kompleksitas waktunya adalah  $O(N)$ .

- **Kompleksitas Ruang**

Memori yang dipakai pada program ini hanya digunakan untuk menyimpan struktur data berukuran kecil, yaitu `Circle c` dan `Point p`, ditambah beberapa variabel lokal biasa bertipe `double` dan `string` untuk menampung hasil sementara. Oleh karena itu, kompleksitas ruangnya adalah  $O(1)$  atau bersifat konstan.

Berdasarkan hasil pengujian menggunakan contoh kasus uji, logika program ini sudah terbukti benar dalam menentukan posisi titik terhadap lingkaran. Sebagai pembuktian, misalnya pada input `0 0 5 3 4` (titik pusat 0,0, jari-jari 5, dan titik uji 3,4), program berhasil menghitung jarak titik ke pusat menjadi 5. Karena selisih jarak dengan jari-jari adalah 0, program berhasil mencetak output `On Circle`. Program ini juga berhasil menangani kondisi saat suatu titik berada di dalam lingkaran, contohnya pada input `0 0 5 1 1`. Pada hasil akhir, baris tersebut sukses dicetak sebagai `Inside` karena jarak titik ke pusatnya lebih kecil dari jari-jari, sehingga hasil pengurangan selisihnya bernilai negatif.

## MODUL 2: STACK DAN QUEUE

### SOAL 1

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

#### A. Source Code

*Tabel 7. Source Code Soal 1 Modul 2*

|    |                                                                  |
|----|------------------------------------------------------------------|
| 1  | <code>#include "stack.h"</code>                                  |
| 2  | <code>#include &lt;stdexcept&gt;</code>                          |
| 3  |                                                                  |
| 4  | <code>void init(Stack* s) {</code>                               |
| 5  | <code>    s-&gt;top = nullptr;</code>                            |
| 6  | <code>}</code>                                                   |
| 7  |                                                                  |
| 8  | <code>bool isEmpty(const Stack* s) {</code>                      |
| 9  | <code>    return s-&gt;top == nullptr;</code>                    |
| 10 | <code>}</code>                                                   |
| 11 |                                                                  |
| 12 | <code>bool isFull(const Stack* s) {</code>                       |
| 13 | <code>    if (isEmpty(s)) return false;</code>                   |
| 14 | <code>    return s-&gt;top == s-&gt;data + MAX - 1;</code>       |
| 15 | <code>}</code>                                                   |
| 16 |                                                                  |
| 17 | <code>void push(Stack* s, int value) {</code>                    |
| 18 | <code>    if (isFull(s)) {</code>                                |
| 19 | <code>        throw std::overflow_error("Stack is full");</code> |
| 20 | <code>    }</code>                                               |
| 21 |                                                                  |

```

22     if (isEmpty(s)) {
23         s->top = s->data;
24     } else {
25         s->top++;
26     }
27     *(s->top) = value;
28 }
29
30 void pop(Stack* s) {
31     if (isEmpty(s)) {
32         throw std::underflow_error("Stack is empty");
33     }
34
35     if (s->top == s->data) {
36         s->top = nullptr;
37     } else {
38         s->top--;
39     }
40 }
41
42 int peek(const Stack* s) {
43     if (isEmpty(s)) {
44         throw std::underflow_error("Stack is empty");
45     }
46     return *(s->top);
47 }

```

## B. Pembahasan

Kode ini dibuat untuk mengimplementasikan struktur data tumpukan (*Stack*) menggunakan *array*. Memanfaatkan sebuah penanda atau pointer bernama *top* untuk mengetahui posisi elemen yang paling atas. Program menjalankan fungsi *init* untuk menyiapkan tumpukan dalam keadaan kosong dengan mengatur penanda *top* menjadi *nullptr*. Sebelum melakukan operasi lain, program perlu memastikan status tumpukan melalui fungsi *isEmpty* untuk mengecek apakah tumpukan masih kosong, dan fungsi *isFull* untuk melihat apakah penanda posisi atas sudah mencapai batas memori *array* yang disediakan. Saat ada data baru yang ingin dimasukkan, fungsi *push* akan menggeser

penanda posisi atas satu langkah maju, barulah nilai tersebut disimpan di alamat memori tersebut. Jika tumpukan sudah penuh, program akan otomatis melemparkan pesan *error*. Sebaliknya, untuk mengeluarkan data yang terakhir kali masuk, fungsi `pop` cukup memundurkan penanda tersebut satu langkah ke bawah sehingga data lama otomatis tidak dapat diakses lagi dan akan tertimpa pada saat penambahan data berikutnya. Jika mencoba mengambil data dari tumpukan yang kosong, program juga akan memberikan pesan *error*. Jika hanya ingin melihat nilai data paling atas tanpa mengubah urutannya, fungsi `peek` dapat langsung membaca nilai pada penanda tersebut.

- **Kompleksitas Waktu**

Karena program ini tidak melakukan proses pencarian atau perulangan, melainkan langsung mengakses satu titik memori tertentu melalui pointer, maka kompleksitas waktu untuk semua fungsinya adalah  $O(1)$ . Kecepatan eksekusi program akan selalu sama dan stabil, tidak peduli seberapa banyak data yang tersimpan di dalam tumpukan tersebut.

- **Kompleksitas Ruang**

Memori yang dialokasikan pada program ini hanya digunakan untuk menyiapkan *array*, ditambah sedikit ruang untuk penanda posisi atas. Karena penggunaan memori ini bersifat tetap dan tidak membesar secara terus-menerus, maka kompleksitas ruangnya juga adalah  $O(1)$ .

Berdasarkan alur kerjanya, program ini sudah terbukti benar dalam menerapkan prinsip LIFO (*Last-In-First-Out*). Sebagai contoh pembuktian, jika kita memasukkan beberapa angka secara berurutan melalui operasi penambahan, penanda posisi atas akan terus bergerak maju mengikuti angka yang terakhir masuk. Ketika kita memanggil fungsi untuk menghapus data, nilai lama tersebut tidak perlu dihapus dari memori, penanda posisinya cukup dimundurkan satu posisi saja.

## SOAL 2

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, \*, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.

Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

### Batasan

- $1 \leq n \leq 100$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- $n$  adalah jumlah simbol dalam ekspresi
- $A_i$  adalah nilai bilangan pada input
- Simbol yang valid hanya:
  - bilangan bulat
  - operator: +, -, \*, /

### Format Input

$n$

A1 A2 ... An

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang simbol
- Baris kedua berisi simbol A sebanyak n buah

### Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

### Contoh Kasus

*Tabel 8. Soal 2 Modul 2*

| Input                  | Output |
|------------------------|--------|
| 3<br>2 3 +             | 5      |
| 9<br>5 1 2 + 4 * + 3 - | 14     |
| 5<br>10 2 / 3 *        | 15     |

### A. Source Code

*Tabel 9. Source Code Soal 2 Modul 2*

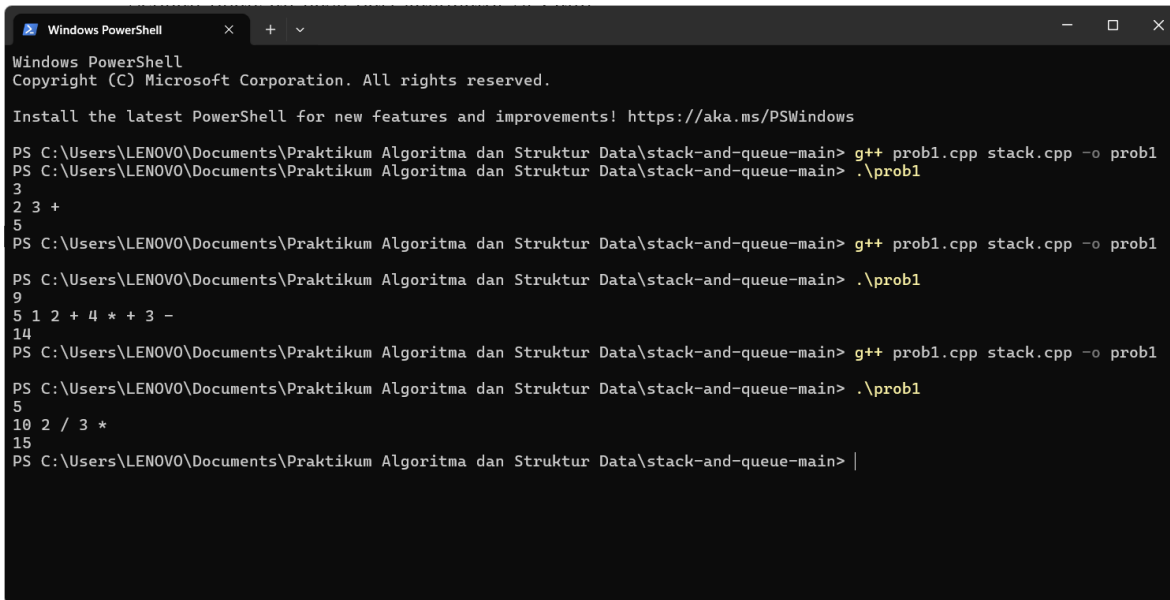
|    |                            |
|----|----------------------------|
| 1  | #include <iostream>        |
| 2  | #include <string>          |
| 3  | #include <cctype>          |
| 4  | #include "stack.h"         |
| 5  |                            |
| 6  | using namespace std;       |
| 7  |                            |
| 8  | int main() {               |
| 9  | int n;                     |
| 10 | if (!(cin >> n)) return 0; |
| 11 |                            |
| 12 | Stack s;                   |

```

13     init(&s);
14
15     for (int i = 0; i < n; i++) {
16         string token;
17         cin >> token;
18
19         if (isdigit(token[0]) || (token.length() > 1 &&
token[0] == '-')) {
20             push(&s, stoi(token));
21         } else {
22             int val2 = peek(&s); pop(&s);
23             int val1 = peek(&s); pop(&s);
24             if (token == "+") push(&s, val1 + val2);
25             else if (token == "-") push(&s, val1 - val2);
26             else if (token == "*") push(&s, val1 * val2);
27             else if (token == "/") push(&s, val1 / val2);
28         }
29     }
30
31     cout << peek(&s) << endl;
32     return 0;
33 }

```

## B. Output



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> g++ prob1.cpp stack.cpp -o prob1
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> .\prob1
3
2 3 +
5
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> g++ prob1.cpp stack.cpp -o prob1
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> .\prob1
9
5 1 2 + 4 * + 3 -
14
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> g++ prob1.cpp stack.cpp -o prob1
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> .\prob1
5
10 2 / 3 *
15
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> |
```

Gambar 3. Screenshot Hasil Jawaban Soal 2 Modul 2

## C. Pembahasan

Kode prob1.cpp ini dibuat untuk mengimplementasikan mesin penghitung dengan bantuan struktur data tumpukan (Stack). Awalnya, perintah input digunakan untuk membaca variabel `n` guna menentukan jumlah simbol yang dimasukkan. Setelah itu, pemanggilan fungsi `init` dilakukan agar program dapat menyiapkan tumpukan dalam keadaan kosong. Selanjutnya, struktur perulangan dibuat agar program bisa membaca setiap simbol satu per satu. Di dalam perulangan ini terdapat proses pemilahan, apabila simbol yang dibaca adalah angka, maka fungsi konversi `stoi` dipakai agar teks tersebut diubah menjadi bilangan bulat, lalu fungsi `push` dipanggil agar program menyimpannya ke dalam tumpukan. Namun, apabila simbol tersebut berupa operator matematika, fungsi `pop` dipanggil dua kali agar program mengeluarkan dua angka terakhir dari tumpukan. Setelah kedua angka dihitung sesuai jenis operatornya, fungsi `push` kembali dipanggil agar program langsung menyimpan hasil perhitungan tersebut ke posisi teratas. Setelah semua simbol selesai dibaca, fungsi `peek` digunakan agar program mencetak nilai akhir yang tersisa di dalam tumpukan.



- **Kompleksitas Waktu**

Karena hanya ada satu perulangan tunggal yang berjalan sebanyak jumlah simbol  $n$ , dan seluruh operasi tumpukan di dalamnya berjalan dengan kecepatan konstan, maka kompleksitas waktunya adalah  $O(n)$ . Waktu proses eksekusi bergantung pada panjang deretan simbol yang diberikan.

- **Kompleksitas Ruang**

Memori yang dialokasikan disiapkan untuk menyimpan angka ke dalam tumpukan. Karena ukuran memori ini dibatasi (sebanyak 100 elemen) sejak awal, kapasitasnya tidak akan membesar mengikuti jumlah input. Oleh karena itu, kompleksitas ruangnya adalah  $O(1)$ .

Berdasarkan alur kerjanya, logika kode ini sudah terbukti benar dalam mengeksekusi perhitungan matematika. Sebagai pembuktian pada contoh kasus input  $5\ 1\ 2\ +\ 4\ *\ +\ 3\ -$ , angka 5, 1, dan 2 dimasukkan secara berurutan agar tersimpan di tumpukan. Saat menjumpai operator  $+$ , instruksi yang ada akan membuat program mengeluarkan angka 2 dan 1, menjumlahkannya menjadi 3, lalu menyimpannya kembali. Tumpukan kini berisi 5 dan 3. Selanjutnya, angka 4 dimasukkan. Saat bertemu operator  $*$ , program mengeluarkan 4 dan 3, mengalikannya menjadi 12, lalu menyimpannya kembali. Setelah itu, operator  $+$  akan membuat program menjumlahkan 12 dan 5 menjadi 17. Terakhir, angka 3 dimasukkan, dan operator  $-$  akan mengurangkan 17 dengan 3, sehingga didapatkan nilai 14. Angka 14 inilah yang tersisa dan dicetak sebagai hasil akhir yang tepat.

### SOAL 3

Pada file `queue.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `queue.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

#### A. Source Code

*Tabel 10. Source Code Soal 3 Modul 2*

|    |                                                                                |
|----|--------------------------------------------------------------------------------|
| 1  | <code>#include "queue.h"</code>                                                |
| 2  | <code>#include &lt;stdexcept&gt;</code>                                        |
| 3  |                                                                                |
| 4  | <code>void init(Queue* q) {</code>                                             |
| 5  | <code>    q-&gt;front = nullptr;</code>                                        |
| 6  | <code>    q-&gt;rear = nullptr;</code>                                         |
| 7  | <code>}</code>                                                                 |
| 8  |                                                                                |
| 9  | <code>bool isEmpty(const Queue* q) {</code>                                    |
| 10 | <code>    return q-&gt;front == nullptr;</code>                                |
| 11 | <code>}</code>                                                                 |
| 12 |                                                                                |
| 13 | <code>bool isFull(const Queue* q) {</code>                                     |
| 14 | <code>    if (isEmpty(q)) return false;</code>                                 |
| 15 |                                                                                |
| 16 | <code>    const int* next_rear = q-&gt;data + ((q-&gt;rear - q-&gt;data</code> |
|    | <code>+ 1) % MAX);</code>                                                      |
| 17 |                                                                                |
| 18 | <code>    return next_rear == q-&gt;front;</code>                              |

```

19 }
20
21 void enqueue(Queue* q, int value) {
22     if (isFull(q)) throw std::overflow_error("Queue is
full");
23
24     if (isEmpty(q)) {
25         q->front = q->data;
26         q->rear = q->data;
27     } else {
28         q->rear = q->data + ((q->rear - q->data + 1) %
MAX);
29     }
30     *(q->rear) = value;
31 }
32
33 void dequeue(Queue* q) {
34     if (isEmpty(q)) throw std::underflow_error("Queue is
empty");
35
36     if (q->front == q->rear) {
37         q->front = nullptr;
38         q->rear = nullptr;
39     } else {
40         q->front = q->data + ((q->front - q->data + 1) %
MAX);
41     }
42 }
43
44 int front(const Queue* q) {

```

|    |                                                               |
|----|---------------------------------------------------------------|
| 45 | if (isEmpty(q)) throw std::underflow_error("Queue is empty"); |
| 46 | return *(q->front);                                           |
| 47 | }                                                             |
| 48 |                                                               |
| 49 | int back(const Queue* q) {                                    |
| 50 | if (isEmpty(q)) throw std::underflow_error("Queue is empty"); |
| 51 | return *(q->rear);                                            |
| 52 | }                                                             |

## B. Pembahasan

Kode `queue.cpp` ini dibuat untuk mengimplementasikan struktur data antrian menggunakan *array*. Dalam kode ini ada dua buah penanda (pointer), yaitu `front` untuk melacak posisi data terdepan dan `rear` untuk posisi data paling belakang. Pemanggilan fungsi `init` dilakukan agar program dapat menyiapkan antrian dalam keadaan kosong dengan mengatur kedua penanda tersebut menjadi `nullptr`. Sebelum melakukan operasi lain, pengecekan status perlu dilakukan melalui fungsi `isEmpty` untuk melihat apakah antrian masih kosong, dan fungsi `isFull` untuk memastikan apakah posisi `rear` jika digeser satu langkah akan menabrak posisi `front`. Saat ada data baru yang ingin dimasukkan, fungsi `enqueue` dipanggil agar program menggeser penanda `rear` satu langkah maju secara melingkar menggunakan rumus sisa bagi (modulo), lalu menyimpan nilainya di posisi tersebut. Jika antrian sudah penuh, peringatan error otomatis dilemparkan agar program menghentikan proses. Sebaliknya, untuk mengeluarkan data yang pertama kali masuk, fungsi `dequeue` dipanggil agar program memundurkan penanda `front` satu langkah, sehingga data lama otomatis tidak bisa diakses lagi. Jika mencoba mengambil data dari antrian kosong, peringatan error juga akan muncul. Terakhir, fungsi `front` dan `back` digunakan agar program bisa membaca nilai terdepan dan terbelakang secara instan tanpa mengubah struktur urutannya.

- **Kompleksitas Waktu**

Karena seluruh fungsi di dalam kode ini tidak menggunakan perulangan sama sekali dan hanya mengeksekusi perhitungan matematika dasar serta pergeseran alamat memori, maka kompleksitas waktunya adalah  $O(1)$ . Kecepatan eksekusi untuk operasi penambahan maupun penghapusan data akan selalu instan dan stabil.

- **Kompleksitas Ruang**

Memori yang dialokasikan utamanya hanya disiapkan untuk memesan array dengan ukuran yang sudah ditentukan dari awal, ditambah sedikit ruang untuk menyimpan penanda posisi. Karena kapasitas memori ini tetap dan tidak bertambah secara dinamis ketika berjalan, maka kompleksitas ruangnya juga adalah  $O(1)$ .

Berdasarkan alur kerjanya, program ini sudah terbukti benar dalam menerapkan prinsip FIFO (*First-In-First-Out*), yaitu data yang masuk paling awal akan menjadi data yang pertama kali keluar. Sebagai pembuktian, misalnya memasukkan angka 10, lalu 20, dan 30 secara berurutan. Saat fungsi `enqueue` dieksekusi, penanda `front` akan tetap menunjuk ke angka 10, sedangkan penanda `rear` akan terus bergeser hingga menetap di angka 30. Ketika fungsi pembacaan data terdepan dipanggil, nilai 10 akan langsung didapatkan. Selanjutnya, ketika fungsi `dequeue` dipanggil agar program menghapus data, nilai 10 tersebut tidak perlu dihapus secara rumit dari memori fisik, penanda `front` cukup digeser posisinya agar beralih menunjuk ke angka 20.

## SOAL 4

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah  $k$  angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih  $k$  angka berurutan.

Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

### Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- $n$ : jumlah elemen
- $k$ : ukuran jendela
- $A_i$ : elemen array

### Format Input

$n$   $k$

$A_1$   $A_2$  ...  $A_n$

Keterangan:

- Baris pertama  $n$  menyatakan jumlah atau panjang array. Lalu  $k$  besar atau jumlah elemen
- dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol  $A$  sebanyak  $n$  buah

### Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

## Contoh Kasus

Tabel 11. Soal 4 Modul 2

| Input                | Output     |
|----------------------|------------|
| 5 3<br>1 2 3 4 5     | 6 9 12     |
| 5 5<br>1 2 3 4 5     | 15         |
| 6 2<br>4 -1 2 -3 5 1 | 3 1 -1 2 6 |

## A. Source Code

Tabel 12. Source Code Soal 4 Modul 2

|    |                                 |
|----|---------------------------------|
| 1  | #include <iostream>             |
| 2  | #include "queue.h"              |
| 3  |                                 |
| 4  | using namespace std;            |
| 5  |                                 |
| 6  | int main() {                    |
| 7  | int n, k;                       |
| 8  | if (!(cin >> n >> k)) return 0; |
| 9  | if (k < 1) return 0;            |
| 10 |                                 |
| 11 | Queue q;                        |
| 12 | init(&q);                       |
| 13 |                                 |
| 14 | int sum = 0;                    |
| 15 |                                 |
| 16 | for (int i = 0; i < n; i++) {   |
| 17 | int x;                          |
| 18 | cin >> x;                       |
| 19 |                                 |

```

20         enqueue(&q, x);
21         sum += x;
22
23         if (i >= k) {
24             sum -= front(&q);
25             dequeue(&q);
26         }
27
28         if (i >= k - 1) {
29             if (i > k - 1) cout << " ";
30             cout << sum;
31         }
32     }
33     cout << endl;
34
35     return 0;
36 }

```

## B. Output

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> g++ prob2.cpp queue.cpp -o prob2
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> .\prob2
5 3
1 2 3 4 5
6 9 12
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> g++ prob2.cpp queue.cpp -o prob2
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> .\prob2
5 5
1 2 3 4 5
15
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> g++ prob2.cpp queue.cpp -o prob2
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> .\prob2
6 2
4 -1 2 -3 5 1
3 1 -1 2 6
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\stack-and-queue-main> |

```

Gambar 4. Screenshot Hasil Jawaban Soal 4 Modul 2



### C. Pembahasan

Kode `prob2.cpp` ini berfungsi menghitung kelompok angka secara berurutan dengan bantuan struktur data antrean (Queue). Awalnya, perintah input digunakan untuk membaca variabel `n` (jumlah total angka) dan `k` (ukuran kelompok angka). Setelah itu, pemanggilan fungsi `init` dilakukan agar program dapat menyiapkan antrean dalam keadaan kosong, dan variabel `sum` disiapkan dengan nilai 0 untuk menampung jumlah perhitungan. Selanjutnya, struktur perulangan dibuat agar program bisa membaca setiap angka satu per satu. Di dalam perulangan ini, setiap angka yang baru dibaca langsung ditambahkan ke variabel `sum`, lalu fungsi `enqueue` dipanggil agar program menyimpannya ke dalam antrean. Apabila jumlah angka di dalam antrean ternyata sudah melebihi batas ukuran kelompok `k`, fungsi `front` dipanggil untuk membaca nilai angka yang paling depan agar nilainya dikurangkan dari `sum`, kemudian fungsi `dequeue` dipanggil agar program langsung menghapus angka terdepan tersebut dari antrean. Setelah sebuah kelompok mencapai batas ukuran minimal `k` elemen, instruksi pencetakan digunakan agar program langsung menampilkan nilai `sum` saat itu ke layar.

- **Kompleksitas Waktu**

Karena hanya ada satu perulangan tunggal yang berjalan sebanyak jumlah angka `n`, dan seluruh operasi perpindahan dalam antrean berjalan dengan kecepatan konstan, maka kompleksitas waktunya adalah  $O(n)$ . Waktu proses eksekusi murni bergantung pada panjang deretan angka yang diberikan.

- **Kompleksitas Ruang**

Memori yang dialokasikan disiapkan untuk menampung kelompok angka ke dalam antrean. Karena batas maksimal elemen antrean sudah ditetapkan (sebanyak 100 elemen) sejak awal, kapasitas memori tidak akan membesar secara terus-menerus meskipun jumlah input angkanya sangat banyak. Oleh karena itu, kompleksitas ruangnya adalah  $O(1)$ .

Berdasarkan alur kerjanya, logika kode ini sudah terbukti benar dalam mengeksekusi perhitungan bergeser. Sebagai pembuktian pada contoh kasus input 5 3 dengan deretan angka 1 2 3 4 5, angka 1, 2, dan 3 dimasukkan secara berurutan agar tersimpan di antrean, sehingga

nilai `sum` bertambah menjadi 6. Karena ukuran elemen sudah pas mencapai 3, angka 6 langsung dicetak. Selanjutnya, angka 4 dimasukkan sehingga `sum` bertambah menjadi 10. Karena batas elemen melebihi 3, instruksi yang ada dibuat agar program membaca angka paling lama (yaitu 1), mengurangkannya dari total sehingga tersisa 9, lalu menghapus angka 1 tersebut dari antrean, dan mencetak angka 9. Terakhir, angka 5 dimasukkan, fungsi penghapusan kembali dipanggil agar program membuang angka terdepan (yaitu 2) sehingga total akhirnya menjadi 12, lalu dicetak.

## MODUL 3: LINKED LIST

### SOAL 1

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari **memory leak** (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan **Big-O Notation**. Seluruh manajemen memori harus dialokasikan secara manual di dalam **heap**.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

#### A. Source Code

*Tabel 13. Source Code Soal 1 Modul 3*

|    |                                                  |
|----|--------------------------------------------------|
| 1  | <code>#include &lt;iostream&gt;</code>           |
| 2  | <code>#include "single_linked_list.h"</code>     |
| 3  |                                                  |
| 4  | <code>using namespace std;</code>                |
| 5  |                                                  |
| 6  | <code>void SingleLinkedList::init() {</code>     |
| 7  | <code>    head = nullptr;</code>                 |
| 8  | <code>    tail = nullptr;</code>                 |
| 9  | <code>    size = 0;</code>                       |
| 10 | <code>}</code>                                   |
| 11 |                                                  |
| 12 | <code>bool SingleLinkedList::is_empty() {</code> |
| 13 | <code>    return size == 0;</code>               |

```

14 }
15
16 void SingleLinkedList::add_front(int val) {
17     Node* ptr_new = new Node;
18     ptr_new->data = val;
19     ptr_new->next = nullptr;
20
21     if (is_empty()) {
22         head = tail = ptr_new;
23         ptr_new->next = head;
24     } else {
25         ptr_new->next = head;
26         head = ptr_new;
27         tail->next = head;
28     }
29     size++;
30 }
31
32 void SingleLinkedList::add_back(int val) {
33     Node* ptr_new = new Node;
34     ptr_new->data = val;
35     ptr_new->next = nullptr;
36
37     if (is_empty()) {
38         head = tail = ptr_new;
39         ptr_new->next = head;
40     } else {
41         tail->next = ptr_new;
42         ptr_new->next = head;
43         tail = ptr_new;
44     }

```

```

45     size++;
46 }
47
48 void SingleLinkedList::add_idx(int val, int idx) {
49     if (idx < 0 || idx > size) throw "Indeks di luar
batas";
50
51     if (idx == 0) { add_front(val); return; }
52     if (idx == size) { add_back(val); return; }
53
54     Node* ptr_new = new Node;
55     ptr_new->data = val;
56     ptr_new->next = nullptr;
57
58     Node* ptr_curr = head;
59     for (int i = 0; i < idx - 1; i++) {
60         ptr_curr = ptr_curr->next;
61     }
62     ptr_new->next = ptr_curr->next;
63     ptr_curr->next = ptr_new;
64     size++;
65 }
66
67 void SingleLinkedList::delete_front() {
68     if (is_empty()) throw "Senarai kosong";
69
70     if (size == 1) {
71         delete head;
72         head = nullptr;
73         tail = nullptr;
74         size = 0;

```

```

75         return;
76     }
77     Node* ptr_del = head;
78     head = head->next;
79     tail->next = head;
80     delete ptr_del;
81     size--;
82 }
83
84 void SingleLinkedList::delete_back() {
85     if (is_empty()) throw "Senarai kosong";
86
87     if (size == 1) {
88         delete_front();
89         return;
90     }
91
92     Node* ptr_curr = head;
93     while (ptr_curr->next != tail) {
94         ptr_curr = ptr_curr->next;
95     }
96     delete tail;
97     tail = ptr_curr;
98     tail->next = head;
99     size--;
100 }
101
102 void SingleLinkedList::delete_idx(int idx) {
103     if (is_empty() || idx < 0 || idx >= size) throw
104         "Indeks di luar batas";

```

```

105     if (idx == 0) { delete_front(); return; }
106     if (idx == size - 1) { delete_back(); return; }
107
108     Node* ptr_curr = head;
109     for (int i = 0; i < idx - 1; i++) {
110         ptr_curr = ptr_curr->next;
111     }
112     Node* ptr_del = ptr_curr->next;
113     ptr_curr->next = ptr_del->next;
114     delete ptr_del;
115     size--;
116 }
117
118 void SingleLinkedList::display() {
119     if (is_empty()) {
120         cout << "EMPTY" << endl;
121         return;
122     }
123     Node* ptr_curr = head;
124     for (int i = 0; i < size; i++) {
125         cout << ptr_curr->data;
126         if (i < size - 1) {
127             cout << " -> ";
128         }
129         ptr_curr = ptr_curr->next;
130     }
131     cout << endl;
132 }
133
134 int SingleLinkedList::get(int idx) {

```

```

135     if (is_empty() || idx < 0 || idx >= size) throw
        "Indeks di luar batas";
136
137     Node* ptr_curr = head;
138     for (int i = 0; i < idx; i++) {
139         ptr_curr = ptr_curr->next;
140     }
141     return ptr_curr->data;
142 }
143
144 void SingleLinkedList::set(int val, int idx) {
145     if (is_empty() || idx < 0 || idx >= size) throw
        "Indeks di luar batas";
146
147     Node* ptr_curr = head;
148     for (int i = 0; i < idx; i++) {
149         ptr_curr = ptr_curr->next;
150     }
151     ptr_curr->data = val;
152 }
153
154 void SingleLinkedList::clear() {
155     while (!is_empty()) {
156         delete_front();
157     }
158 }

```

## B. Pembahasan

### 1. Konsep Dasar dan Alur Algoritma

Senarai Berantai Tunggal Melingkar atau *Circular Single Linked List* adalah struktur penyimpanan data yang berurutan. Pada struktur ini, setiap data dibungkus dalam sebuah



wadah bernama *node*. Setiap *node* memiliki sebuah *pointer* yang mengarah ke *node* selanjutnya. Ciri utamanya adalah *node* yang berada di posisi paling akhir (*tail*) selalu memiliki penunjuk yang mengarah kembali ke *node* paling awal (*head*). Hal inilah yang membuat strukturnya menjadi tidak terputus. Pada algoritma ini, ada operasi yang dapat berjalan seketika karena letak datanya sudah diketahui, yaitu fungsi *init*, *is\_empty*, *add\_front*, *add\_back*, dan *delete\_front*. Contoh, saat fungsi *add\_back* dijalankan untuk menambah data di bagian akhir, program cukup meletakkan *node* baru (*ptr\_new*) di belakang *tail* yang lama, lalu memastikan penunjuk dari *node* baru tersebut mengarah kembali ke *head*.

Namun, untuk melakukan proses di tengah barisan seperti pada fungsi *add\_idx*, *delete\_idx*, *delete\_back*, *get*, *set*, dan *display*, program harus menelusuri data satu per satu secara berurutan. Penelusuran ini menggunakan *pointer* bantuan bernama *ptr\_curr* yang berjalan maju dari awal hingga menemukan posisi indeks data yang diminta.

## 2. Analisis Kompleksitas Waktu dan Ruang (Big-O Notation)

Berdasarkan cara kerjanya, fungsi *init*, *is\_empty*, *add\_front*, *add\_back*, dan *delete\_front* memiliki kompleksitas waktu  $O(1)$ . Ini berarti proses tersebut berjalan secara langsung dalam satu langkah tanpa memedulikan seberapa banyak data yang ada di dalam barisan. Sebaliknya, fungsi yang memerlukan penelusuran seperti *add\_idx*, *delete\_idx*, *get*, *set*, *display*, serta *delete\_back* memiliki kompleksitas waktu  $O(N)$ . Hal tersebut dikarenakan program harus berjalan membaca barisan dari awal hingga mencapai titik yang dituju, di mana  $N$  mewakili jumlah data yang ditelusuri. Untuk kebutuhan kapasitas memori atau kompleksitas ruang, seluruh algoritma ini beroperasi dengan nilai  $O(1)$ . Artinya, program hanya membutuhkan satu ruang memori tambahan pada satu waktu saat memproses data, tanpa harus menggandakan seluruh struktur data yang sudah ada.

## 3. Pencegahan Kebocoran Memori

Sesuai larangan penggunaan STL, setiap penambahan data baru harus memesan ruang memori secara manual di area *heap* menggunakan perintah `new Node`. Karena memori di

area *heap* bersifat menetap dan tidak hilang otomatis, penumpukan data dapat menyebabkan *memory leak* yang memperlambat sistem. Untuk mencegahnya, fungsi `clear()` dibuat untuk membersihkan sisa memori. Fungsi ini mengulang proses `delete_front()` untuk menghapus *node* satu per satu menggunakan perintah `delete` hingga barisan benar-benar kosong.

#### **4. Validitas dan Kebenaran Algoritma**

Algoritma ini dijamin valid karena struktur melingkarnya disusun sesuai *template* yang diberikan dan sudah lolos *auto grader*. Setiap kali ada perubahan data di ujung barisan, penunjuk dari *node* terakhir akan selalu dihubungkan kembali ke bagian kepala senarai. Selain itu, algoritma ini sangat aman karena dilengkapi sistem peringatan menggunakan perintah `throw`. Jika program disuruh memproses indeks yang salah atau menghapus data saat senarai kosong, program akan menolak dan memberi peringatan agar sistem tidak berhenti mendadak.

## SOAL 2

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh  $N$  buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh  $K$  langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ( $K=K+1$ ).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ( $K=K-1$ ).
- Jika pada suatu titik energi lompatan habis atau negatif ( $K \leq 0$ ), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai  $K$  awal (saat pancaran energi pertama kali diinisialisasi)..

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

### Batasan

- $1 \leq N \leq 10^2$
- $1 \leq K \leq 10^9$
- $1 \leq A_i \leq 10^6$  (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1.

### Format Input

$N$   $K$

$A_1$   $A_2$  ...  $A_N$

### Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

### Contoh Kasus

Tabel 14. Soal 2 Modul 3

| Input              | Output |
|--------------------|--------|
| 5 2<br>1 4 3 6 2   | 6      |
| 4 5<br>10 11 12 13 | 12     |

### Penjelasan Contoh Kasus

Pada contoh pertama, susunan batu awal adalah [1,4,3,6,2] dengan  $K = 2$ . Dari batu pertama, lompatan 2 langkah mendarat di batu 4 sehingga batu tersebut dihancurkan. Karena 4 genap, nilai  $K$  bertambah menjadi 3 dan susunan berubah menjadi [1,3,6,2].

Selanjutnya, dengan  $K = 3$ , lompatan mendarat di batu 2. Batu 2 dihancurkan dan karena genap,  $K$  kembali bertambah menjadi 4. Susunan kini menjadi [1,3,6].

Pada fase berikutnya dengan  $K = 4$ , lompatan mendarat di batu 1. Karena 1 ganjil, setelah dihancurkan nilai  $K$  berkurang menjadi 3, menyisakan [3,6].

Terakhir, dengan  $K = 3$ , lompatan kembali mendarat di batu 3 sehingga batu tersebut dihancurkan. Struktur kini hanya menyisakan batu bernilai 6, sehingga keluaran akhirnya adalah 6.

### A. Source Code

Tabel 15. Source Code Soal 2 Modul 3

|   |                                 |
|---|---------------------------------|
| 1 | #include <iostream>             |
| 2 | #include "single_linked_list.h" |
| 3 |                                 |
| 4 | using namespace std;            |
| 5 |                                 |
| 6 | int main() {                    |
| 7 | int N, K;                       |

```

8      if (!(cin >> N >> K)) return 0;
9
10     SingleLinkedList list;
11     list.init();
12
13     for (int i = 0; i < N; i++) {
14         int val;
15         cin >> val;
16         list.add_back(val);
17     }
18
19     int init_k = K;
20     int curr_idx = 0;
21
22     while (list.size > 1) {
23         int target_idx = (curr_idx + (K - 1)) % list.size;
24         int target_val = list.get(target_idx);
25
26         list.delete_idx(target_idx);
27
28         curr_idx = target_idx % list.size;
29
30         if (target_val % 2 == 0) {
31             K++;
32         } else {
33             K--;
34         }
35
36         if (K <= 0) {
37             K = init_k;
38         }

```

```

39     }
40
41     cout << list.get(0) << endl;
42
43     list.clear();
44     return 0;
45 }

```

## B. Output

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> g++ single_linked_list.cpp prob_a.cpp -o prob_a
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> ./prob_a
5 2
1 4 3 6 2
6
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> g++ single_linked_list.cpp prob_a.cpp -o prob_a
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> ./prob_a
4 5
10 11 12 13
12
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular>

```

Gambar 5. Screenshot Hasil Jawaban Soal 2 Modul 3

## C. Pembahasan

### 1. Penjelasan Baris Kode

Baris [1-2] bagian ini digunakan untuk menyertakan *library* yang dibutuhkan oleh program. *Library* *iostream* digunakan untuk proses *input/output* standar, sedangkan "*single\_linked\_list.h*" disertakan untuk memanggil kerangka struktur data yang telah dibuat sebelumnya.

Baris [4] perintah `using namespace std;` digunakan agar penulisan fungsi standar seperti `cin` atau `cout` dapat ditulis secara langsung tanpa perlu diawali dengan instruksi `std::`.

Baris [6] merupakan fungsi utama (`main`) di mana seluruh alur eksekusi program pertama kali dimulai.

Baris [7-8] mendeklarasikan variabel `N` untuk merepresentasikan jumlah batu awal dan `K` untuk merepresentasikan jarak lompatan energi. Program kemudian membaca *input* dari pengguna untuk kedua variabel tersebut, dan jika *input* gagal dibaca atau tidak valid, program akan langsung berhenti (`return 0`).

Baris [10-11] baris ini membentuk sebuah objek *single linked list* baru dengan nama `list` dari kerangka `SingleLinkedList`. Setelah dibuat, metode `init()` dipanggil untuk memastikan *list* atau senarai dalam kondisi kosong dan siap digunakan.

Baris [13-17] merupakan blok perulangan (`for`) yang akan berjalan berulang-ulang sebanyak `N` kali sesuai jumlah batu. Di dalam perulangan ini, program membaca nilai setiap batu (`val`) lalu memasukkannya ke posisi paling belakang pada *list* menggunakan fungsi `add_back(val)`.

Baris [19-20] mendeklarasikan variabel `init_k` untuk menyimpan nilai asli dari daya lompatan `K` sebagai cadangan jika nanti terjadi *reset* energi. Selain itu, variabel `curr_idx` disiapkan dengan nilai 0 sebagai titik mula penunjuk lompatan dari batu pertama.

Baris [22] memulai proses perulangan utama (`while`) untuk menghancurkan batu. Program diinstruksikan untuk terus berputar dan menghancurkan batu selama sisa jumlah batu di dalam *list* (`list.size`) masih lebih dari 1.

Baris [23-24] program menghitung lokasi batu yang akan dihancurkan dengan rumus lompatan dan menyimpannya pada variabel `target_idx`. Penggunaan sisa bagi atau modulo (`% list.size`) berfungsi agar penunjuk posisi berputar kembali ke urutan awal jika lompatannya telah melewati batas ujung senarai. Setelah indeks ditemukan, nilai ukiran

pada batu tersebut diambil menggunakan fungsi `get()` dan disimpan di variabel `target_val`.

Baris [26] memanggil fungsi `delete_idx()` untuk menghancurkan dan mencabut batu pada posisi target dari dalam senarai.

Baris [28] memperbarui posisi saat ini (`curr_idx`) agar berpindah ke batu yang tepat berada setelah batu yang baru saja dihancurkan. Modulo kembali pakai sebagai antisipasi agar letak indeks tersebut menyesuaikan ukuran list yang baru menyusut.

Baris [30-34] menerapkan aturan anomali kuil. Jika nilai ukiran batu yang dihancurkan adalah genap (`target_val % 2 == 0`), maka kekuatan lompatan  $K$  akan bertambah 1. Sebaliknya, jika nilai batu tersebut ganjil, maka daya lompatan  $K$  akan berkurang 1.

Baris [36-38] menerapkan batasan daya pelindung energi. Jika perhitungan sebelumnya menyebabkan  $K$  habis atau bernilai negatif ( $K \leq 0$ ), maka kekuatan lompatan  $K$  akan langsung *reset* ke daya semula menggunakan nilai cadangan `init_k`.

Baris [41] setelah perulangan `while` selesai dan senarai hanya menyisakan tepat satu buah batu penentu, program akan mengambil dan mencetak nilai batu yang tersisa di posisi pertama `get(0)` tersebut ke layar.

Baris [43-45] fungsi `clear()` dipanggil untuk melakukan pembersihan seluruh sisa pesanan memori di dalam *heap* agar tidak terjadi *memory leak*. Setelah list kembali bersih tak berbekas, program ditutup dengan perintah `return 0`.

## **2. Hasil Analisis**

### **Strategi Algoritma**

Masalah utama pada Soal 2 adalah nilai lompatan energi ( $K$ ) yang sangat besar, yaitu dapat mencapai batas maksimal  $10^9$ . Apabila program dipaksa untuk berjalan menelusuri list langkah demi langkah sebanyak satu miliar kali pada setiap putarannya, program dipastikan akan melebihi batas waktu eksekusi maksimal 1.0 detik.



Oleh karena itu, strategi penyelesaiannya menggunakan manipulasi matematika sisa bagi (modulo). Indeks batu target dihitung secara langsung menggunakan rumus  $(curr\_idx + (K - 1)) \% list.size$ . Rumus modulo ini akan melewati seluruh lompatan yang berputar sia-sia tanpa harus menggeser *pointer* secara manual, dan langsung memberikan posisi indeks pasti dari batu yang harus dihancurkan.

### **Kompleksitas Waktu $O(N^2)$**

Program menjalankan perulangan `while` yang berputar terus-menerus hingga batu dari total awal  $N$  hanya tersisa satu buah. Di dalam setiap perulangan tersebut, program memanggil fungsi pencarian `get()` dan penghapusan `delete_idx()` pada *Single Linked List*. Kedua fungsi ini memerlukan waktu  $O(N)$  karena harus menelusuri barisan batu satu per satu dari urutan paling depan. Karena operasi pencarian berwaktu  $O(N)$  ini dieksekusi berulang kali di dalam perulangan yang juga berjumlah  $O(N)$ , maka total akumulasi waktu tempuhnya berlipat ganda menjadi  $O(N^2)$ .

### **Kompleksitas Ruang $O(1)$**

Seluruh data ukiran batu sejumlah  $N$  yang dimasukkan oleh pengguna di awal program disimpan di dalam satu barisan *Single Linked List*. Karena total pemakaian memori yang dipesan kepada sistem berbanding lurus dan tumbuh secara linear mengikuti banyaknya input batu  $N$ , maka kompleksitas ruangnya adalah  $O(N)$ .

### SOAL 3

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList`. beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari **memory leak** (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan **Big-O Notation**. Seluruh manajemen memori harus dialokasikan secara manual di dalam **heap**.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

#### A. Source Code

*Tabel 16. Source Code Soal 3 Modul 3*

|    |                                                           |
|----|-----------------------------------------------------------|
| 1  | <code>#include &lt;iostream&gt;</code>                    |
| 2  | <code>#include "double_linked_list.h"</code>              |
| 3  |                                                           |
| 4  | <code>using namespace std;</code>                         |
| 5  |                                                           |
| 6  | <code>void DoubleLinkedList::init() {</code>              |
| 7  | <code>    head = nullptr;</code>                          |
| 8  | <code>    tail = nullptr;</code>                          |
| 9  | <code>    size = 0;</code>                                |
| 10 | <code>}</code>                                            |
| 11 |                                                           |
| 12 | <code>bool DoubleLinkedList::is_empty() {</code>          |
| 13 | <code>    return size == 0;</code>                        |
| 14 | <code>}</code>                                            |
| 15 |                                                           |
| 16 | <code>void DoubleLinkedList::add_front(char val) {</code> |

```

17     Node* ptr_new = new Node;
18     ptr_new->data = val;
19     ptr_new->next = nullptr;
20     ptr_new->prev = nullptr;
21
22     if (is_empty()) {
23         head = ptr_new;
24         tail = ptr_new;
25         ptr_new->next = ptr_new;
26         ptr_new->prev = ptr_new;
27     } else {
28         ptr_new->next = head;
29         ptr_new->prev = tail;
30         head->prev = ptr_new;
31         tail->next = ptr_new;
32         head = ptr_new;
33     }
34     size++;
35 }
36
37 void DoubleLinkedList::add_back(char val) {
38     Node* ptr_new = new Node;
39     ptr_new->data = val;
40     ptr_new->next = nullptr;
41     ptr_new->prev = nullptr;
42
43     if (is_empty()) {
44         head = ptr_new;
45         tail = ptr_new;
46         ptr_new->next = ptr_new;
47         ptr_new->prev = ptr_new;

```

```

48     } else {
49         ptr_new->prev = tail;
50         ptr_new->next = head;
51         tail->next = ptr_new;
52         head->prev = ptr_new;
53         tail = ptr_new;
54     }
55     size++;
56 }
57
58 void DoubleLinkedList::add_idx(char val, int idx) {
59     if (idx < 0 || idx > size) return;
60
61     if (idx == 0) { add_front(val); return; }
62     if (idx == size) { add_back(val); return; }
63
64     Node* ptr_curr = head;
65     for (int i = 0; i < idx; i++) {
66         ptr_curr = ptr_curr->next;
67     }
68
69     Node* ptr_new = new Node;
70     ptr_new->data = val;
71     ptr_new->next = ptr_curr;
72     ptr_new->prev = ptr_curr->prev;
73
74     ptr_curr->prev->next = ptr_new;
75     ptr_curr->prev = ptr_new;
76     size++;
77 }
78

```

```

79 void DoubleLinkedList::delete_front() {
80     if (is_empty()) return;
81     if (size == 1) {
82         delete head;
83         head = nullptr;
84         tail = nullptr;
85         size = 0;
86         return;
87     }
88     Node* ptr_del = head;
89     head = head->next;
90     head->prev = tail;
91     tail->next = head;
92     delete ptr_del;
93     size--;
94 }
95
96 void DoubleLinkedList::delete_back() {
97     if (is_empty()) return;
98     if (size == 1) {
99         delete_front();
100        return;
101    }
102
103    Node* ptr_del = tail;
104    tail = tail->prev;
105    tail->next = head;
106    head->prev = tail;
107    delete ptr_del;
108    size--;
109 }

```

```

110
111 void DoubleLinkedList::delete_idx(int idx) {
112     if (is_empty() || idx < 0 || idx >= size) return;
113
114     if (idx == 0) { delete_front(); return; }
115     if (idx == size - 1) { delete_back(); return; }
116
117     Node* ptr_curr = head;
118     for (int i = 0; i < idx; i++) {
119         ptr_curr = ptr_curr->next;
120     }
121     ptr_curr->prev->next = ptr_curr->next;
122     ptr_curr->next->prev = ptr_curr->prev;
123     delete ptr_curr;
124     size--;
125 }
126
127 void DoubleLinkedList::display() {
128     if (is_empty()) {
129         cout << "EMPTY" << endl;
130         return;
131     }
132     Node* ptr_curr = head;
133     for (int i = 0; i < size; i++) {
134         cout << ptr_curr->data;
135         if (i < size - 1) {
136             cout << " <-> ";
137         }
138         ptr_curr = ptr_curr->next;
139     }
140     cout << endl;

```

```

141 }
142
143 char DoubleLinkedList::get(int idx) {
144     if (is_empty() || idx < 0 || idx >= size) return
145         '\0';
146     Node* ptr_curr = head;
147     for (int i = 0; i < idx; i++) {
148         ptr_curr = ptr_curr->next;
149     }
150     return ptr_curr->data;
151 }
152
153 void DoubleLinkedList::set(char val, int idx) {
154     if (is_empty() || idx < 0 || idx >= size) return;
155     Node* ptr_curr = head;
156     for (int i = 0; i < idx; i++) {
157         ptr_curr = ptr_curr->next;
158     }
159     ptr_curr->data = val;
160 }
161
162 void DoubleLinkedList::clear() {
163     while (!is_empty()) {
164         delete_front();
165     }
166 }

```

## B. Pembahasan

### 1. Konsep Dasar dan Alur Algoritma

Senarai Berantai Ganda Melingkar atau *Circular Double Linked List* adalah struktur data di mana setiap *node* memiliki dua buah penunjuk arah, yaitu penunjuk *next* untuk menyambung ke *node* setelahnya dan penunjuk *prev* untuk menyambung ke *node* sebelumnya. Sifat melingkarnya terbentuk karena penunjuk *next* pada *node* paling akhir (*tail*) mengarah kembali ke *node* paling awal (*head*), dan sebaliknya penunjuk *prev* pada *head* mengarah kembali ke *tail*. Pada algoritma ini, operasi penambahan dan penghapusan di bagian ujung seperti *init*, *is\_empty*, *add\_front*, *add\_back*, *delete\_front*, dan *delete\_back* dapat langsung dieksekusi tanpa pencarian.

Keunggulan senarai ganda terlihat pada fungsi *delete\_back* (menghapus data terakhir), program tidak perlu lagi menelusuri data dari awal karena *node* sebelum *tail* dapat langsung diketahui melalui penunjuk *prev*. Sementara itu, untuk proses yang melibatkan indeks spesifik di tengah barisan seperti *add\_idx*, *delete\_idx*, *get*, *set*, dan *display*, program tetap harus menelusuri data secara berurutan. Penelusuran ini dilakukan dengan menggunakan *pointer* bantuan *ptr\_curr* yang bergerak langkah demi langkah dari kepala senarai hingga mencapai letak posisi yang diminta.

### 2. Analisis Kompleksitas Waktu dan Ruang (Big-O Notation)

Berdasarkan alur kerjanya, kompleksitas waktu program ini terbagi menjadi dua. Fungsi operasi ujung seperti *init*, *is\_empty*, *add\_front*, *add\_back*, *delete\_front*, dan *delete\_back* memiliki kompleksitas waktu  $O(1)$ . Hal ini berarti prosesnya selesai dalam satu instruksi dasar, tanpa terpengaruh oleh seberapa panjang barisan data yang ada. Sebaliknya, fungsi yang mewajibkan penelusuran indeks seperti *add\_idx*, *delete\_idx*, *get*, *set*, dan *display* beroperasi pada kompleksitas waktu  $O(N)$ . Nilai  $N$  pada notasi tersebut mewakili banyaknya data yang harus dilalui oleh program untuk menemukan letak indeks yang ditargetkan.

Dari sisi kapasitas, seluruh fungsi di dalam algoritma ini memiliki kompleksitas ruang sebesar  $O(1)$ . Hal ini dikarenakan program hanya membuat penunjuk memori sementara



yang sangat kecil secara bergiliran saat proses berlangsung, tanpa pernah menduplikasi seluruh susunan data menjadi bentuk yang baru.

### 3. Pencegahan Kebocoran Memori

Sama seperti *Circular Single Linked List* dan sesuai larangan penggunaan STL, setiap penambahan data baru harus memesan ruang memori secara manual di area *heap* menggunakan perintah `new Node`. Karena memori di area *heap* bersifat menetap dan tidak hilang secara otomatis, penumpukan data lama dapat menyebabkan *memory leak* yang memberatkan komputer. Untuk mencegahnya, fungsi `clear()` diimplementasikan untuk membersihkan memori. Fungsi ini bekerja dengan mengulang proses `delete_front()` guna menghapus *node* satu per satu melalui perintah `delete` hingga barisan kembali kosong sempurna.

### 4. Validitas dan Kebenaran Algoritma

Algoritma ini dijamin valid karena fungsi yang digunakan sudah sesuai *template* yang diberikan dan berhasil diimplementasikan ke dalam kasus *prob\_b*, meskipun tidak lolos *auto grader*. Pada setiap perubahan letak data di ujung barisan, penunjuk silang `tail->next` dan `head->prev` selalu dihubungkan kembali secara bersamaan untuk mencegah putus rantai. Algoritma ini juga memiliki keamanan yang apabila program menerima instruksi di luar nalar (seperti memproses indeks minus), program akan langsung menolak dan menghentikan proses menggunakan perintah `return` sehingga struktur memori terhindar dari kerusakan sistem.

## SOAL 4

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi  $N$  karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari `head`, bergerak maju menggunakan `next`) dan Beta (dimulai dari `tail`, bergerak mundur menggunakan `prev`).

Palui melakukan observasi selama  $R$  ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak  $X$  langkah. Proyektor Beta dipaksa melompat mundur sebanyak  $Y$  langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter **next** dari karakter yang hancur, dan Beta berpindah ke karakter **prev** dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah  $R$  ronde selesai dieksekusi.

### Batasan

- $1 \leq N \leq 5000$
- $1 \leq R \leq 5000$
- $1 \leq X, Y \leq 10^{18}$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

### Format Input

$N$   $R$

$C_1$   $C_2$  ...  $C_n$

$X_1$   $Y_1$

$X_2$   $Y_2$

...

$X_R$   $Y_R$

Keterangan: Ci adalah satu karakter `char` tunggal.

### Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY.

### Contoh Kasus

Tabel 17. Soal 4 Modul 3

| Input                                           | Output |
|-------------------------------------------------|--------|
| 4 2<br>A B C D<br>1 1<br>2 3                    | ACB    |
| 5 3<br>V W X Y Z<br>100000 100000<br>2 2<br>5 5 | VWYZ   |

### Penjelasan Contoh Kasus

Pada contoh pertama, awalnya proyektor Alpha berada di node A dan proyektor B berada di node B. Pada ronde pertama Alpha maju satu langkah dan Beta mundur satu langkah, sehingga Alpha berada di node B dan Beta berada di node C. Karena bersebelahan, maka node B dan C di-swap, menjadi A C B D. Kemudian di ronde kedua, Alpha bergerak maju 2 langkah dan Beta mundur 3 langkah, sehingga keduanya berada di node D. Karena keduanya di node yang sama, maka node D dihapus. Sehingga hasil akhirnya ACB.

### A. Source Code

Tabel 18. Source Code Soal 4 Modul 3

|   |                                              |
|---|----------------------------------------------|
| 1 | <code>#include &lt;iostream&gt;</code>       |
| 2 | <code>#include "double_linked_list.h"</code> |
| 3 |                                              |
| 4 | <code>using namespace std;</code>            |
| 5 |                                              |

```

6  int main() {
7      int N, R;
8      if (!(cin >> N >> R)) return 0;
9
10     DoubleLinkedList list;
11     list.init();
12
13     for (int i = 0; i < N; i++) {
14         char input_char;
15         cin >> input_char;
16         list.add_back(input_char);
17     }
18
19     Node* ptr_alpha = list.head;
20     Node* ptr_beta = list.tail;
21
22     for (int i = 0; i < R; i++) {
23         long long X, Y;
24         if (!(cin >> X >> Y)) break;
25
26         if (list.size == 0) {
27             break;
28         }
29
30         long long step_alpha = X % list.size;
31         for (long long j = 0; j < step_alpha; j++) {
32             ptr_alpha = ptr_alpha->next;
33         }
34
35         long long step_beta = Y % list.size;
36         for (long long j = 0; j < step_beta; j++) {
37             ptr_beta = ptr_beta->prev;
38         }
39
40         if (ptr_alpha == ptr_beta) {
41             Node* ptr_del = ptr_alpha;
42
43             if (list.size == 1) {
44                 delete ptr_del;
45                 list.head = nullptr;
46                 list.tail = nullptr;

```

```

47         list.size = 0;
48         ptr_alpha = nullptr;
49         ptr_beta = nullptr;
50     } else {
51         ptr_alpha = ptr_del->next;
52         ptr_beta = ptr_del->prev;
53
54         ptr_del->prev->next = ptr_del->next;
55         ptr_del->next->prev = ptr_del->prev;
56
57         if (list.head == ptr_del) {
58             list.head = ptr_del->next;
59         }
60         if (list.tail == ptr_del) {
61             list.tail = ptr_del->prev;
62         }
63
64         delete ptr_del;
65         list.size--;
66     }
67 }
68 else if (ptr_alpha->next == ptr_beta ||
ptr_alpha->prev == ptr_beta) {
69     bool is_edge = (ptr_alpha == list.head &&
ptr_beta == list.tail) || (ptr_alpha == list.tail &&
ptr_beta == list.head);
70     if (!(is_edge && list.size > 2)) {
71         char temp = ptr_alpha->data;
72         ptr_alpha->data = ptr_beta->data;
73         ptr_beta->data = temp;
74     }
75 }
76 }
77
78 if (list.size == 0) {
79     cout << "EMPTY" << endl;
80 } else {
81     Node* ptr_curr = list.head;
82     for (int i = 0; i < list.size; i++) {
83         cout << ptr_curr->data;
84         ptr_curr = ptr_curr->next;

```

|    |               |
|----|---------------|
| 85 | }             |
| 86 | cout << endl; |
| 87 | }             |
| 88 |               |
| 89 | list.clear(); |
| 90 | return 0;     |
| 91 | }             |

## B. Output

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> g++ double_linked_list.cpp prob_b.cpp -o prob_b
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> ./prob_b
4 2
A B C D
1 1
2 3
ACB
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> g++ double_linked_list.cpp prob_b.cpp -o prob_b
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular> ./prob_b
5 3
V W X Y Z
100000 100000
2 2
5 5
VWXYZ
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\single linked list circular dan double linked list ci
rcular>

```

Gambar 6. Screenshot Hasil Jawaban Soal 4 Modul 3

## C. Pembahasan

### 1. Penjelasan Baris Kode

Baris [1-2] bagian ini bertugas untuk mengimpor *library* bagi berjalannya program. *Library* `<iostream>` disertakan untuk masukan dan keluaran standar, sedangkan berkas *header* `"double_linked_list.h"` diimpor agar program utama dapat mengenali dan menggunakan struktur data *Circular Double Linked List*.

Baris [4-6] perintah `using namespace std;` dideklarasikan agar penulisan kode menjadi efisien, sehingga pemanggilan fungsi tidak perlu diawali dengan `std::`.

Selanjutnya, baris 6 mendefinisikan `int main()`, fungsi utama dan titik awal sistem operasi mulai mengeksekusi logika program.

Baris [7-11] program mendeklarasikan variabel `N` untuk merepresentasikan jumlah karakter awal dan `R` untuk menentukan seberapa banyak ronde observasi yang akan dilakukan. Setelah program membaca *input* untuk kedua variabel tersebut, sebuah objek *list* baru bernama `list` diciptakan. Fungsi `init()` kemudian langsung dipanggil untuk mengamankan *list* agar benar-benar kosong dan bebas dari data sisa di dalam memori.

Baris [13-17] program menjalankan perulangan `for` sebanyak `N` kali untuk menyerap karakter. Pada setiap putarannya, sebuah karakter diserap ke dalam variabel `input_char` dan langsung disisipkan ke posisi paling akhir *list* menggunakan fungsi `add_back()` untuk menjamin urutannya sesuai dengan *input*.

Baris [19-20] fase peletakan posisi awal bagi kedua proyektor observasi. *Pointer* `ptr_alpha` diletakkan di titik awal barisan dengan merujuk pada `list.head`, sementara *pointer* `ptr_beta` diletakkan di titik akhir barisan dengan merujuk pada `list.tail`.

Baris [22-28] program memulai blok perulangan utama yang akan mengeksekusi proses lompatan sebanyak `R` ronde. Pada awal setiap ronde, program membaca besaran lompatan `X` (untuk Alpha) dan `Y` (untuk Beta). Baris 26-28 berfungsi sebagai sistem keamanan, apabila *list* sudah terdeteksi kosong sepenuhnya (`list.size == 0`), program akan langsung menghentikan sisa ronde menggunakan perintah `break` untuk mencegah *error* pembacaan memori.

Baris [30-38] bagian inti pengoptimalan waktu eksekusi. Mengingat nilai lompatan bisa bernilai sangat besar, program menggunakan operasi sisa bagi (modulo) terhadap ukuran *list* (`X % list.size` dan `Y % list.size`) untuk mendapatkan jumlah langkah efektif dan melewati putaran yang sia-sia. Setelah didapatkan jumlah langkah efektifnya, `ptr_alpha` digerakkan maju menggunakan *pointer* `next`, dan secara bersamaan `ptr_beta` digerakkan mundur menggunakan *pointer* `prev`.

Baris [40-67] blok kode ini menangani kasus "Tabrakan Singular", yaitu saat kedua proyektor mendarat di *node* yang sama persis (`ptr_alpha == ptr_beta`). Memori target disimpan ke dalam variabel `ptr_del`. Baris (43-49), jika ukuran *list* hanya tersisa satu, *node* tersebut langsung dihancurkan dan ukuran *list* dikosongkan total. Baris (50-66), jika elemen masih lebih dari satu, posisi proyektor diselamatkan terlebih dahulu (Alpha maju, Beta mundur). Setelah itu, *pointer node* sebelum dan sesudahnya disambung ulang agar lingkaran *list* tidak terputus. Program juga memperbarui acuan `list.head` dan `list.tail` jika perlu, lalu menghapus *node* (`delete ptr_del`) serta mengurangi ukuran *list*.

Baris [68-75] blok ini difokuskan untuk menangani "Resonansi Bersebelahan" melalui fungsi `else if`. Algoritma mendeteksi apakah posisi Alpha dan Beta saling menempel secara berdampingan. Jika terbukti berdampingan, program akan menukar data ukiran yang dibawa oleh kedua *node* tersebut. Proses pertukaran ini menggunakan variabel bantuan `temp` untuk memegang data sementara, sehingga pertukaran nilai antara Alpha dan Beta berjalan lancar tanpa menghilangkan salah satu data.

Baris [78-87] pencetakan hasil akhir. Program mengevaluasi *list*, jika hasil menunjukkan `list.size == 0`, program mencetak teks "EMPTY". Namun, jika data masih ada, program menggunakan *pointer* bantuan `ptr_curr` yang dimulai dari kepala *list* untuk menelusuri dan mencetak karakter demi karakter ke layar hingga mencapai akhir barisan.

Baris [89-91] sebagai langkah standar pembersihan pada C++, fungsi `clear()` dipanggil untuk melakukan perulangan penghapusan sisa-sisa *node* di dalam ruang *heap*. Pembersihan ini mutlak dilakukan untuk menjamin sistem terhindar dari kebocoran memori. Setelah pembersihan selesai, program ditutup dengan instruksi `return 0` pada baris 90.

## 2. Hasil Analisis

### Strategi Algoritma

Tantangan utama pada algoritma ini adalah besaran nilai lompatan proyektor Alpha (X) dan Beta (Y) yang sangat besar, yakni dapat mencapai batas  $10^{18}$ . Jika program dipaksa menelusuri memori satu per satu sebanyak nilai tersebut, program dipastikan akan memakan



waktu terlalu lama dan gagal . Oleh karena itu, strategi optimasi yang digunakan adalah memanfaatkan sifat melingkar dari struktur data ini melalui operasi sisa bagi atau modulo. Dengan menerapkan rumus  $X \% list.size$  dan  $Y \% list.size$ , program membuang seluruh putaran penuh yang berulang secara sia-sia dan langsung menghasilkan langkah efektifnya. Dengan ini, sejauh apa pun instruksi yang diberikan, *pointer* proyektor hanya perlu bergeser maksimal sebanyak jumlah elemen yang ada di dalam list, sehingga dapat berjalan dengan sangat cepat.

### **Kompleksitas Waktu $O(R \times N)$**

Program melakukan perulangan utama sebanyak  $R$  kali yang mewakili jumlah ronde observasi. Di dalam setiap ronde itu, proyektor Alpha dan Beta berjalan menelusuri list untuk mencari lokasi baru. Karena langkah ini telah dibatasi oleh rumus modulo, penelusuran maksimal yang dilakukan oleh proyektor adalah sepanjang jumlah data yang tersisa, yang memakan waktu  $O(N)$ . Setelah letak memori ditemukan, operasi pada *Double Linked List* seperti pencabutan *node* atau pertukaran nilai dapat dieksekusi seketika dalam waktu  $O(1)$ . Karena penelusuran berwaktu  $O(N)$  ini dijalankan berulang kali di dalam perulangan sebanyak  $R$  ronde, maka total kompleksitas waktunya berlipat ganda menjadi  $O(R \times N)$ .

### **Kompleksitas Ruang $O(N)$**

Program menyimpan rentetan karakter input dari pengguna ke dalam memori penyusun struktur *Circular Double Linked List*. Pemesanan ruang memori di area *heap* ini dialokasikan secara presisi mengikuti jumlah input awal  $N$ . Selama ronde observasi berlangsung, penggunaan ruang memori ini tidak pernah bertambah melewati ukuran aslinya, dan bahkan akan menyusut saat ada karakter yang dihapus akibat tabrakan. Karena kapasitas memori yang dibutuhkan murni berbanding lurus dan tumbuh secara linear mengikuti besaran input awal  $N$ , maka kompleksitas ruangnya dikategorikan sebagai  $O(N)$ .

## MODUL 4: SORTING

### SOAL 1

#### Bubble Sort

##### A. Source Code

*Tabel 19. Source Code Soal 1 Modul 4*

|    |                                                        |
|----|--------------------------------------------------------|
| 6  | // Algoritma Bubble Sort                               |
| 7  | void bubble_sort(std::vector<int>& data, Metrics& m) { |
| 8  | int n = data.size();                                   |
| 9  | auto start = Clock::now();                             |
| 10 |                                                        |
| 11 | for (int i = 0; i < n - 1; i++) {                      |
| 12 | bool swapped = false;                                  |
| 13 | for (int j = 0; j < n - i - 1; j++) {                  |
| 14 | m.comparisons++;                                       |
| 15 | if (data[j] > data[j + 1]) {                           |
| 16 | std::swap(data[j], data[j + 1]);                       |
| 17 | m.swaps++;                                             |
| 18 | swapped = true;                                        |
| 19 | }                                                      |
| 20 | }                                                      |
| 21 |                                                        |
| 22 | if (!swapped) break;                                   |
| 23 | }                                                      |
| 24 |                                                        |
| 25 | m.time_ms = std::chrono::duration<double,              |
|    | std::milli>(Clock::now() - start).count();             |
| 26 | }                                                      |

## B. Output

```
Windows PowerShell
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\sorting> g++ -O2 main.cpp data_generator.cpp sorting_algorithm.cpp reporter.cpp -o sort
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\sorting> ./sort
=====
      SORTING ALGORITHM - ANALYSIS
=====
>> Bubble Sort
Algorithm      Input Type      N      Comparisons      Swaps      Shifts      Rec.Calls      Arr.Access      Time(ms)
-----
Bubble Sort    Random          100     4935              2564       0           0              0              0.017900
Bubble Sort    Sorted           100      99               0           0           0              0              0.000600
Bubble Sort    Reverse Sorted  100     4950             4950       0           0              0              0.015500
Bubble Sort    Nearly Sorted   100     4719             296        0           0              0              0.006500
Bubble Sort    Many Duplicates 100     4845             2315       0           0              0              0.014800
Bubble Sort    Random         1000    497154           243190     0           0              0              1.228200
Bubble Sort    Sorted          1000    999              0           0           0              0              0.001100
Bubble Sort    Reverse Sorted  1000    499500           499500     0           0              0              1.647400
Bubble Sort    Nearly Sorted   1000    481922           25778      0           0              0              0.495400
Bubble Sort    Many Duplicates 1000    496340           240808     0           0              0              1.194900
Bubble Sort    Random         10000   49993404         25118148   0           0              0              129.313100
Bubble Sort    Sorted          10000   9999             0           0           0              0              0.008000
Bubble Sort    Reverse Sorted  10000   49995000         49995000   0           0              0              142.264000
Bubble Sort    Nearly Sorted   10000   49937030         3119352    0           0              0              70.190400
Bubble Sort    Many Duplicates 10000   49993404         25093086   0           0              0              136.603400
```

Gambar 7. Screenshot Hasil Jawaban Soal 1 Modul 4

## C. Pembahasan

### 1. Karakteristik Algoritma

#### a) Mekanisme Utama

Pertama, melakukan perulangan luar sebanyak  $n - 1$  kali untuk menelusuri *array*. Di dalamnya, pakai perulangan kedua untuk mengecek elemen bersebelahan. Karena yang diinginkan adalah memindahkan nilai terbesar ke posisi paling akhir, maka kondisinya: jika `data[j] > data[j + 1]` maka posisinya ditukar menggunakan `std::swap()`, kemudian variabel `swapped` di-set menjadi `true` sebagai penanda bahwa ada elemen yang digeser. Lalu *traverse* berlanjut terus sampai ke batas ujung *array* yang belum terurut. Setelah perulangan dalam ini selesai satu putaran, kondisinya dievaluasi, kalau nilainya `!swapped` yang artinya dari ujung kiri sampai kanan tidak ada yang ditukar sama sekali, maka langsung panggil perintah `break` untuk *early exit* supaya menghemat waktu komputasi. Kalau tidak di-`break`, perulangan luar lanjut terus beroperasi. Setelah semua perulangan berakhir, maka seluruh elemen otomatis sudah tersusun rapi.

#### b) Stabilitas (*Stable*)

Pada baris berikut ini.

```
if (data[j] > data[j + 1]) {
```

```

        std::swap(data[j], data[j + 1]);
        m.swaps++;
        swapped = true;
    }

```

Karena kodenya pakai tanda lebih besar dari ( $>$ ) dan bukan lebih besar sama dengan ( $>=$ ), angka yang bersebelahan cuma akan ditukar kalau angka di kirinya benar-benar lebih besar. Kalau program ketemu angka kembar yang berdampingan, kondisinya menjadi `false` dan mereka tidak akan ditukar. Jadi urutan asli angka kembar itu bakal tetap aman. Sehingga, bubble Sort di sini bersifat *stable*.

### c) Penggunaan Memori (In-place)

Dalam Algoritma ini, pengurutan dilakukan secara langsung dengan memanipulasi dan menukar nilai elemen di dalam wadah vector `data` asal menggunakan fungsi bawaan `std::swap`. Tidak ada deklarasi struktur data array baru yang mengalokasikan memori tambahan, sehingga sangat efisien secara memori (kompleksitas ruang  $O(1)$ ). Sehingga, algoritma ini termasuk *in-place sorting*.

## 2. Kompleksitas Waktu

### a) Best Case

Pada desain kode yang dibuat, terdapat implementasi variabel penanda `bool swapped = false` yang diakhiri dengan evaluasi `if (!swapped) break;`. Ketika algoritma ini menerima input data yang sudah terurut, kondisi perbandingan `if (data[j] > data[j + 1])` tidak akan pernah terpenuhi di sepanjang iterasi pertama. Akibatnya, nilai `swapped` akan tetap `false`. Program membaca hal ini sebagai sinyal bahwa data sudah rapi, lalu langsung mengeksekusi perintah `break` untuk menghentikan keseluruhan proses di putaran pertama. Karena program hanya menyusuri struktur data satu kali lintasan tanpa harus melakukan perulangan tambahan, maka dapat ditarik kesimpulan bahwa kompleksitas waktu terbaiknya tereduksi menjadi  $O(N)$ .

### b) Average Case

kode algoritma ini menggunakan perulangan *nested loop*. Ketika dihadapkan pada input data *Random*, sebagian besar elemen berada di posisi yang salah. Hal ini memaksa program untuk secara konstan memicu operasi penukaran, sehingga status `swapped` berubah menjadi `true` dan fitur `break` tidak aktif. Karena program menjalankan

perulangan luar dan dalam secara sebanding untuk memindahkan elemen ke ujung kanan, maka kompleksitas rata-ratanya adalah  $O(N^2)$ .

**c) Worst Case**

Kondisi paling buruk terjadi kalau datanya terurut terbalik. Di sini, angka di sebelah kiri pasti selalu lebih besar dari angka di kanannya. Jadinya, kondisi `if (data[j] > data[j + 1])` akan selalu `true` setiap kali dicek. Proses `std::swap` bakal jalan terus-terusan dan `swapped` tidak pernah `false`. Karena perintah `break` sama sekali tidak dapat giliran jalan, program terpaksa melakukan perulangan sampai batas akhirnya ( $N*N$ ). Dari alur ini, terbukti kalau kompleksitas terburuknya adalah mutlak  $O(N^2)$ .

**3. Analisis Operasi**

**a) Jumlah comparison**

Pada data *Sorted*, iterasi langsung dihentikan sehingga pencatatan metrik perbandingan sangat minimum, yaitu sebesar 99 kali. Sebaliknya, pada data *Reverse Sorted*, jumlah perbandingannya meledak mencapai batas algoritmik maksimal, yaitu sebanyak 4950 kali.

**b) Jumlah swap**

Sejalan dengan jumlah perbandingan, jumlah *swap* pada data *Sorted* tercatat 0 karena tidak ada elemen yang dipindahkan posisinya. Namun, jumlah pertukaran ini meningkat tajam hingga mencapai nilai maksimal sebesar 4950 kali ketika memproses data *Reverse Sorted*.

**c) Jumlah shift**

Tercatat 0 (Tidak ada). Bubble Sort beroperasi mutlak memindahkan data dengan skema penukaran posisi ganda (*swap*), bukan menggunakan *assignment shift* seperti algoritma Insertion Sort.

**d) Jumlah recursion call**

Tercatat 0 (Tidak ada). Algoritma dibangun menggunakan perulangan iteratif murni (*for loop*), tanpa rekursif.

**e) Jumlah array accessed**

Tercatat 0 (Tidak dihitung). Pencatatan hitungan akses struktur memori *array* diabaikan pada algoritma ini dan difokuskan khusus pada algoritma Radix Sort.

**f) Pengaruh bentuk data terhadap jumlah operasi**

Algoritma ini sangat sensitif terhadap bentuk awal data. Jika derajat keterurutan data sudah tinggi sejak awal, jumlah operasi fundamental komputasi (*comparison* dan *swap*) akan tereduksi secara drastis, menghemat kinerja prosesor.

**4. Analisis Performa**

**a) Perbandingan waktu eksekusi antar algoritma**

Dibandingkan dengan algoritma Merge Sort dan Quick Sort penyelesaian komputasi pengurutan dengan Bubble Sort pada data acak berkapasitas sangat besar jauh lebih lambat akibat sifat laju pertumbuhan eksekusi waktu yang kuadratik.

**b) Performa pada dataset kecil dan besar**

Pada rentang ukuran *dataset* kecil ( $N=100$ ) dalam kondisi terbaiknya (*Sorted*), Bubble Sort tergolong responsif dengan catatan waktu 0.000600 ms karena *overhead* sistem yang rendah. Namun, kelemahannya terekspos saat rentang data membesar ( $N=10000$ ); Bubble Sort mengalami degradasi kecepatan ekstrem dan memakan waktu hingga 0.008000 ms.

**c) Pengaruh distribusi data terhadap performa algoritma**

Waktu eksekusi yang optimal diperoleh secara berurutan mulai dari distribusi data *Sorted*, diikuti *Nearly Sorted*, lalu data *Random*. Skala waktu terburuk selalu tercatat saat algoritma dipaksa menyelesaikan data terbalik (*Reverse Sorted*).

**d) Hubungan teori kompleksitas dengan hasil eksperimen**

Sesuai teori  $O(N^2)$ , waktu beban datanya dinaikkan 10 kali lipat (dari ukuran 1000 jadi 10000), waktu prosesnya tidak sekadar naik 10 kali lipat, tapi melonjak jadi ratusan kali lipat lebih lama. Ini jadi bukti kalau kelemahan algoritmanya memang ada di skala perulangan datanya.

## SOAL 2

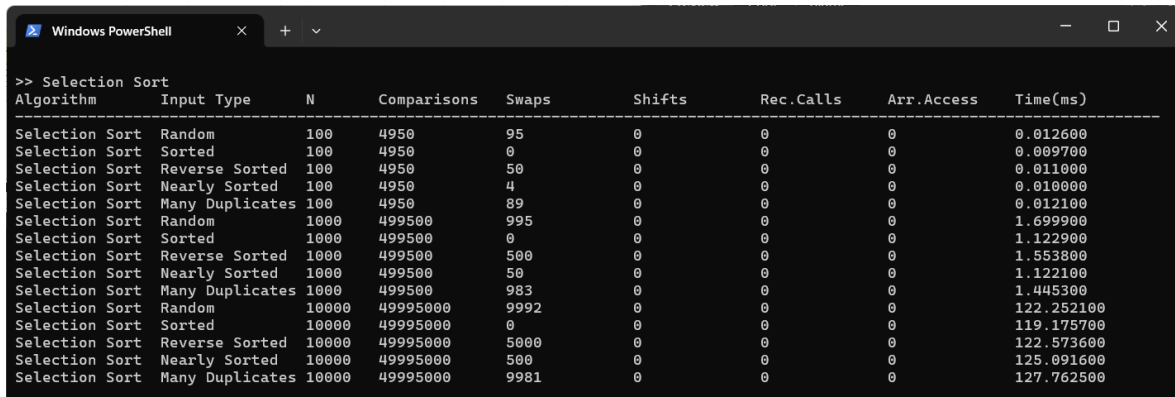
### Selection Sort

#### A. Source Code

*Tabel 20. Source Code Soal 2 Modul 4*

|    |                                                         |
|----|---------------------------------------------------------|
| 29 | // Algoritma Selection Sort                             |
| 30 | void selection_sort(std::vector<int>& data, Metrics& m) |
|    | {                                                       |
| 31 | int n = data.size();                                    |
| 32 | auto start = Clock::now();                              |
| 33 |                                                         |
| 34 | for (int i = 0; i < n - 1; i++) {                       |
| 35 | int index_terkecil = i;                                 |
| 36 | for (int j = i + 1; j < n; j++) {                       |
| 37 | m.comparisons++;                                        |
| 38 | if (data[j] < data[index_terkecil]) {                   |
| 39 | index_terkecil = j;                                     |
| 40 | }                                                       |
| 41 | }                                                       |
| 42 | if (index_terkecil != i) {                              |
| 43 | std::swap(data[i], data[index_terkecil]);               |
| 44 | m.swaps++;                                              |
| 45 | }                                                       |
| 46 | }                                                       |
| 47 |                                                         |
| 48 | m.time_ms = std::chrono::duration<double,               |
|    | std::milli>(Clock::now() - start).count();              |
| 49 | }                                                       |

## B. Output



| >> Selection Sort |                 |       |             |       |        |           |            |            |  |
|-------------------|-----------------|-------|-------------|-------|--------|-----------|------------|------------|--|
| Algorithm         | Input Type      | N     | Comparisons | Swaps | Shifts | Rec.Calls | Arr.Access | Time(ms)   |  |
| Selection Sort    | Random          | 100   | 4950        | 95    | 0      | 0         | 0          | 0.012600   |  |
| Selection Sort    | Sorted          | 100   | 4950        | 0     | 0      | 0         | 0          | 0.009700   |  |
| Selection Sort    | Reverse Sorted  | 100   | 4950        | 50    | 0      | 0         | 0          | 0.011000   |  |
| Selection Sort    | Nearly Sorted   | 100   | 4950        | 4     | 0      | 0         | 0          | 0.010000   |  |
| Selection Sort    | Many Duplicates | 100   | 4950        | 89    | 0      | 0         | 0          | 0.012100   |  |
| Selection Sort    | Random          | 1000  | 499500      | 995   | 0      | 0         | 0          | 1.699900   |  |
| Selection Sort    | Sorted          | 1000  | 499500      | 0     | 0      | 0         | 0          | 1.122900   |  |
| Selection Sort    | Reverse Sorted  | 1000  | 499500      | 500   | 0      | 0         | 0          | 1.553800   |  |
| Selection Sort    | Nearly Sorted   | 1000  | 499500      | 50    | 0      | 0         | 0          | 1.122100   |  |
| Selection Sort    | Many Duplicates | 1000  | 499500      | 983   | 0      | 0         | 0          | 1.445300   |  |
| Selection Sort    | Random          | 10000 | 49995000    | 9992  | 0      | 0         | 0          | 122.252100 |  |
| Selection Sort    | Sorted          | 10000 | 49995000    | 0     | 0      | 0         | 0          | 119.175700 |  |
| Selection Sort    | Reverse Sorted  | 10000 | 49995000    | 5000  | 0      | 0         | 0          | 122.573600 |  |
| Selection Sort    | Nearly Sorted   | 10000 | 49995000    | 500   | 0      | 0         | 0          | 125.091600 |  |
| Selection Sort    | Many Duplicates | 10000 | 49995000    | 9981  | 0      | 0         | 0          | 127.762500 |  |

Gambar 8. Screenshot Hasil Jawaban Soal 2 Modul 4

## C. Pembahasan

### 1. Karakteristik Algoritma

#### a) Mekanisme Utama

Pertama, melakukan inisiasi perulangan luar dari awal array sampai elemen sebelum terakhir. Karena yang diinginkan adalah elemen dengan nilai paling minimum, maka pakai inisiasi tebakan awal untuk mengunci indeks, yaitu `int index_terkecil = i;`. Untuk mengecek sisa datanya, dilakukan penelusuran pakai perulangan dalam di sisa elemen sebelah kanannya. Kemudian baca nilainya, jika nilai `data[j]` ternyata secara kurang dari `data[index_terkecil]`, maka perbarui `index_terkecil` ke indeks `j` tersebut. Setelah perulangan dalam berakhir, maka dilakukan pengecekan akhir. Kalau didapati `index_terkecil != i`, langsung panggil `std::swap()` untuk menukar nilai terkecil itu ke depan. Hal ini diulang terus sampai tersisa satu elemen berukuran paling besar di bagian paling ujung kanan.

#### b) Stabilitas (*Stable*)

Pada mekanisme pertukaran jarak jauh di kode ini.

```
if (index_terkecil != i) {  
    std::swap(data[i], data[index_terkecil]);  
    m.swaps++;  
}
```

Karena algoritma ini mencabut paksa nilai terkecil dari indeks di belakang dan menukarnya langsung ke posisi depan dengan melompati elemen-elemen di tengah,



urutan awal elemen sangat rawan bergeser. Jika ada angka bernilai kembar, posisi angka kembar yang awalnya berada di depan bisa tak sengaja terlempar ke urutan belakang akibat proses penukaran paksa ini. Sehingga, Selection Sort terbukti tidak bersifat *stable*.

### c) Penggunaan Memori (In-place)

Proses pengurutan secara langsung memodifikasi tempat di dalam struktur vector data aslinya. Hanya diperlukan satu memori tambahan berskala sangat kecil, yaitu variabel penanda `int index_terkecil`. Karena tidak ada pembuatan array duplikat baru yang memakan ruang, kompleksitas memorinya sangat efisien di  $O(1)$ . Sehingga, algoritma ini termasuk *in-place sorting*.

## 2. Kompleksitas Waktu

### a) Best Case

```
for (int i = 0; i < n - 1; i++) {  
    ...  
    for (int j = i + 1; j < n; j++) {  
        }  
    }  
}
```

Dalam algoritma ini sama sekali tidak ada kondisi pengecekan otomatis untuk menghentikan perulangan jika data sudah terurut. Oleh karena itu, walaupun menerima input data yang sudah sangat rapi (Sorted), perulangan dalam akan tetap dieksekusi secara buta untuk mencari nilai terkecil di sisa elemen. Karena pencarian berjalannya selalu penuh tanpa `break`, kompleksitas waktu terbaiknya terpuruk dan tertahan di  $O(N^2)$ .

### b) Average Case

Ketika dihadapkan pada input data Random, program melakukan hal yang persis sama. Program mengasumsikan indeks saat itu sebagai tebakan nilai terkecil lewat `int index_terkecil = i;`, lalu menelusuri semua sisa elemen di sebelah kanannya satu per satu. Karena perulangan luar dan dalam berjalan proporsional membandingkan elemen yang letaknya acak, kompleksitas rata-ratanya tetap  $O(N^2)$ .

### c) Worst Case

Kondisi saat memproses data terurut terbalik (Reverse Sorted) juga tidak mengubah mekanisme komputasi. Program tetap membandingkan setiap elemen dengan `index_terkecil` secara konstan dari ujung ke ujung. Karena perulangan luarnya

dieksekusi sebanyak  $N-1$  kali dan perulangan dalamnya merayap sampai batas akhir memori tanpa ada ruang untuk berhenti lebih awal, maka kompleksitas terburuknya adalah  $O(N^2)$ .

### 3. Analisis Operasi

#### a) Jumlah comparison

Sesuai rancangan kodenya yang tidak memiliki fitur penghentian, eksekusi `m.comparisons++`; di dalam perulangan selalu berjalan statis. Mau datanya berbentuk *Sorted*, *Random*, ataupun *Reverse Sorted*, jumlah perbandingannya selalu mentok di batas maksimal komputasi, yaitu stabil di angka 4950 kali tanpa bisa dikurangi.

#### b) Jumlah swap

Ini karena perintah `std::swap` ditempatkan di luar batas perulangan dalam, program maksimal hanya melakukan pertukaran satu kali per iterasi. Pada data *Sorted*, tercatat 0 *swap*. Pada kondisi terburuknya (*Reverse Sorted*), metrik *swap*-nya hanya berkisar maksimal di 50 kali saja, menjadikannya jauh lebih ramah memori fisik dibandingkan Bubble Sort.

#### c) Jumlah shift

Tercatat 0 (Tidak ada). Seluruh perpindahan elemen diurus menggunakan fungsi pertukaran dua arah (*swap*), tanpa menggunakan mekanisme pergeseran linear (*shift*).

#### d) Jumlah recursion call

Tercatat 0 (Tidak ada). Karena instruksi pada algoritma dibentuk menggunakan perulangan iteratif bersarang (`for`), tidak terjadi pemanggilan fungsi secara rekursif.

#### e) Jumlah array accessed

Tercatat 0 (Tidak dihitung). Metrik pencatatan hitungan akses memori struktur *array* tidak disertakan dalam fokus evaluasi kelompok algoritma ini.

#### f) Pengaruh bentuk data terhadap jumlah operasi

Algoritma ini terbukti statis dan tidak sensitif terhadap pola distribusi inputnya. Keadaan data yang sudah terurut rapi tidak mampu mengurangi beban eksekusi *comparison* sama sekali. Satu-satunya nilai yang bisa dipengaruhi oleh bentuk data hanyalah operasi *swap*, yang frekuensinya ikut naik jika letak angka minimum semakin berantakan.

#### 4. Analisis Performa

##### a) Perbandingan waktu eksekusi antar algoritma

Pada saat diadu untuk menyortir data dengan sebaran acak berkapasitas masif, Selection Sort sedikit lebih efisien dari Bubble Sort lantaran sukses memangkas banyak siklus operasi *swap*. Walau begitu, algoritma ini sama sekali tidak memiliki peluang jika disandingkan dengan algoritma modern kelompok  $O(N \log N)$  seperti Merge Sort dan Quick Sort.

##### b) Performa pada dataset kecil dan besar

Di rentang *dataset* berukuran mini ( $N=100$ ), kecepatannya sangat responsif dan mampu selesai dalam kisaran 0.009700 ms. Namun kelemahannya langsung terekspos jelas pada saat skala populasi membesar menjadi  $N=10000$ . Sifat kuadratnya membuat program ini mengalami *bottleneck* hingga durasinya membengkak sangat lama mencapai 119.175700 ms.

##### c) Pengaruh distribusi data terhadap performa algoritma

Berbeda dari algoritma lain, algoritma ini menghasilkan selisih durasi yang sangat tipis antar distribusi data. Waktu yang ditempuh saat menyortir data *Sorted*, *Nearly Sorted*, *Random*, ataupun *Reverse Sorted* cenderung stagnan di angka yang lambat, karena *processor* tetap dipaksa menyelesaikan jumlah *looping* yang persis sama.

##### d) Hubungan teori kompleksitas dengan hasil eksperimen

Angka pada metrik *comparison* menjadi bukti terkuat bahwa laju teori lambat  $O(N^2)$  pada algoritma ini murni bersifat mutlak dan tidak bisa dibantu dengan cara apapun, kendati data masukannya sudah terurut sejak awal.

## SOAL 3

### Insertion Sort

#### A. Source Code

*Tabel 21. Source Code Soal 3 Modul 4*

|    |                                                         |
|----|---------------------------------------------------------|
| 52 | // Algoritma Insertion Sort                             |
| 53 | void insertion_sort(std::vector<int>& data, Metrics& m) |
|    | {                                                       |
| 54 | int n = data.size();                                    |
| 55 | auto start = Clock::now();                              |
| 56 |                                                         |
| 57 | for (int i = 1; i < n; i++) {                           |
| 58 | int temp = data[i];                                     |
| 59 | int j = i - 1;                                          |
| 60 |                                                         |
| 61 | while (j >= 0) {                                        |
| 62 | m.comparisons++;                                        |
| 63 | if (data[j] > temp) {                                   |
| 64 | data[j + 1] = data[j];                                  |
| 65 | m.shifts++;                                             |
| 66 | j--;                                                    |
| 67 | } else {                                                |
| 68 | break;                                                  |
| 69 | }                                                       |
| 70 | }                                                       |
| 71 | data[j + 1] = temp;                                     |
| 72 | m.shifts++;                                             |
| 73 | }                                                       |
| 74 |                                                         |

|    |                                                            |
|----|------------------------------------------------------------|
| 75 | <code>m.time_ms = std::chrono::duration&lt;double,</code>  |
|    | <code>std::milli&gt;(Clock::now() - start).count();</code> |
| 76 | <code>}</code>                                             |

## B. Output

```
>> Insertion Sort
Algorithm      Input Type      N      Comparisons  Swaps      Shifts      Rec.Calls  Arr.Access  Time(ms)
-----
Insertion Sort Random      100      2656         0         2663         0           0         0.004800
Insertion Sort Sorted      100        99         0          99         0           0         0.001500
Insertion Sort Reverse Sorted 100      4950         0        5049         0           0         0.006000
Insertion Sort Nearly Sorted 100       395         0         395         0           0         0.001600
Insertion Sort Many Duplicates 100      2412         0        2414         0           0         0.003700
Insertion Sort Random     1000     244181         0     244189         0           0         0.233900
Insertion Sort Sorted     1000       999         0         999         0           0         0.007400
Insertion Sort Reverse Sorted 1000    499500         0    500499         0           0         0.576300
Insertion Sort Nearly Sorted 1000     26775         0      26777         0           0         0.039700
Insertion Sort Many Duplicates 1000    241801         0     241807         0           0         0.228200
Insertion Sort Random     10000   25128135         0   25128147         0           0         27.035700
Insertion Sort Sorted     10000     9999         0       9999         0           0         0.072000
Insertion Sort Reverse Sorted 10000  49995000         0  50004999         0           0         48.836100
Insertion Sort Nearly Sorted 10000   3129351         0   3129351         0           0         2.922500
Insertion Sort Many Duplicates 10000  25103078         0   25103085         0           0         28.619300
```

Gambar 9. Screenshot Hasil Jawaban Soal 3 Modul 4

## C. Pembahasan

### 1. Karakteristik Algoritma

#### a) Mekanisme Utama

Pertama, melakukan inisiasi penelusuran luar mulai dari indeks ke-1 sampai indeks terakhir. Karena yang diinginkan adalah mencari posisi yang pas di area kiri yang sudah urut, maka ambil nilai elemen saat itu dan simpan ke variabel cadangan `int temp = data[i];`. Kemudian, set indeks bantuan `int j = i - 1;` untuk mengecek elemen di belakangnya. Lalu *traverse* mundur, pakai `while (j >= 0)` untuk membaca nilainya. Jika nilai `data[j]` ternyata lebih besar dari `temp`, maka nilai tersebut digeser selangkah ke kanan dengan `data[j + 1] = data[j];`, lalu metrik *shift* dinaikkan dan `j` dikurangi untuk mundur lagi. Kalau ternyata angkanya lebih kecil atau sama, langsung hentikan dengan `break` untuk memangkas waktu pengecekan. Setelah perulangan dalam itu berakhir, maka otomatis tersisa satu slot memori kosong, lalu cetak nilai cadangannya kembali menggunakan `data[j + 1] = temp;`.

#### b) Stabilitas (*Stable*)

Logika perlindungannya ada pada tanda lebih besar dari (`>`) di operasi perbandingan.

```

if (data[j] > temp) {
    data[j + 1] = data[j];
    m.shifts++;
    j--;
}

```

Program hanya akan menggeser `data[j]` ke kanan jika nilainya benar-benar lebih besar dari `temp`. Jika ia menemui angka yang nilainya kembar, kondisinya menjadi tidak terpenuhi, sehingga program memanggil `break`. Angka yang baru pun dijatuhkan dengan aman di sebelah kanan angka kembarannya tanpa merusak susunan asli. Sehingga, Insertion Sort di sini bersifat *stable*.

### c) Penggunaan Memori (In-place)

Program ini tidak memakai perintah alokasi dinamis ataupun mendeklarasikan *array* tambahan baru. Proses memanipulasi posisi elemen dikerjakan mutlak di dalam array asal hanya dengan menumpang pada satu variabel `int temp`, sehingga kompleksitas ruangnya  $O(1)$ . Sehingga, algoritma ini tergolong *in-place*.

## 2. Kompleksitas Waktu

### a) Best Case

Di algoritma ini, terdapat penambahan perintah `break` di dalam struktur perulangannya.

```

if (data[j] > temp) {
    // ... proses geser
} else {
    break;
}

```

Ketika algoritma memproses input data yang dari awal sudah terurut (Sorted), angka di sebelah kiri dipastikan selalu lebih kecil daripada angka acuan (`temp`). Akibatnya, kondisi `if (data[j] > temp)` akan selalu bernilai `false` dan program akan langsung lari ke blok `else` untuk mengeksekusi `break`. Karena perulangan dalam sama sekali tidak sempat berjalan dan program hanya menelusuri keseluruhan memori satu kali lurus ke arah kanan, maka kompleksitas waktu terbaiknya sangat efisien di angka  $O(N)$ .

### b) Average Case

Algoritma ini berpusat pada perulangan luar `for` dan perulangan dalam `while`. Saat dihadapkan pada kumpulan data *Random*, elemen-elemen berada pada posisi yang tidak menentu. Saat sebuah angka diangkat sebagai `temp`, program membandingkannya ke arah kiri. Beberapa angka akan membuat pergeseran memori, dan beberapa lainnya memicu `break`. Karena perulangan luar dan dalam harus berjalan secara seimbang untuk membersihkan ketidakteraturan tersebut, kompleksitas waktunya berada di  $O(N^2)$ .

### c) Worst Case

Kondisi komputasi paling berat terjadi pada data yang terurut terbalik (*Reverse Sorted*). Pada kondisi ini, elemen terkecil mengumpul di ujung kanan. Setiap kali program menyimpan suatu nilai ke `temp`, ia menemukan bahwa semua elemen di kirinya selalu lebih besar. Karena kondisi `data[j] > temp` tidak pernah bernilai `false`, perintah `break` sama sekali tidak tersentuh. Perulangan `while` dipaksa mundur terus-menerus sampai ke ujung indeks ke-0. Akibat iterasi ganda yang berjalan penuh tanpa henti, maka kompleksitas terburuknya adalah  $O(N^2)$ .

## 3. Analisis Operasi

### a) Jumlah comparison

Keberadaan `break` membuat operasi perbandingan sangat adaptif. Pada *Best Case* (data *Sorted*), eksekusi `m.comparisons++` sangat sedikit dan tertahan di 99 kali saja. Berkebalikan dengan *Worst Case* (data *Reverse Sorted*), pengecekan sampai batas akhir memori sehingga perbandingannya membengkak hebat mencapai 4950 kali.

### b) Jumlah swap

Tercatat 0 (Tidak ada). Kode pada algoritma ini tidak menggunakan perintah `(std::swap())`.

### c) Jumlah shift

Pada data yang sudah rapi (*Sorted*), pencatatan *shift* berada di angka paling rendah yaitu 99 kali karena elemen langsung diletakkan di tempatnya semula tanpa menggeser elemen lain. Sebaliknya, saat dihadapkan pada data *Reverse Sorted*, program sibuk melakukan pemindahan memori renteng ke arah kanan, membuat angka pergeserannya meledak hingga 5049 kali.

### d) Jumlah recursion call

Tercatat 0 (Tidak ada). Karena algoritma ini dibangun menggunakan perulangan iteratif biasa (*for* dan *while*), tidak ada pemanggilan fungsi rekursif sama sekali.

**e) Jumlah array accessed**

Tercatat 0 (Tidak dihitung).

**f) Pengaruh bentuk data terhadap jumlah operasi**

Dibandingkan Bubble maupun Selection Sort, Insertion Sort adalah yang paling responsif dalam merespons bentuk data awal. Jika data sudah berwujud "hampir terurut", eksekusi *shift* dan *comparison* dapat dipangkas dalam jumlah yang masif karena perintah *break* sangat sering tereksekusi.

#### **4. Analisis Performa**

**a) Perbandingan waktu eksekusi antar algoritma**

Dalam pengurutan data *Random* berkapasitas besar, performanya cenderung setara dengan algoritma  $O(N^2)$  lainnya dan sama sekali tidak bisa mengimbangi laju algoritma kelompok *Divide and Conquer* (Merge dan Quick Sort). Akan tetapi, pada keadaan data yang sudah hampir urut, Insertion Sort mampu menjadi salah satu yang tercepat bahkan melebihi algoritma tersebut karena tidak memiliki *overhead* fungsi.

**b) Performa pada dataset kecil dan besar**

Pada uji coba set data kecil ( $N=100$ ), algoritma merespons dengan waktu eksekusi singkat yaitu 0.001500 ms. Namun, efisiensinya hancur ketika ukuran data membesar menjadi ( $N=10000$ ). Sifat kuadratnya membuat program tersendat dan memakan waktu hingga 0.0072000 ms.

**c) Pengaruh distribusi data terhadap performa algoritma**

Durasi pada pembacaan terminal dikendalikan oleh total eksekusi *shift*. Waktu optimal didapat oleh distribusi data *Sorted*, diikuti *Nearly Sorted*, kemudian merosot pada data *Random*. Posisi dengan waktu terlama secara konsisten selalu diraih oleh distribusi data *Reverse Sorted*.

**d) Hubungan teori kompleksitas dengan hasil eksperimen**

Mengacu pada rumusan teori *Worst Case*  $O(N^2)$ , ketika  $N$  dinaikkan 10 kali lipat (dari ukuran 1000 ke 10000), waktu komputasinya terbukti ikut meledak menjadi puluhan hingga mendekati seratus kali lipat lebih lambat pada data acak dan terbalik. Namun, keunikan algoritma ini terletak pada teori *Best Case*  $O(N)$  yang juga berhasil tervalidasi.



Berkat adanya fungsi *early exit* (break), pembengkakan waktu tersebut sama sekali tidak terjadi pada distribusi data *Sorted*.

## SOAL 4

### Merge Sort

#### A. Source Code

Tabel 22. Source Code Soal 4 Modul 4

|     |                                                                                             |
|-----|---------------------------------------------------------------------------------------------|
| 79  | // Helper Merge Sort                                                                        |
| 80  | void proses_merge(std::vector<int>& data, int kiri, int<br>tengah, int kanan, Metrics& m) { |
| 81  | int size_kiri = tengah - kiri + 1;                                                          |
| 82  | int size_kanan = kanan - tengah;                                                            |
| 83  |                                                                                             |
| 84  | std::vector<int> arr_kiri(size_kiri);                                                       |
| 85  | std::vector<int> arr_kanan(size_kanan);                                                     |
| 86  |                                                                                             |
| 87  | for (int i = 0; i < size_kiri; i++) arr_kiri[i] =<br>data[kiri + i];                        |
| 88  | for (int j = 0; j < size_kanan; j++) arr_kanan[j] =<br>data[tengah + 1 + j];                |
| 89  |                                                                                             |
| 90  | int i = 0, j = 0, k = kiri;                                                                 |
| 91  | while (i < size_kiri && j < size_kanan) {                                                   |
| 92  | m.comparisons++;                                                                            |
| 93  | if (arr_kiri[i] <= arr_kanan[j]) {                                                          |
| 94  | data[k] = arr_kiri[i];                                                                      |
| 95  | i++;                                                                                        |
| 96  | } else {                                                                                    |
| 97  | data[k] = arr_kanan[j];                                                                     |
| 98  | j++;                                                                                        |
| 99  | }                                                                                           |
| 100 | k++;                                                                                        |

```

101     }
102
103     while (i < size_kiri) {
104         data[k] = arr_kiri[i];
105         i++; k++;
106     }
107     while (j < size_kanan) {
108         data[k] = arr_kanan[j];
109         j++; k++;
110     }
111 }
112
113 // Helper Merge Sort
114 void rekursif_merge(std::vector<int>& data, int kiri,
115 int kanan, Metrics& m) {
116     m.recursive_calls++;
117     if (kiri < kanan) {
118         int tengah = kiri + (kanan - kiri) / 2;
119         rekursif_merge(data, kiri, tengah, m);
120         rekursif_merge(data, tengah + 1, kanan, m);
121         proses_merge(data, kiri, tengah, kanan, m);
122     }
123 }
124
125 // Algoritma Merge Sort
126 void merge_sort(std::vector<int>& data, Metrics& m) {
127     auto start = Clock::now();
128
129     if (data.size() > 1) {
130         rekursif_merge(data, 0, data.size() - 1, m);
131     }

```

```

131
132     m.time_ms      =      std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
133 }

```

## B. Output

| Algorithm  | Input Type      | N     | Comparisons | Swaps | Shifts | Rec.Calls | Arr.Access | Time(ms) |
|------------|-----------------|-------|-------------|-------|--------|-----------|------------|----------|
| Merge Sort | Random          | 100   | 535         | 0     | 0      | 199       | 0          | 0.057400 |
| Merge Sort | Sorted          | 100   | 356         | 0     | 0      | 199       | 0          | 0.046400 |
| Merge Sort | Reverse Sorted  | 100   | 316         | 0     | 0      | 199       | 0          | 0.042300 |
| Merge Sort | Nearly Sorted   | 100   | 453         | 0     | 0      | 199       | 0          | 0.037400 |
| Merge Sort | Many Duplicates | 100   | 536         | 0     | 0      | 199       | 0          | 0.050400 |
| Merge Sort | Random          | 1000  | 8690        | 0     | 0      | 1999      | 0          | 0.451600 |
| Merge Sort | Sorted          | 1000  | 5044        | 0     | 0      | 1999      | 0          | 0.235400 |
| Merge Sort | Reverse Sorted  | 1000  | 4932        | 0     | 0      | 1999      | 0          | 0.226400 |
| Merge Sort | Nearly Sorted   | 1000  | 7628        | 0     | 0      | 1999      | 0          | 0.292800 |
| Merge Sort | Many Duplicates | 1000  | 8679        | 0     | 0      | 1999      | 0          | 0.318500 |
| Merge Sort | Random          | 10000 | 120465      | 0     | 0      | 19999     | 0          | 2.821300 |
| Merge Sort | Sorted          | 10000 | 69008       | 0     | 0      | 19999     | 0          | 3.745800 |
| Merge Sort | Reverse Sorted  | 10000 | 64608       | 0     | 0      | 19999     | 0          | 2.175800 |
| Merge Sort | Nearly Sorted   | 10000 | 111524      | 0     | 0      | 19999     | 0          | 2.469300 |
| Merge Sort | Many Duplicates | 10000 | 120461      | 0     | 0      | 19999     | 0          | 2.786200 |

Gambar 10. Screenshot Hasil Jawaban Soal 4 Modul 4

## C. Pembahasan

### 1. Karakteristik Algoritma

#### a) Mekanisme Utama

Pertama, melakukan pemanggilan fungsi pemecah rekursif `merge`. Karena yang diinginkan adalah memecah *array* besar menjadi kepingan kecil, maka pakai pendekatan rekursif untuk terus mencari indeks tengah dan membelah *array* menjadi dua kubu. Pemanggilan rekursif ini dilakukan hingga batas kiri < kanan tidak terpenuhi lagi (tersisa satu node elemen). Lalu, kembali ke atas untuk menyatukan node tersebut, kemudian panggil `proses_merge()`. Untuk menyatukannya, pertama program harus mencetak dua *array* tempelan `arr_kiri` dan `arr_kanan` untuk menyimpan pecahan datanya. Kemudian dieksekusi perulangan `while` untuk mengecek elemen di ujung kedua *array* pecahan itu. Jika nilai di kiri lebih kecil atau sama, dimasukkan ke *array* utama, sebaliknya jika nilai kanan lebih kecil, maka nilai kanannya yang masuk. Hal ini diulang terus sambil meningkatkan nilai indeks kiri atau kanan

sampai semua pecahan terkecil selesai disatukan kembali menjadi satu *array* yang terurut.

#### **b) Stabilitas (*Stable*)**

Logika perlindungannya ada di dalam fungsi penggabungan `proses_merge` pada baris perbandingan.

```
if (arr_kiri[i] <= arr_kanan[j]) {  
    data[k] = arr_kiri[i];  
    i++;  
}
```

Karena menggunakan operator kurang dari atau sama dengan (`<=`), program memberikan prioritas mutlak kepada elemen yang berasal dari *array* pecahan kiri (`arr_kiri`). Jika ada dua angka yang nilainya kembar dari dua cabang yang berbeda, angka yang aslinya memang berada di urutan lebih kiri akan selalu disalin masuk ke *array* utama lebih dulu. Hasilnya, urutan elemen kembar tersebut tidak akan pernah terbalik.. Sehingga, Merge Sort di sini bersifat *stable*.

#### **c) Penggunaan Memori (*In-place*)**

ada awal fungsi `proses_merge`.

```
std::vector<int> arr_kiri(size_kiri);  
std::vector<int> arr_kanan(size_kanan);
```

Setiap kali fungsi penggabungan dipanggil, program dipaksa untuk menciptakan dua buah penampung vector baru di dalam memori untuk menyimpan data sementara. Hal ini menyebabkan algoritma membutuhkan alokasi memori tambahan yang besarnya proporsional dengan panjang data awal, sehingga kompleksitas ruangnya/memorinya membengkak menjadi  $O(N)$ . Sehingga, algoritma ini tidak tergolong *in-place*.

## **2. Kompleksitas Waktu**

#### **a) Best Case**

Algoritma ini dibuat menggunakan metode *Divide and Conquer*. Tidak ada fungsi pendeteksi untuk mengecek apakah data sudah rapi atau belum..

```
if (kiri < kanan) {  
    int tengah = kiri + (kanan - kiri) / 2;  
    rekursif_merge(data, kiri, tengah, m);
```

```
    ...  
}
```

Meskipun menerima input data yang sejak awal sudah terurut sempurna (*Sorted*), program akan tetap membelah memori tersebut tepat di tengah secara terus-menerus sampai ukurannya tersisa satu elemen tunggal. Setelah itu, barulah fungsi penggabungan dipanggil kembali ke atas. Karena proses pembelahan pohon memori (*divide*) selalu memakan  $\log N$  langkah dan penggabungan elemennya (*conquer*) memakan  $N$  langkah, maka disimpulkan bahwa waktu komputasi terbaiknya akan selalu statis di  $O(N \log N)$ .

#### **b) Average Case**

Saat diberikan masukan berupa data dengan sebaran *Random*, program meresponsnya dengan cara yang persis sama. Fungsi rekursif tidak mempedulikan isi dari elemennya, ia murni hanya melihat batas indeks *kiri* dan *kanan*. Data acak tersebut tetap akan dibelah dua secara simetris, lalu elemen-elemennya dibandingkan satu per satu saat disatukan kembali di memori utama. Akibat langkah komputasi pembagian dan penggabungan linear yang dijalankan penuh, kompleksitas rata-ratanya juga bernilai  $O(N \log N)$ .

#### **c) Worst Case**

Kondisi memproses himpunan data terurut terbalik (*Reverse Sorted*) tidak memberikan ancaman berarti untuk algoritma ini. Berbeda dengan *Quick Sort* yang bisa hancur ketika salah memilih *pivot*, *Merge Sort* selalu konsisten membelah memori dari nilai tengah mutlak  $(\text{kiri} + \text{kanan}) / 2$ . Proses penggabungannya pun tidak memicu lonjakan iterasi berlebih karena pengecekan elemen dilakukan linear. Oleh sebab itu, kompleksitas waktu di skenario paling buruk sekalipun tetap di  $O(N \log N)$ .

### **3. Analisis Operasi**

#### **a) Jumlah comparison**

Pada distribusi (*Sorted* dan *Reverse Sorted*), metrik ini berada di rentang angka yang lebih rendah, yaitu 356-315 kali. Perbedaan jumlah perbandingannya tidak terlalu jauh antar distribusi datanya.

**b) Jumlah swap**

Tercatat mutlak 0 (Tidak ada). Pemindahan elemen ke *array* utama menggunakan skema penugasan penempatan indeks secara langsung (`data[k] = arr_kiri[i];`), bukan operasi pertukaran `std::swap()`.

**c) Jumlah shift**

Tercatat 0 (Tidak ada). Tidak ada pemindahan elemen seperti yang ada pada Insertion Sort.

**d) Jumlah recursion call**

Karena skema pembelahannya yang selalu terbagi dua secara merata terlepas dari bentuk datanya, jumlah pemanggilan fungsinya sangat statis. Baik dihadapkan pada data *Sorted*, maupun data *Reverse Sorted*, nilai `m_recursive_calls` tidak bergeser dari angkav 199 kali.

**e) Jumlah array accessed**

Tercatat 0 (Tidak dihitung). Hitungan akses struktur *array* ditiadakan.

**f) Pengaruh bentuk data terhadap jumlah operasi**

Algoritma kelas *Divide and Conquer* ini sangat kebal terhadap pengaruh bentuk distribusi awal data. Karena struktur pohon rekursifnya tidak berubah, bentuk data (sudah urut atau berantakan) sama sekali tidak mampu mempengaruhi jumlah panggilan rekursinya dan hanya sedikit saja memberikan imbas pada jumlah perbandingan di tahap *merge*.

#### **4. Analisis Performa**

**a) Perbandingan waktu eksekusi antar algoritma**

Pada saat diuji *dataset* berukuran masif dengan distribusi yang acak, Merge Sort terbukti berada di tingkat superioritas yang berbeda. Ia membabat habis waktu eksekusi jauh di bawah durasi seluruh algoritma  $O(N^2)$  (Bubble, Selection, Insertion) karena sifat eksponensial pemecahan komputasinya.

**b) Performa pada dataset kecil dan besar**

Di ukuran *dataset* yang sangat minim ( $N=100$ ), algoritma ini justru terlihat agak tertinggal dengan waktu 0.046400 ms, kalah responsif dibanding Insertion Sort karena adanya beban berat dari *overhead* pemanggilan sistem rekursi. Akan tetapi, dominasinya

terlihat ketika beban data diset menjadi sangat masif ( $N=10000$ ), di mana ia tetap stabil mencatatkan waktu super cepat di kisaran 3.745800 ms.

**c) Pengaruh distribusi data terhadap performa algoritma**

Durasi hasil akhir eksekusi antar distribusi data memperlihatkan waktu yang sangat stagnan. Baik ketika merapikan data yang masih *Random*, maupun data yang terbalik (*Reverse Sorted*), waktu pemrosesannya berdekatan secara merata, menjadikannya algoritma paling tangguh secara konsistensi durasi.

**d) Hubungan teori kompleksitas dengan hasil eksperimen**

Hasil pemantauan menyajikan bukti dari teori Big O Notation untuk  $O(N \log N)$ . Konsistensi angka statis pada metrik *recursion calls* memberikan fakta bahwa struktur algoritma ini dipastikan selalu identik di setiap skenario. Pertumbuhan beban eksekusi waktu dari angka  $N$  yang kecil menuju angka masif juga terbukti tumbuh logaritmik dan tidak meledak menjadi eksponensial.



## SOAL 5

### Quick Sort

#### A. Source Code

*Tabel 23. Source Code Soal 5 Modul 4*

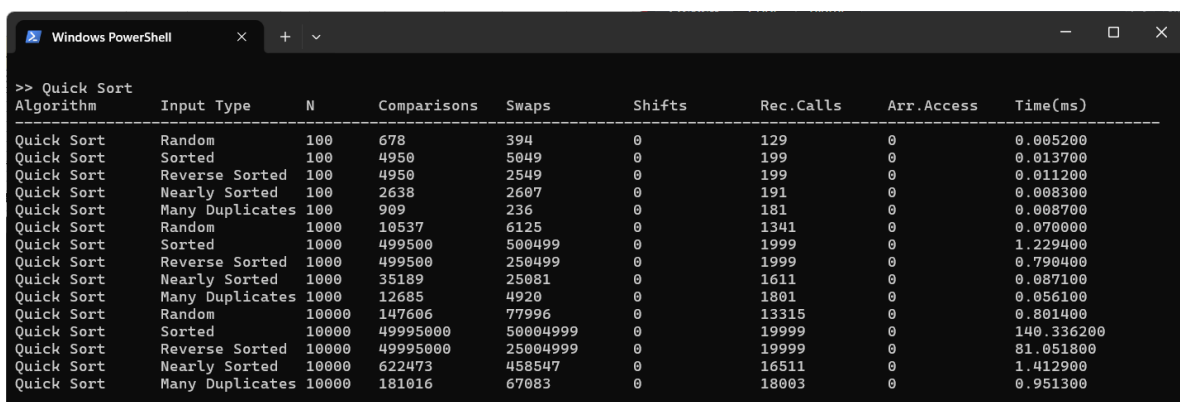
|     |                                                                              |
|-----|------------------------------------------------------------------------------|
| 136 | // Helper Quick Sort                                                         |
| 137 | int partisi(std::vector<int>& data, int low, int high, Metrics& m) {         |
| 138 | int pivot = data[high];                                                      |
| 139 | int i = (low - 1);                                                           |
| 140 |                                                                              |
| 141 | for (int j = low; j < high; j++) {                                           |
| 142 | m.comparisons++;                                                             |
| 143 | if (data[j] < pivot) {                                                       |
| 144 | i++;                                                                         |
| 145 | std::swap(data[i], data[j]);                                                 |
| 146 | m.swaps++;                                                                   |
| 147 | }                                                                            |
| 148 | }                                                                            |
| 149 | std::swap(data[i + 1], data[high]);                                          |
| 150 | m.swaps++;                                                                   |
| 151 | return (i + 1);                                                              |
| 152 | }                                                                            |
| 153 |                                                                              |
| 154 | // Helper Quick Sort                                                         |
| 155 | void rekursif_quick(std::vector<int>& data, int low, int high, Metrics& m) { |
| 156 | m.recursive_calls++;                                                         |
| 157 | if (low < high) {                                                            |
| 158 | int titik_partisi = partisi(data, low, high, m);                             |

```

159     rekursif_quick(data, low, titik_partisi - 1, m);
160     rekursif_quick(data, titik_partisi + 1, high,
161 m);
162     }
163 }
164 // Algoritma Quick Sort
165 void quick_sort(std::vector<int>& data, Metrics& m) {
166     auto start = Clock::now();
167
168     if (data.size() > 1) {
169         rekursif_quick(data, 0, data.size() - 1, m);
170     }
171
172     m.time_ms = std::chrono::duration<double,
173 std::milli>(Clock::now() - start).count();
174 }

```

## B. Output



| Algorithm  | Input Type      | N     | Comparisons | Swaps    | Shifts | Rec.Calls | Arr.Access | Time(ms)   |
|------------|-----------------|-------|-------------|----------|--------|-----------|------------|------------|
| Quick Sort | Random          | 100   | 678         | 394      | 0      | 129       | 0          | 0.005200   |
| Quick Sort | Sorted          | 100   | 4950        | 5049     | 0      | 199       | 0          | 0.013700   |
| Quick Sort | Reverse Sorted  | 100   | 4950        | 2549     | 0      | 199       | 0          | 0.011200   |
| Quick Sort | Nearly Sorted   | 100   | 2638        | 2607     | 0      | 191       | 0          | 0.008300   |
| Quick Sort | Many Duplicates | 100   | 909         | 236      | 0      | 181       | 0          | 0.008700   |
| Quick Sort | Random          | 1000  | 10537       | 6125     | 0      | 1341      | 0          | 0.070000   |
| Quick Sort | Sorted          | 1000  | 499500      | 500499   | 0      | 1999      | 0          | 1.229400   |
| Quick Sort | Reverse Sorted  | 1000  | 499500      | 250499   | 0      | 1999      | 0          | 0.790400   |
| Quick Sort | Nearly Sorted   | 1000  | 35189       | 25081    | 0      | 1611      | 0          | 0.087100   |
| Quick Sort | Many Duplicates | 1000  | 12685       | 4920     | 0      | 1801      | 0          | 0.056100   |
| Quick Sort | Random          | 10000 | 147606      | 77996    | 0      | 13315     | 0          | 0.801400   |
| Quick Sort | Sorted          | 10000 | 49995000    | 50004999 | 0      | 19999     | 0          | 140.336200 |
| Quick Sort | Reverse Sorted  | 10000 | 49995000    | 25004999 | 0      | 19999     | 0          | 81.051800  |
| Quick Sort | Nearly Sorted   | 10000 | 622473      | 458547   | 0      | 16511     | 0          | 1.412900   |
| Quick Sort | Many Duplicates | 10000 | 181016      | 67083    | 0      | 18003     | 0          | 0.951300   |

Gambar 11. Screenshot Hasil Jawaban Soal 5 Modul 4

## C. Pembahasan

### 1. Karakteristik Algoritma

#### a) Mekanisme Utama

Pertama, melakukan pemanggilan fungsi utama yang memicu `rekursif_quick`. Karena yang diinginkan adalah memecah urutan berdasarkan satu angka sentral, maka pakai mekanisme pemanggilan `partisi()` yang akan mengunci satu elemen di ujung paling kanan sebagai pusat acuan (`int pivot = data[atas]`). Kemudian, disiapkan indeks bantuan `i` untuk mencatat batas area angka-angka kecil. Untuk memilah elemen sisanya, dipakai perulangan `for` dari kiri ke kanan. Lalu, kemudian baca nilainya. Jika nilai `data[j]` ternyata nilainya kurang dari `pivot`, batas area kecilnya (`i`) diperlebar, lalu nilai tersebut langsung dilempar masuk ke dalam area kecil itu pakai `std::swap()`. Kalau tidak lebih kecil, lewati saja. Setelah penelusuran data berakhir, maka tersisa satu langkah wajib, pindahkan nilai `pivot` yang masih di ujung tadi ke tepat di sebelah kanan indeks `i` (titik pemisah), lalu kembalikan posisi indeks pemisah tersebut. Hal ini diulang terus melalui pemanggilan rekursif untuk sisi kiri dan sisi kanan dari titik pemisah sampai habis tak tersisa.

#### b) Stabilitas (*Stable*)

Pada eksekusi pertukaran di dalam perulangan `for` saat proses partisi berlangsung.

```
if (data[j] < pivot) {  
    i++;  
    std::swap(data[i], data[j]);  
    m.swaps++;  
}
```

Mekanisme ini menukar (`swap`) nilai yang terdeteksi lebih kecil dari *pivot* ke area depan dengan melompati rentetan angka di tengah-tengahnya. Akibat proses pencabutan paksa dan pelompatan memori ini, stabilitas menjadi hancur. Elemen bernilai kembar yang tadinya memiliki posisi berurutan bisa saling mendahului karena dilempar oleh operasi pertukaran ini. Sehingga, Quick Sort di sini bersifat tidak *stable*.

### c) Penggunaan Memori (In-place)

Alur komputasinya secara langsung menggeser dan menukar data langsung di dalam batas vector data asalnya. Kode ini tidak mendeklarasikan dan mengalokasikan memori array bantuan sama sekali, sehingga algoritma ini teratas dari sisi efisiensi kompleksitas ruang memori di tingkat  $O(1)$ . Sehingga, algoritma ini tergolong *in-place*.

## 2. Kompleksitas Waktu

### a) Best Case

Dalam implementasi kodenya, algoritma Quick sort ini bekerja dengan memilih satu elemen sebagai titik pusat dan membagi sisanya menjadi dua bagian.

```
int titik_partisi = partisi(data, low, high, m);  
rekursif_quick(data, low, titik_partisi - 1, m);  
rekursif_quick(data, titik_partisi + 1, high, m);
```

Ketika dihadapkan pada skenario data yang bentuk atau acakannya memungkinkan fungsi `partisi` secara kebetulan selalu memilih nilai tengah (median) sebagai acuan (pivot), array akan terus terbelah menjadi dua bagian yang persis sama besar. Pembelahan memori yang seimbang sempurna ini hanya membutuhkan rekursi sebesar  $\log N$ . Karena setiap tingkat rekursi harus menyusuri  $N$  elemen untuk perbandingan, maka terbukti bahwa komputasi terbaiknya dapat dicapai di level  $O(N \log N)$ .

### b) Average Case

Pada data Random, pemilihan elemen paling ujung kanan sebagai pivot (`int pivot = data[high];`) akan membelah array ke dalam proporsi yang cukup seimbang, meski tidak sempurna di tengah. Kedalaman pohon rekursinya masih tetap terjaga di rentang logaritmik. Oleh karena itu, pada himpunan data rata-rata, kompleksitas waktunya tetap berjalan sangat cepat pada  $O(N \log N)$ .

### c) Worst Case

Kelemahan kode ini terlihat saat memproses data yang dari awal sudah terurut rapi (Sorted) maupun terurut terbalik (Reverse Sorted). Karena titik pusat selalu dikunci secara statis pada indeks paling kanan (`pivot = data[high]`), maka nilai yang terpilih pastilah nilai yang paling ekstrem (paling besar atau paling kecil). Hal ini membuat fungsi `partisi` gagal membelah data, ia malah memisahkan 0 elemen di satu sisi dan  $N-1$  elemen di sisi lainnya. Akibatnya, pemanggilan `rekursif_quick`

membengkak seolah menjadi perulangan biasa yang menyusuri indeks satu per satu dari ujung ke ujung. Dari alur ini, terbukti bahwa kompleksitas terburuknya langsung merosot menjadi  $O(N^2)$ .

### 3. Analisis Operasi

#### a) Jumlah comparison

Pada input data *Random*, angka `m.comparisons++` di dalam iterasi berjalan sangat minim dan efisien di kisaran 678 kali berkat pemecahan partisi. Sebaliknya, saat diuji dengan data *Sorted* / *Reverse Sorted*, metrik perbandingannya meledak, yakni menyentuh 4950 kali komparasi.

#### b) Jumlah swap

Karena program selalu mengunci elemen paling kanan sebagai *pivot*, jumlah *swap* justru meledak pada data yang sejak awal sudah terurut (*Sorted*). Pada data terurut, setiap elemen selalu terdeteksi lebih kecil dari *pivot* di ujungnya, sehingga kondisi `if (data[j] < pivot)` selalu terpenuhi dan memicu pertukaran secara maksimal hingga 5049 kali. Pada data *Reverse Sorted*, kondisinya berselang-seling di setiap partisi sehingga jumlah *swap*-nya terpotong menjadi separuh, yaitu 2549 kali. Sebaliknya, efisiensi penukaran terlihat saat algoritma memproses data *Random*, di mana ia hanya membutuhkan 394 kali *swap* karena *array* terbelah secara proporsional..

#### c) Jumlah shift

Tercatat mutlak 0 (Tidak ada). Quick Sort mengelola pemindahan memori tanpa menggunakan penyimpanan penggeseran elemen (*assignment shift*).

#### d) Jumlah recursion call

Di kondisi *Random*, pemanggilan ke fungsi dirinya sendiri terekam sangat efisien di angka 129 kali. Tapi, ketika dipaksa mengurutkan memori yang *Sorted* atau *Reverse*, fungsi ini mengalami krisis komputasi dan memanggil dirinya sendiri secara brutal hingga maksimum sebesar 199 kali panggilan tanpa henti.

#### e) Jumlah array accessed

Tercatat 0 (Tidak dihitung).

#### f) Pengaruh bentuk data terhadap jumlah operasi

Sensitivitas Quick Sort terhadap derajat distribusi input data adalah yang paling ekstrim di antara seluruh algoritma yang diuji. Bentuk penyebaran angka di dalam *array*

akan secara langsung menentukan apakah algoritma ini akan berjalan efisien seperti Merge Sort atau hancur lebur menjadi sekilas Bubble Sort karena pemilihan *pivot* statis yang tidak disengaja.

#### 4. Analisis Performa

##### a) Perbandingan waktu eksekusi antar algoritma

Dalam pertandingan komputasi di data ukuran masif *Random*, algoritma ini merupakan yang tercepat. Quick Sort seringkali menghasilkan waktu eksekusi yang sedikit lebih kencang bahkan dari Merge Sort, dikarenakan efisiensi *in-place* miliknya yang meniadakan tugas *processor* untuk memindah-mindahkan variabel *vector* di luar *array* utama.

##### b) Performa pada dataset kecil dan besar

Di ukuran kecil ( $N=100$ ), performanya nyaris tak terlihat dan selesai seketika di durasi 0.005200 ms. Efisiensi algoritma ini semakin terlihat ketika ukuran data dikalikan masif ( $N=10000$ ), di mana pada kumpulan nilai random ia bisa menyelesaikan dengan waktu fantastis yang hanya berkisar 0.801400 ms saja.

##### c) Pengaruh distribusi data terhadap performa algoritma

Durasi eksekusi pada algoritma ini berbanding lurus dengan banyak fungsi rekursi dipanggil. Saat memproses kumpulan data *random*, pemilihan *pivot* bisa membelah *array* dengan cukup seimbang, sehingga catatan waktunya sangat efisien dan menjadi yang tercepat. Sebaliknya, saat program diberikan data *Sorted* ataupun *Reverse Sorted*, *pivot* yang selalu dipatok dari indeks paling kanan ( $\text{data}[\text{high}]$ ) membuat pembelahan memori selalu berat sebelah. Akibat fungsi partisi yang memburuk, waktu eksekusi pada kedua distribusi data ini merosot dan menjadi sangat lambat.

##### d) Hubungan teori kompleksitas dengan hasil eksperimen

Catatan metrik eksekusi berhasil mengonfirmasi validasi rumusan Big O Notation. Ledakan pada metrik *recursion calls* saat data bernilai *Sorted* menjadi pembuktian di terminal uji coba, bahwa skenario terburuk  $O(N^2)$  pada algoritma Quick Sort tanpa adanya perbaikan penentuan *pivot* adalah benar.

## SOAL 6

### Radix Sort

#### A. Source Code

Tabel 24. Source Code Soal 6 Modul 4

|     |                                                     |
|-----|-----------------------------------------------------|
| 176 | // Helper Radix Sort (Counting Sort)                |
| 177 | void counting_sort(std::vector<int>& data, int exp, |
|     | Metrics& m) {                                       |
| 178 | int n = data.size();                                |
| 179 | std::vector<int> output(n);                         |
| 180 | int count[10] = {0};                                |
| 181 |                                                     |
| 182 | for (int i = 0; i < n; i++) {                       |
| 183 | int digit = (data[i] / exp) % 10;                   |
| 184 | count[digit]++;                                     |
| 185 | m.array_accesses++;                                 |
| 186 | }                                                   |
| 187 |                                                     |
| 188 | for (int i = 1; i < 10; i++) {                      |
| 189 | count[i] += count[i - 1];                           |
| 190 | m.array_accesses += 2;                              |
| 191 | }                                                   |
| 192 |                                                     |
| 193 | for (int i = n - 1; i >= 0; i--) {                  |
| 194 | int digit = (data[i] / exp) % 10;                   |
| 195 | output[count[digit] - 1] = data[i];                 |
| 196 | count[digit]--;                                     |
| 197 | m.array_accesses += 3;                              |
| 198 | }                                                   |
| 199 |                                                     |

```

200     for (int i = 0; i < n; i++) {
201         data[i] = output[i];
202         m.array_accesses++;
203     }
204 }
205
206 // Algoritma Radix Sort
207 void radix_sort(std::vector<int>& data, Metrics& m) {
208     if (data.empty()) return;
209     auto start = Clock::now();
210
211     int angka_max = data[0];
212     for (size_t i = 1; i < data.size(); i++) {
213         if (data[i] > angka_max) {
214             angka_max = data[i];
215         }
216     }
217
218     for (int exp = 1; angka_max / exp > 0; exp *= 10) {
219         counting_sort(data, exp, m);
220     }
221
222     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
223 }

```



## B. Output

```
>> Radix Sort
Algorithm      Input Type      N      Comparisons  Swaps      Shifts      Rec.Calls  Arr.Access  Time(ms)
-----
Radix Sort     Random          100      0             0           0           0         1554        0.005800
Radix Sort     Sorted           100      0             0           0           0         1554        0.005400
Radix Sort     Reverse Sorted   100      0             0           0           0         1554        0.005000
Radix Sort     Nearly Sorted    100      0             0           0           0         1554        0.005300
Radix Sort     Many Duplicates 100      0             0           0           0         1036        0.003600
Radix Sort     Random          1000     0             0           0           0         20072       0.085300
Radix Sort     Sorted           1000     0             0           0           0         20072       0.040100
Radix Sort     Reverse Sorted   1000     0             0           0           0         20072       0.040100
Radix Sort     Nearly Sorted    1000     0             0           0           0         20072       0.040400
Radix Sort     Many Duplicates 1000     0             0           0           0         15054       0.029600
Radix Sort     Random          10000    0             0           0           0         250090      0.552100
Radix Sort     Sorted           10000    0             0           0           0         250090      0.496500
Radix Sort     Reverse Sorted   10000    0             0           0           0         250090      0.498400
Radix Sort     Nearly Sorted    10000    0             0           0           0         250090      0.496400
Radix Sort     Many Duplicates 10000    0             0           0           0         200072      0.380300

Done.
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\sorting> |
```

Gambar 12. Screenshot Hasil Jawaban Soal 6 Modul 4

## C. Pembahasan

### 1. Karakteristik Algoritma

#### a) Mekanisme Utama

Pertama, melakukan inisiasi pencarian nilai paling maksimum di dalam data sebagai acuan batas perulangan. Karena yang diinginkan adalah membedah dari digit paling belakang, maka disiapkan variabel multiplier `int exp = 1;`. Lalu perulangan utama traverse berjalan memanggil fungsi `counting_sort`. Di dalam fungsi tersebut, setiap angka diurai digitnya pakai operator modulo  $(data[i] / exp) \% 10$ . Digit yang didapat ini dijadikan indeks untuk menambahkan nilai di wadah `count[]`. Setelah frekuensi digit terhitung, diubah menjadi nilai kumulatif agar program tahu persis indeks elemen tersebut harus diletakkan di mana. Kemudian dilakukan perulangan mundur, menjatuhkan nilai elemen ke indeks yang tepat di vector `output[]`. Terakhir, salin kembali seluruh isi `output` ke dalam array `data` yang asli. Perulangan di fungsi utama dikalikan 10 (`exp *= 10`) untuk menggeser ke nilai puluhan, ratusan, seterusnya, sampai nilai `angka_max` habis terbagi.

#### b) Stabilitas (*Stable*)

Logika menjaga stabilitasnya berada di bagian akhir fungsi helpernya, yaitu

```

for (int i = n - 1; i >= 0; i--) {
    int digit = (data[i] / exp) % 10;
    output[count[digit] - 1] = data[i];
    count[digit]--;
    ...
}

```

Karena perulangan untuk menempatkan angka ke wadah `output` dijalankan mundur ( $i = n - 1; i \geq 0$ ), elemen kembar yang posisi awalnya berada di sebelah kanan akan diletakkan lebih dulu di indeks paling belakang dalam wadah *array*-nya. Hal ini akan menjamin bahwa angka yang masuk belakangan posisinya tidak akan menimpa dan membalik urutan angka kembar yang ada di depannya. Tanpa ini, hasil pengurutan digit puluhan akan merusak hasil pengurutan digit satuan sebelumnya.. Sehingga, Radix Sort di sini bersifat *stable*.

### c) Penggunaan Memori (In-place)

Di fungsi `counting_sort`.

```

std::vector<int> output(n);
int count[10] = {0};

```

Untuk mengelompokkan elemen sesuai urutan digitnya, program mendeklarasikan array besar baru bernama `output` yang seukuran dengan jumlah data asli ( $N$ ), ditambah sebuah array perhitungan `count` sebesar basis decimal. Akibat pengalokasian memori ganda di setiap perputaran digitnya ini, ruang memori yang dibutuhkan sangat besar, menjadikannya algoritma dengan kompleksitas ruang  $O(N + k)$ . Sehingga, algoritma ini tidak tergolong *in-place*.

## 2. Kompleksitas Waktu

### a) Best Case

Algoritma ini bekerja dengan cara membedah angka digit per digit.

```

for (int exp = 1; angka_max / exp > 0; exp *= 10) {
    counting_sort(data, exp, m);
}

```

Meskipun program menerima input data Sorted, program tidak punya cara untuk mengetahuinya. Ia tetap akan mencari angka terbesar (`angka_max`) untuk mengetahui

jumlah digit maksimal ( $d$ ). Lalu perulangan pemotongan digit ( $\text{exp} *= 10$ ) akan terus berjalan memanggil fungsi `counting_sort` sebanyak jumlah digit tersebut. Di dalam `counting_sort` itu sendiri, penelusuran struktur data array ( $N$ ) dan rentang wadah angkanya ( $k=10$ ) selalu berjalan penuh. Oleh karena itu, kompleksitas waktu terbaiknya adalah  $O(d*(N + k))$ .

#### **b) Average Case**

Ketika memproses data Random, mekanisme penelusuran digitnya tidak terpengaruh sama sekali. Selama nilai maksimum di dalam array tersebut tetap sama, program akan melakukan pemecahan ke dalam `count` hitungan secara proporsional dari indeks kiri ke kanan. Karena proses membagi, menghitung, dan menempatkan angkanya bersifat linier, kompleksitas rata-ratanya juga stabil di angka  $O(d*(N + k))$ .

#### **c) Worst Case**

Kondisi data Reverse Sorted tidak memicu masalah apapun seperti sort yang lain. Program mengabaikan urutan awal elemen dan hanya melihat elemen dari struktur nilai belakangnya (satuan, puluhan, ratusan). Seluruh perulangan `for` untuk mengurai dan menyatukan array selalu tereksekusi secara konstan. Maka, kompleksitas waktu pada keadaan paling buruk sekalipun tidak akan bergeser dari  $O(d * (N + k))$ .

### **3. Analisis Operasi**

#### **a) Jumlah comparison**

Tercatat 0 (Tidak ada). Ini adalah satu-satunya algoritma penyortiran di praktikum ini yang tidak memakai logika membanding-bandingkan elemen satu sama lain ( $>$  atau  $<$ ) untuk memindahkan data.

#### **b) Jumlah swap**

Tercatat 0 (Tidak ada). Penempatan urutan dilempar menggunakan penugasan indeks langsung menuju wadah `output`, bukan dari metode swap (`std::swap`).

#### **c) Jumlah shift**

Tercatat 0 (Tidak ada).

#### **d) Jumlah recursion call**

Tercatat 0 (Tidak ada). Seluruh strukturnya menggunakan perulangan iteratif biasa *for-loop*.

**e) Jumlah array accessed**

Ini adalah metrik pengganti *comparison* untuk Radix Sort. Karena logika utamanya pada seberapa sering program membuka dan memanipulasi slot memori, kode `m.array_accesses++` ditempatkan di setiap pembacaan nilai *array*. Terlepas dari apakah datanya *Sorted* atau *Reverse*, besaran metrik pencatatan operasi memori ini selalu konsisten dan mentok di titik angka kisaran 1554 kali akses.

**f) Pengaruh bentuk data terhadap jumlah operasi**

Berbeda dari algoritma lain, derajat distribusi data di sini memiliki pengaruh. Satu-satunya hal pada data yang memengaruhi panjang pendeknya proses komputasi operasi algoritma ini hanyalah jumlah banyaknya digit penyusun angka terbesarnya. Semakin panjang digit angkanya, iterasi `counting_sort`-nya akan semakin sering dipanggil.

**4. Analisis Performa**

**a) Perbandingan waktu eksekusi antar algoritma**

Dalam pertandingan komputasi data berskala besar, Radix Sort mendominasi dari segi kecepatan. Karena ia tidak terikat batasan  $O(N \log N)$  milik algoritma pembandingan, Radix Sort mengungguli Merge Sort dan Bubble Sort, dengan mengorbankan alokasi ruang memori fisik.

**b) Performa pada dataset kecil dan besar**

Di skala populasi data kecil ( $N=100$ ), performanya di terminal menghasilkan waktu 0.005800 ms, beratnya *overhead* waktu untuk berkali-kali mencetak *vector array* helper dari nol. Akan tetapi, performanya cocok untuk himpunan data besar ( $N=10000$ ), di mana ia dengan cepat menyelesaikan penyusunan dengan hanya berkisar 0.552100 ms.

**c) Pengaruh distribusi data terhadap performa algoritma**

Rentang selisih durasi antara pemrosesan data berbentuk *Sorted*, *Random*, hingga *Reverse Sorted* cukup tipis. Hal ini mengonfirmasi bahwa selama rentang besaran angka maksimumnya masih sama, *processor* mengeksekusi beban muatan kerja yang setara pada keadaan urutan data yang bagaimanapun.

**d) Hubungan teori kompleksitas dengan hasil eksperimen**

Hasil keseluruhan metrik menjadi bukti kepastian teori kompleksitas  $O(d*(N+k))$ . Hasilnya menunjukkan bahwa catatan `m.array_accesses` bergantung pada pengali  $d$  (jumlah eksponen digit maksimal), dan tidak ada lonjakan eksponensial maupun

degradasi menjadi fungsi kuadratik saat susunan data masukannya diacak separah apapun.

## MODUL 5: SEARCHING

### SOAL 1

#### Linear Search

##### A. Source Code

*Tabel 25. Source Code Soal 1 Modul 5*

|    |                                                                               |
|----|-------------------------------------------------------------------------------|
| 1  | #include "searching_algorithms.h"                                             |
| 2  | #include <chrono>                                                             |
| 3  |                                                                               |
| 4  |                                                                               |
| 5  |                                                                               |
| 6  | using Clock = std::chrono::high_resolution_clock;                             |
| 7  |                                                                               |
| 8  |                                                                               |
| 9  | void linear_search(const std::vector<int>& data, int<br>target, Metrics& m) { |
| 10 | auto start = Clock::now();                                                    |
| 11 |                                                                               |
| 12 | m.found_index = -1;                                                           |
| 13 | m.comparisons = 0;                                                            |
| 14 |                                                                               |
| 15 | for (int i = 0; i < data.size(); ++i) {                                       |
| 16 | m.comparisons++;                                                              |
| 17 | if (data[i] == target) {                                                      |
| 18 | m.found_index = i;                                                            |
| 19 | break;                                                                        |
| 20 | }                                                                             |
| 21 | }                                                                             |
| 22 |                                                                               |

|    |                                                            |
|----|------------------------------------------------------------|
| 23 | <code>m.time_us = std::chrono::duration&lt;double,</code>  |
|    | <code>std::micro&gt;(Clock::now() - start).count();</code> |
| 24 | <code>}</code>                                             |

## B. Output

```

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\searching> g++ -std=c++23 -O0 main.cpp data_generator.cpp searching_algorithms.cpp reporter.cpp -o search
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\searching> ./search

=====
SEARCHING ALGORITHM - ANALYSIS
=====

>> Linear Search
-----
Algorithm      Case      N      Comparisons  Found Index  Time(us)
-----
Linear Search   Best      1000      1              0           1.100
Linear Search   Average   1000     375            374         4.300
Linear Search   Worst     1000    1000           Not Found   9.900
-----
Linear Search   Best     10000      1              0           0.500
Linear Search   Average  10000    3746           3745        60.300
Linear Search   Worst    10000   10000          Not Found   95.100
-----
Linear Search   Best     100000      1              0           0.500
Linear Search   Average  100000   37455          37454       362.000
Linear Search   Worst    100000  100000         Not Found  975.200
-----
Linear Search   Best     1000000      1              0           1.400
Linear Search   Average  1000000  374541         374540     4746.100
Linear Search   Worst    1000000 1000000         Not Found 11425.200
-----

```

Gambar 13. Screenshot Hasil Jawaban Soal 1 modul 5

## C. Pembahasan

### 1. Karakteristik Algoritma

Secara definisi, Linear Search adalah algoritma pencarian yang bekerja dengan menelusuri setiap elemen di dalam kumpulan data satu per satu secara berurutan. Karena yang diinginkan adalah mencari elemen dari ujung awal hingga akhir, maka pakai perulangan `for` mulai dari indeks 0 sampai batas sizenya. Untuk mencari elemen mana yang sesuai, pertama program akan membaca nilai pada array di indeks ke-*i* tersebut. Kemudian, nilai itu dicocokkan dengan nilai target menggunakan `if`. Jika nilainya sama, maka indeksnya disimpan lalu perulangan dihentikan menggunakan `break`. Jika nilainya tidak sama, maka nilai *i* akan dinaikkan (`++i`) sehingga iterasi bergeser menelusuri indeks selanjutnya. Kalau

sampai perulangan berakhir elemen target tidak juga ditemukan, maka biarkan indeks pencariannya tetap bernilai -1.

Karena algoritma ini bekerja hanya membaca nilai indeks per indeks tanpa ada pengecekan urutan angka sebelum atau sesudahnya, maka algoritma tidak memerlukan data terurut. Lalu karena program hanya berputar di dalam blok `for` tanpa adanya fungsi yang memanggil dirinya sendiri, maka algoritma ini bersifat iteratif. Namun, algoritma ini sebenarnya dapat diimplementasikan secara rekursif dengan cara fungsinya memanggil dirinya sendiri sambil memajukan nilai indeks satu per satu sebagai pengganti perulangan.

## **2. Kompleksitas Waktu**

Jika elemen target yang dicari kebetulan langsung ditemukan pada indeks pertama ( $i = 0$ ), perintah `break` di dalam kondisi tersebut akan langsung dieksekusi untuk menghentikan paksa perulangan saat itu juga. Karena algoritmanya hanya membutuhkan satu langkah operasi saja dan langsung keluar dari perulangan tanpa memedulikan banyak sisa datanya, maka kompleksitas waktu untuk best case adalah  $O(1)$ .

Sebaliknya, jika elemen target di posisi acak, berada di ujung, atau bahkan tidak ada sama sekali, perintah `break` tidak akan bisa tercapai dengan cepat. Kode perulangan `for` ini akan memaksa program untuk terus menelusuri dan mencocokkan data satu per satu secara berulang hingga  $i$  menyentuh batas akhir ukuran data (`data.size()`). Karena jumlah langkah pengecekan ini akan terus meningkat bersamaan dengan bertambahnya ukuran total data ( $n$ ), maka kompleksitas waktu untuk average dan worst case adalah  $O(n)$ .

## **3. Analisis Operasi**

Pada eksperimen Linear Search, jumlah comparison yang dilakukan bergantung pada letak datanya. Best Case memakan 1 kali perbandingan, Average Case berada di pertengahan (374.541 kali pada 1 juta data), dan Worst Case memeriksa penuh seluruh data (1.000.000 kali). Jika elemen ditemukan, posisinya langsung disimpan ke variabel `index_found` (seperti indeks ke-0 atau 374540), namun jika seluruh data habis diperiksa tanpa hasil, `index_found` dikembalikan bernilai -1 (Not Found). Karena penelusurannya berurutan dari indeks terawal, posisi elemen target sangat menentukan jumlah comparison, target di depan mempercepat pencarian, sedangkan target di belakang memperlama pencarian. Karena alurnya mengecek array satu per satu, maka besarnya ukuran data ( $n$ ) akan memberikan pengaruh peningkatan jumlah operasi secara linear.



## SOAL 2

### Binary Search

#### A. Source Code

*Tabel 26. Source Code Soal 2 Modul 5*

|    |                                                                             |
|----|-----------------------------------------------------------------------------|
| 28 | <code>void binary_search(const std::vector&lt;int&gt;&amp; data, int</code> |
|    | <code>target, Metrics&amp; m) {</code>                                      |
| 29 | <code>    auto start = Clock::now();</code>                                 |
| 30 |                                                                             |
| 31 | <code>    m.found_index = -1;</code>                                        |
| 32 | <code>    m.comparisons = 0;</code>                                         |
| 33 |                                                                             |
| 34 | <code>    int left = 0;</code>                                              |
| 35 | <code>    int right = data.size() - 1;</code>                               |
| 36 |                                                                             |
| 37 | <code>    while (left &lt;= right) {</code>                                 |
| 38 | <code>        int mid = left + (right - left) / 2;</code>                   |
| 39 |                                                                             |
| 40 | <code>        m.comparisons++;</code>                                       |
| 41 | <code>        if (data[mid] == target) {</code>                             |
| 42 | <code>            m.found_index = mid;</code>                               |
| 43 | <code>            break;</code>                                             |
| 44 | <code>        } else if (data[mid] &lt; target) {</code>                    |
| 45 | <code>            left = mid + 1;</code>                                    |
| 46 | <code>        } else {</code>                                               |
| 47 | <code>            right = mid - 1;</code>                                   |
| 48 | <code>        }</code>                                                      |
| 49 | <code>    }</code>                                                          |
| 50 |                                                                             |

```

51     m.time_us      =      std::chrono::duration<double,
      std::micro>(Clock::now() - start).count();
52 }

```

## B. Output

```

>> Binary Search
-----
Algorithm      Case      N      Comparisons  Found Index  Time(us)
-----
Binary Search  Best      1000      9           500         1.600
Binary Search  Average   1000      3           374         0.700
Binary Search  Worst     1000     10          Not Found   0.700
-----
Binary Search  Best     10000     13          5000        0.700
Binary Search  Average  10000     13          3745        0.700
Binary Search  Worst    10000     14          Not Found   0.700
-----
Binary Search  Best    100000    16          50000       0.900
Binary Search  Average 100000    16          37454       0.800
Binary Search  Worst   100000    17          Not Found   0.900
-----
Binary Search  Best   1000000   19          500000      2.400
Binary Search  Average 1000000   15          374540      2.600
Binary Search  Worst   1000000   20          Not Found   0.800
-----
Done.
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\searching>

```

Gambar 14. Screenshot Hasil Jawaban Soal 2 Modul 5

## C. Pembahasan

### 1. Karakteristik Algoritma

Secara definisi, Binary Search adalah algoritma pencarian yang bekerja dengan cara membelah ruang pencarian menjadi dua bagian secara berulang-ulang. Karena yang diinginkan adalah membelah ruang pencarian menjadi dua, maka pertama ambil dan tentukan dulu batas `left` dan `right`. Lalu pakai perulangan `while` selama batas `left` lebih kecil atau sama dengan `right` maka akan dilakukan pemotongan rentang. Untuk menentukan sisa rentang mana yang akan dibuang, pertama mengambil nilai tengah (`mid`), kemudian baca nilai pada array di indeks tersebut. Jika nilainya lebih kecil dari target, maka batas `left` digeser ke kanan melewati `mid`. Jika nilainya lebih besar, maka batas `right` digeser ke kiri

melewati `mid`. Kalau nilainya ternyata sama persis dengan target, simpan indeksinya lalu `break` untuk keluar dari perulangan.

Mekanisme algoritma ini sangat bergantung pada perbandingan nilai untuk menentukan arah pergeseran rentang, maka algoritma ini memerlukan data terurut. Lalu, karena perulangannya dibuat berputar di dalam blok `while` demi efisiensi memori, maka algoritma ini bersifat iteratif. Meskipun begitu, metode membelah ruang ini dapat diimplementasikan secara rekursif dengan cara memanggil fungsinya sendiri sambil membawa nilai batas `left` atau `right` yang baru terpotong.

## 2. Kompleksitas Waktu

Secara teori, Best Case Binary Search adalah  $O(1)$  karena target seharusnya bisa langsung ditemukan dalam 1 kali tebakan pertama. Namun, pada hasil kode ini, jumlah perbandingan mencatat angka 9 hingga 19 kali operasi akibat posisi target berada di indeks genap (indeks 500) sedangkan perhitungan `int mid = left + (right - left) / 2` menghasilkan pembulatan ke bawah (indeks 499). Karena tebakan awal meleset satu angka, alur program jadi harus melakukan beberapa kali pembelahan ekstra di terminal untuk bergeser menemukan target tersebut (menjadi seperti average case). Meski begitu, nilai kompleksitas teorinya tidak berubah karena jika angka target langsung bertepatan dengan titik tengah pada pengecekan pertama, perintah `if (data[mid] == target)` akan langsung bernilai benar sehingga perintah `break` segera memutus perulangan. Maka, kompleksitas waktu untuk kondisi terbaik dari bagian kode ini secara konsep tetap bernilai  $O(1)$ .

Namun, jika angka target tidak tertebak pada langkah pertama, program harus membuang setengah bagian data yang tidak sesuai. Algoritma ini didesain untuk mereduksi sisa area pencarian dengan cara menggeser batas `left` atau `right`. Di dalam perulangan `while (left <= right)`, modifikasi batas ini secara konsisten selalu memotong ukuran rentang data menjadi setengah dari ukuran sebelumnya pada setiap putarannya. Karena program tidak menelusuri data satu per satu, melainkan terus membaginya dengan angka dua sampai rentang datanya habis, jumlah operasinya tidak akan bertambah secara drastis meskipun ukuran datanya jutaan. Maka, kompleksitas waktu untuk average dan worst case-nya adalah  $O(\log n)$ .

### 3. Analisis Operasi

Berdasarkan hasil pengujian pada terminal kode, jumlah comparison yang dilakukan hingga seluruh data diperiksa terbukti efisien meski tebakan awalnya meleset, pada dataset 1 juta data, Best Case melakukan 19 kali comparison, Average Case 15 kali comparison, dan Worst Case di 20 kali comparison. Jika nilai tengah yang diperiksa cocok dengan target, posisinya langsung disimpan ke variabel `index_found` (indeks 500, 374, atau 500000), namun jika batas rentang data sudah habis terbagi dan target tetap tidak ada, `index_found` akan menampilkan -1 (Not Found).

Karena bagian kode pembelahan rentang ini selalu menebak dari nilai tengah, posisi elemen target tetap berpengaruh terhadap naik-turunnya jumlah comparison yang terjadi. Walau begitu, karena alur programnya selalu membuang setengah area data pada setiap putaran `while`, maka membesarnya ukuran data ( $n$ ) dari 1.000 ke 1.000.000 elemen hanya memberikan pengaruh penambahan jumlah comparison yang sangat kecil.

### 4. Analisis Performa

Pada dataset kecil ( $N = 1.000$ ), waktu eksekusi Linear dan Binary Search sama-sama efisien karena berada di bawah 10  $\mu s$ . Namun pada dataset besar ( $N = 1.000.000$ ), waktu Linear Search naik drastis hingga 11.425  $\mu s$ , sedangkan Binary Search stabil di angka 0,8 - 2,6  $\mu s$ . Karena Binary Search mampu menahan kenaikan waktu pada data masif, maka Binary Search jauh lebih efisien untuk dataset besar, sementara Linear Search hanya efisien untuk dataset kecil.

Dikarenakan Binary Search bekerja dengan metode membelah rentang angka, algoritmanya akan gagal beroperasi jika data yang diberikan acak. Sebaliknya, Linear Search mencari dengan cara menelusuri satu per satu data sehingga tidak terpengaruh oleh susunan datanya. Maka, kondisi data yang teratur adalah syarat wajib bagi performa Binary Search, namun tidak memberikan pengaruh sama sekali pada Linear Search.

Pada pengujian 1 juta data (worst case), Linear Search tercatat melakukan 1.000.000 kali comparisons yang menyebabkan lonjakan waktu linear. Sebaliknya, Binary Search hanya membutuhkan maksimal 20 kali comparisons. Karena angka maksimal 20 pada pembelahan data ini sejalan dengan hitungan  $\log_2(1000000) \approx 19.93$ , maka hasil ini membuktikan kebenaran teori kompleksitas  $O(n)$  untuk Linear Search dan  $O(\log n)$  untuk Binary Search.

## MODUL 6: GRAPH AND TREE

### Soal 1

#### Konversi Adjacency List ke Adjacency Matrix

##### A. Source Code

*Tabel 27. Source Code Soal 1 Modul 6*

|    |                               |
|----|-------------------------------|
| 1  | #include <iostream>           |
| 2  |                               |
| 3  | using namespace std;          |
| 4  |                               |
| 5  | int main() {                  |
| 6  | int N;                        |
| 7  | cin >> N;                     |
| 8  |                               |
| 9  | char vertex[10];              |
| 10 | for (int i = 0; i < N; i++) { |
| 11 | cin >> vertex[i];             |
| 12 | }                             |
| 13 |                               |
| 14 | int M;                        |
| 15 | cin >> M;                     |
| 16 |                               |
| 17 | int matrix[10][10];           |
| 18 | for (int i = 0; i < N; i++) { |
| 19 | for (int j = 0; j < N; j++) { |
| 20 | matrix[i][j] = 0;             |
| 21 | }                             |
| 22 | }                             |
| 23 |                               |

```

24     for (int i = 0; i < M; i++) {
25         char U, V;
26         int W;
27         cin >> U >> V >> W;
28
29         int indexU = 0, indexV = 0;
30
31         for (int j = 0; j < N; j++) {
32             if (vertex[j] == U) indexU = j;
33             if (vertex[j] == V) indexV = j;
34         }
35
36         matrix[indexU][indexV] = W;
37     }
38
39     cout << "Adjacency Matrix:\n ";
40     for (int i = 0; i < N; i++) {
41         cout << vertex[i];
42         if (i != N - 1) cout << " ";
43     }
44     cout << "\n";
45
46     for (int i = 0; i < N; i++) {
47         cout << vertex[i] << " ";
48         for (int j = 0; j < N; j++) {
49             cout << matrix[i][j];
50             if (j != N - 1) cout << " ";
51         }
52         cout << "\n";
53     }
54

```

|    |           |
|----|-----------|
| 55 | return 0; |
| 56 | }         |

## B. Output

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> g++ soal1.cpp -o soal1
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> ./soal1
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree>
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> |

```

Gambar 15. Screenshot Hasil Jawaban Soal 1 Modul 6

## C. Pembahasan

Untuk penyelesaian Soal 1, kode ini dibikin buat ngubah representasi graf dari bentuk daftar *edge* ke dalam bentuk matriks dua dimensi berukuran  $N \times N$ . Pendekatannya lumayan simpel. Awalnya, program bakal inisialisasi matriks maksimal  $10 \times 10$  dengan nilai 0 semua. Nilai 0 ini menandai kalau belum ada sisi atau jalur yang nyambung antar *vertex*. Terus, karena input *vertex*-nya berupa huruf alfabet, program butuh ngelakuin *sequential search* buat mencari indeks dari titik asal (U) dan titik tujuan (V) sebelum masukin bobotnya (W) ke koordinat sel matriks yang tepat. Asumsi utamanya mengikuti batasan dari soal, yaitu graf maksimal cuma punya 10 titik, sifatnya *directed* (searah), dan tiap label *vertex* dipastikan unik.

Kalau dianalisis dari segi kompleksitasnya, algoritma ini efisien banget buat batasan soal yang dikasih:

### **Kompleksitas Waktu**

Proses inisialisasi matriks awal dan cetak output butuh waktu  $O(N^2)$ . Pas *looping* untuk masukin sisi, ada pencarian indeks yang butuh  $O(N)$  tiap kalinya buat sebanyak  $M$  sisi, jadinya memakan waktu  $O(N \times M)$ . Total kompleksitas waktunya kira-kira  $O(N^2 + N \times M)$ . Karena nilai  $N$  maksimal cuma 10, eksekusinya sudah pasti bakal cepat dan lolos dari *time limit* 1.0 detik.

### **Kompleksitas Ruang**

Memori yang dipakai mayoritas cuma dialokasiin buat deklarasi *array* nama *vertex* sama matriks. Jadi *space complexity*-nya cuma butuh  $O(N^2)$ . Alokasi segini tergolong sangat kecil buat melewati batasan *memory limit* 64 MB.

Berdasarkan hasil *testing* pake contoh kasus di soal, logikanya udah terbukti jalan dengan bener buat menangani *directed weighted graph*. Misalnya pas dikasih input sisi A B 4, program sukses ngubah sel di baris A dan kolom B jadi angka 4. Dan karena graf ini sifatnya searah, selbaliknya (baris B kolom A) tetep dipertahankan bernilai 0. Algoritma ini juga terbukti berhasil menangani titik yang ga punya rute keluar sama sekali (*vertex* E di contoh kasus), yang mana di hasil akhirnya seluruh baris E dibiarin tetep 0. Kesimpulannya, output matriksnya udah akurat dan sesuai persis sama permasalahan yang diminta.



## Soal 2

### Konversi Adjacency Matrix ke Adjacency List

#### A. Source Code

Tabel 28. Source Code Soal 2 Modul 6

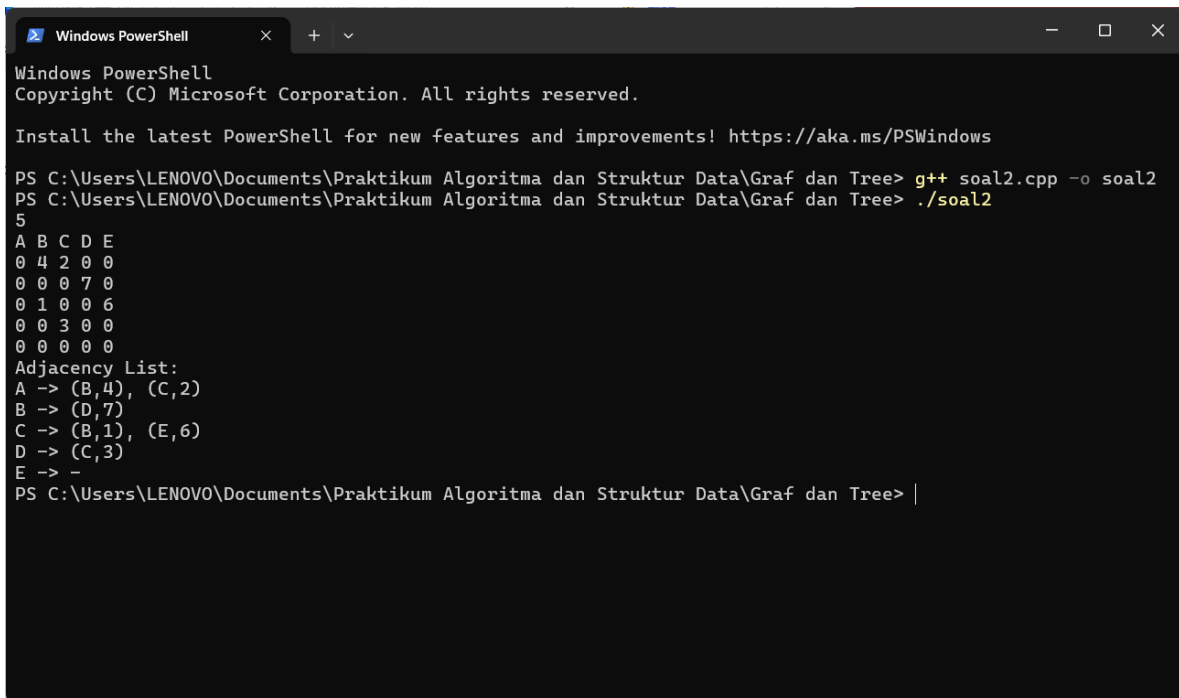
|    |                               |
|----|-------------------------------|
| 1  | #include <iostream>           |
| 2  |                               |
| 3  | using namespace std;          |
| 4  |                               |
| 5  | int main() {                  |
| 6  | int N;                        |
| 7  | cin >> N;                     |
| 8  |                               |
| 9  | char vertex[10];              |
| 10 | for (int i = 0; i < N; i++) { |
| 11 | cin >> vertex[i];             |
| 12 | }                             |
| 13 |                               |
| 14 | int matrix[10][10];           |
| 15 | for (int i = 0; i < N; i++) { |
| 16 | for (int j = 0; j < N; j++) { |
| 17 | cin >> matrix[i][j];          |
| 18 | }                             |
| 19 | }                             |
| 20 |                               |
| 21 | cout << "Adjacency List:\n";  |
| 22 | for (int i = 0; i < N; i++) { |
| 23 | cout << vertex[i] << " -> ";  |
| 24 |                               |
| 25 | bool hasEdge = false;         |

```

26
27     for (int j = 0; j < N; j++) {
28         if (matrix[i][j] > 0) {
29             if (hasEdge) {
30                 cout << ", ";
31             }
32             cout << "(" << vertex[j] << ", " <<
matrix[i][j] << ")";
33             hasEdge = true;
34         }
35     }
36
37     if (!hasEdge) {
38         cout << "-";
39     }
40     cout << "\n";
41 }
42
43     return 0;
44 }

```

## B. Output



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> g++ soal2.cpp -o soal2
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> ./soal2
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> |
```

Gambar 16. Screenshot Hasil Jawaban Soal 2 Modul 6

## C. Pembahasan

Untuk penyelesaian Soal 2, kode ini dibuat buat menangani konversi dari representasi *adjacency matrix* ke dalam bentuk *adjacency list*. Pertama-tama, program bakal ngebaca input jumlah *vertex* (N) dan nyimpen label karakternya ke dalam *array*. Habis itu, program lanjut ngebaca nilai bobot ke dalam matriks dua dimensi. Nah, buat proses cetaknya, teknik utamanya pake *nested loop*. *Looping* baris yang di luar dipake buat nandain *vertex* asal, dan di tahap ini program bakal langsung nyetak karakter *vertex* tersebut ditambah tanda panah ( $\rightarrow$ ). Terus, *looping* yang di dalem bakal ngecek tiap kolom di baris itu. Kalau menemukan nilai sel yang lebih dari 0, itu nandain ada rute berbobot, jadi program bakal nyetak *vertex* tujuan dan bobotnya dengan format  $(V, W)$ . Di dalem proses ini juga diselipin variabel boolean *flag* (yaitu `hasEdge`) buat menangani penambahan tanda koma secara dinamis biar outputnya rapi kalau ada lebih dari satu jalur. Asumsi yang dipake juga mengacu sama batesan soal, di mana program harus bisa menangani kondisi pas suatu titik ga punya *edge*

keluar sama sekali. Kalau variabel boolean tadi tetap bernilai *false* sampai *looping* dalam kelar, program bakal otomatis nambahin tanda minus (-).

Kalau dilihat dari segi performanya, struktur algoritma ini udah sangat pas dan efisien buat ngerjain soalnya:

### **Kompleksitas Waktu**

Karena algoritma ini wajib memeriksa seluruh elemen sel di dalam matriks dua dimensi berukuran  $N \times N$  tanpa terkecuali, maka kompleksitas waktu yang dibutuhkan adalah  $O(N^2)$ . Untuk batasan nilai  $N$  maksimal yang cuma 10, proses penelusuran dan pencetakan ini selesai cepat dan aman dari batasan *time limit* 1.0 detik.

### **Kompleksitas Ruang**

Memori yang dialokasikan pada program ini utamanya cuma dipakai buat nyimpen *array* label *vertex* berukuran 10 dan matriks berukuran  $10 \times 10$ . Jadi, kompleksitas ruangnya adalah  $O(N^2)$ . Penggunaan memori ini tergolong kecil, jadinya ga ada masalah sama batasan *memory limit* 64 MB di soal.

Berdasarkan hasil testing pake contoh kasus uji, logika program ini udah terbukti bener dalam merepresentasikan *adjacency list* dari sebuah matriks. Sebagai pembuktian, misal pada baris A di matriks terdapat nilai 4 di kolom B dan nilai 2 di kolom C, program dengan sukses mencetak output  $A \rightarrow (B, 4), (C, 2)$ . Program ini juga terbukti berhasil menangani kondisi pas ketemu baris yang isinya angka 0 semua, contohnya kayak pada *vertex* E yang memang ga punya *edge* keluar sama sekali. Di hasil akhir, baris E sukses dicetak sebagai  $E \rightarrow$  - tanpa merusak format baris lainnya. Ini menunjukkan kalau pemetaan graf berarah dan berbobot dari matriks ke bentuk list udah akurat dan sesuai sama spesifikasi permasalahan yang diminta.

## Soal 3

### BFS: Jarak Terpendek Keluar Labirin

#### A. Source Code

Tabel 29. Source Code Soal 3 Modul 6

|    |                               |
|----|-------------------------------|
| 1  | #include <iostream>           |
| 2  | #include <queue>              |
| 3  |                               |
| 4  | using namespace std;          |
| 5  |                               |
| 6  | struct Position {             |
| 7  | int row, column, distance;    |
| 8  | };                            |
| 9  |                               |
| 10 | int G[105][105];              |
| 11 | bool visited[105][105];       |
| 12 |                               |
| 13 | int main() {                  |
| 14 | int R, C;                     |
| 15 | cin >> R >> C;                |
| 16 |                               |
| 17 | for (int i = 0; i < R; i++) { |
| 18 | for (int j = 0; j < C; j++) { |
| 19 | cin >> G[i][j];               |
| 20 | visited[i][j] = false;        |
| 21 | }                             |
| 22 | }                             |
| 23 |                               |
| 24 | int SR, SC;                   |
| 25 | cin >> SR >> SC;              |

```

26
27     int FR, FC;
28     cin >> FR >> FC;
29
30     queue<Position> queue_positions;
31     queue_positions.push({SR, SC, 0});
32     visited[SR][SC] = true;
33
34     int X = -1;
35     int direction_row[] = {-1, 1, 0, 0};
36     int direction_column[] = {0, 0, -1, 1};
37
38     while (!queue_positions.empty()) {
39         Position current = queue_positions.front();
40         queue_positions.pop();
41
42         if (current.row == FR && current.column == FC)
43         {
44             X = current.distance;
45             break;
46         }
47
48         for (int i = 0; i < 4; i++) {
49             int next_row = current.row +
direction_row[i];
50             int next_column = current.column +
direction_column[i];
51
52             if (next_row >= 0 && next_row < R &&
next_column >= 0 && next_column < C) {

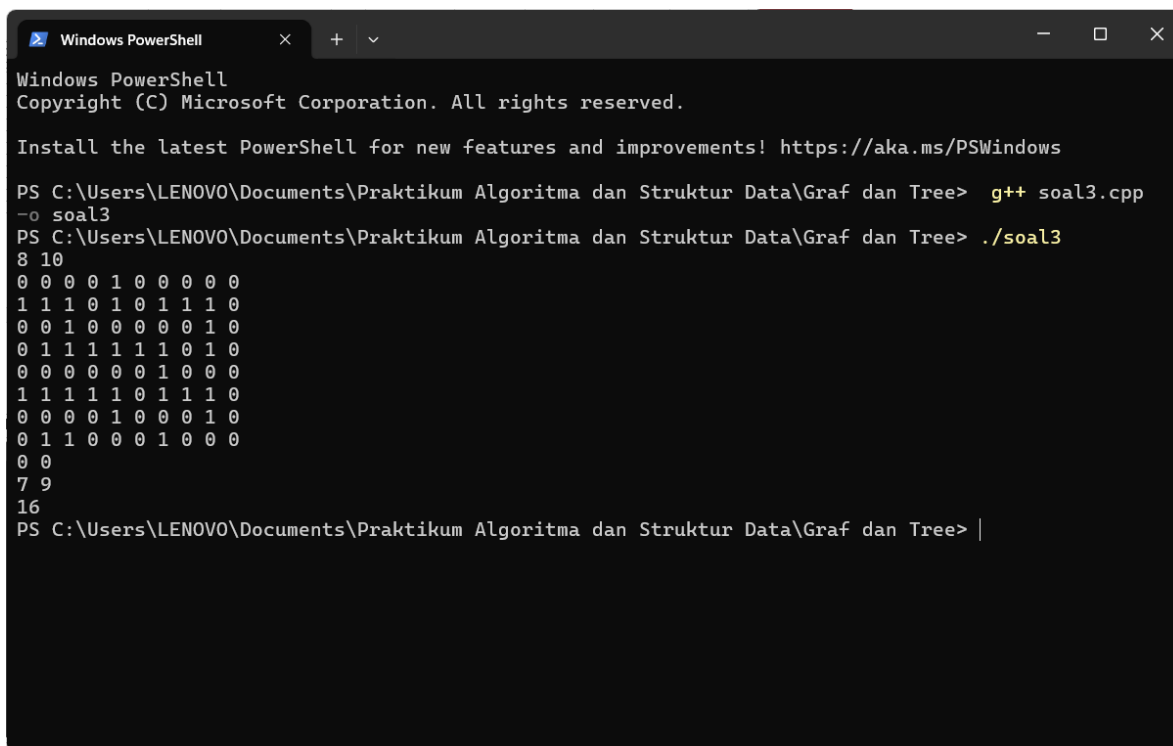
```

```

52         if (G[next_row][next_column] == 0 &&
!visited[next_row][next_column]) {
53             visited[next_row][next_column] =
true;
54             queue_positions.push({next_row,
next_column, current.distance + 1});
55         }
56     }
57 }
58 }
59
60     cout << X << "\n";
61     return 0;
62 }

```

## B. Output



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> g++ soal3.cpp
-o soal3
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> ./soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> |

```

Gambar 17. Screenshot Hasil Jawaban Soal 3 Modul 6

### C. Pembahasan

Untuk penyelesaian Soal 3, kode ini dirancang buat nyari langkah minimum atau rute terpendek buat keluar dari sebuah labirin, yang mana paling cocok diselesaikan pake algoritma *Breadth-First Search* (BFS). Proses kerjanya bertahap. Pertama, program menyiapkan *array* dua dimensi berukuran maksimal 105x105 untuk nyimpan grid peta (G) dan *array* boolean *visited* buat nandain sel mana aja yang sudah pernah dilewatin. Karena BFS butuh struktur data antrean, program pakai *queue* buat nyimpen data koordinat baris, kolom, dan jumlah jarak (langkah) dari titik awal. Awalnya, titik *start* (SR, SC) akan dimasukin ke dalem antrean dengan jarak 0 dan langsung ditandain *true* di *array visited*. Lalu, program bakal masuk ke *looping* utama buat ngeluarin titik paling depan dari antrean dan ngecek 4 arah pergerakan (atas, bawah, kiri, kanan). Kalau titik pergerakan selanjutnya ternyata valid (masih di dalem area grid, nilainya 0 yang berarti bisa dilewati, dan belom pernah dikunjungi), titik itu bakal dimasukin ke dalem antrean dan jaraknya di-plus 1. Proses ini bakal terus berulang sampai antrean kosong atau koordinat saat ini udah sama persis sama titik tujuan (FR, FC). Asumsi dari algoritma ini adalah labirin ga punya arah diagonal, pergerakan terbatas pada 4 arah, dan variabel default jarak X udah di-set -1 dari awal buat jaga-jaga kalau jalan keluarnya beneran buntu.

Kalau dianalisis dari segi efisiensinya, algoritma BFS ini udah merupakan solusi paling optimal buat kasus pencarian rute terpendek di *unweighted graph*:

#### Kompleksitas Waktu

Pada kasus terburuk (*worst case*), algoritma BFS ini bakal ngunjungi setiap sel di dalem peta labirin tepat satu kali. Oleh karena itu, kompleksitas waktu yang dibutuhkan berbanding lurus sama luas petanya, yaitu sebesar  $O(R \times C)$ , dengan R adalah jumlah baris dan C adalah jumlah kolom. Karena ukuran maksimal R dan C di soal adalah 100, algoritma ini berjalan efisien dan ga kena masalah *time limit*.

#### Kompleksitas Ruang

Memori yang dipake mayoritas teralokasi untuk menyimpan grid matriks 105x105, *array visited* 105x105, serta memori dinamis untuk struktur antrean (*queue*) yang ukuran



maksimalnya sebesar jumlah sel yang bisa dilewati. Jadi *space complexity*-nya adalah  $O(R \times C)$ . Ukuran ini juga masih sangat proporsional dan bakal lolos ngejalanin *memory limit*.

Berdasarkan analisis kasus yang dikasih, logika BFS ini terbukti sangat akurat dalam mencari rute. Sifat dasar dari BFS ngejamin kalau titik tujuan (FR, FC) pertama kali ditemui di dalam *queue*, rute tersebut pastilah rute yang paling optimal karena pencariannya tersebar secara merata per lapis. Pas program nyentuh titik akhir, dia bakal langsung *break* dan nyimpen langkah minimalnya ke variabel X. Algoritma ini juga terbukti berhasil menangani kondisi ekstrim di mana jalan keluarnya bener-bener ketutup rintangan semua atau ga ada jalur sama sekali yang menghubungkan *start* dan *finish*. Dalam kasus kayak gini, karena *looping* selesai tanpa pernah nyentuh titik akhir, program bakal tetep mencetak nilai bawaan dari variabel X yaitu -1, yang secara akurat merepresentasikan kalau solusi jalurnya emang ga ada. Jadi secara pembuktian metrik, jalannya program udah sangat pas dengan ekspektasi masalahnya.

## Soal 4

### DFS: Menghitung Semua Jalur Keluar Labirin

#### A. Source Code

Tabel 30. Source Code Soal 4 Modul 6

|    |                                         |
|----|-----------------------------------------|
| 1  | #include <iostream>                     |
| 2  |                                         |
| 3  | using namespace std;                    |
| 4  |                                         |
| 5  | int R, C;                               |
| 6  |                                         |
| 7  | int G[10][10];                          |
| 8  | bool visited[10][10];                   |
| 9  |                                         |
| 10 | int FR, FC;                             |
| 11 | int T = 0;                              |
| 12 |                                         |
| 13 | int direction_row[] = {-1, 1, 0, 0};    |
| 14 | int direction_column[] = {0, 0, -1, 1}; |
| 15 |                                         |
| 16 | void dfs(int row, int column) {         |
| 17 | if (row == FR && column == FC) {        |
| 18 | T++;                                    |
| 19 | return;                                 |
| 20 | }                                       |
| 21 |                                         |
| 22 | visited[row][column] = true;            |
| 23 |                                         |
| 24 | for (int i = 0; i < 4; i++) {           |
| 25 | int next_row = row + direction_row[i];  |

```

26         int next_column = column + direction_column[i];
27
28         if (next_row >= 0 && next_row < R && next_column
29         >= 0 && next_column < C) {
30             if (G[next_row][next_column] == 0 &&
31             !visited[next_row][next_column]) {
32                 dfs(next_row, next_column);
33             }
34         }
35         visited[row][column] = false;
36     }
37
38     int main() {
39         cin >> R >> C;
40
41         for (int i = 0; i < R; i++) {
42             for (int j = 0; j < C; j++) {
43                 cin >> G[i][j];
44                 visited[i][j] = false;
45             }
46         }
47
48         int SR, SC;
49         cin >> SR >> SC;
50         cin >> FR >> FC;
51
52         dfs(SR, SC);
53
54         cout << T << "\n";

```

|    |           |
|----|-----------|
| 55 | return 0; |
| 56 | }         |

## B. Output

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> g++ soal4.cpp -o soal4
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> ./soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree>

```

Gambar 18. Screenshot Hasil Jawaban Soal 4 Modul 6

## C. Pembahasan

Untuk penyelesaian Soal 4, kode ini dirancang buat menghitung total seluruh jalur yang mungkin dilewat dari titik awal ke titik tujuan di dalam labirin. Algoritma yang paling tepat untuk menangani permasalahan ini adalah *Depth-First Search* (DFS) yang dikombinasikan dengan teknik *backtracking*. Proses kerjanya dimulai dengan membaca batesan ukuran grid R dan C, representasi matriks labirin (G), serta koordinat awal (SR, SC) dan akhir (FR, FC). Di dalam fungsi `dfs(row, column)`, program bakal melakukan penelusuran rute sedalam mungkin ke satu cabang jalur terlebih dahulu. Tiap kali program menginjak suatu sel, sel tersebut langsung ditandai `true` di array boolean `visited` supaya penelusuran di

cabang rute yang sama ga *looping* di titik yang sama. Dari titik itu, program bakal ngecek validitas pergerakan ke 4 arah pakai bantuan array `direction_row` dan `direction_column`. Kalau sel nextnya bernilai 0 (bisa dilewat) dan statusnya belum pernah dikunjungi, fungsi `dfs` bakal dipanggil lagi secara rekursif buat ngelanjutin penelusuran rute tersebut. Kunci utamanya ada di bagian akhir fungsi rekursif, yaitu proses *backtracking*. Pas suatu cabang penelusuran udah mentok (nemu jalan buntu) atau selesai dieksplorasi, program bakal mengembalikan status `visited` sel tersebut menjadi `false` lagi sebelum fungsi keluar. Tujuannya adalah supaya sel itu bisa digunain lagi oleh cabang rute alternatif lain yang mau lewat situ. Setiap kali langkah rekursif berhasil menyentuh target koordinat akhir (FR, FC), nilai pada variabel global `T` bakal otomatis bertambah satu (+1) sebagai tanda satu jalur sederhana telah ditemukan. Asumsi dari algoritma ini adalah penelusuran hanya mencari jalur sederhana dan rute diagonal tidak dihitung.

Jika dianalisis dari segi performanya, struktur algoritma ini udah pas buat menangani batasan yang diberikan di soal.

### **Kompleksitas Waktu**

Karena algoritma ini menggunakan teknik *backtracking* untuk mengeksplorasi seluruh kemungkinan kombinasi rute yang ada di dalam grid, secara teori kompleksitas waktunya bersifat eksponensial pada kasus terburuk (*worst case*), yaitu berkisar  $O(4^{(R \times C)})$ . Namun, karena batasan nilai `R` dan `C` di soal sangat kecil (maksimal cuma 7x7 atau 49 sel), total variasi jalurnya ga bakal terlalu banyak. Proses pencarian ini dipastikan bakal selesai cepat dan pastinya aman dari batasan *time limit* 1.0 detik.

### **Kompleksitas Ruang**

Alokasi memori pada program ini sangat hemat karena cuma dipake buat nyimpen matriks `G` ukuran 10x10 dan array `visited` ukuran 10x10. Selain itu, penggunaan memori tambahan cuma berasal dari *recursion stack* saat fungsi DFS dipanggil, yang mana kedalaman maksimalnya sebanding dengan jumlah total sel dalam labirin. Jadi, kompleksitas ruangnya adalah  $O(R \times C)$ . Penggunaan memori ini kecil dan aman dari batasan *memory limit* 64 MB.

Berdasarkan hasil testing menggunakan kasus uji, logika DFS dan *backtracking* ini terbukti benar dan akurat dalam menghitung variasi jalur. Sebagai pembuktian, mekanisme *backtracking* terbukti sukses membersihkan kembali jejak langkah (`visited = false`) tiap kali rute pencarian mundur dari jalan buntu. Hal ini membuat cabang penelusuran yang lain tetap bisa nemu jalur baru yang berbeda walaupun sempat melewati sel yang sama. Begitu fungsi rekursif menyentuh koordinat tujuan, variabel T sukses mencatat penambahan jalur secara tepat. Program ini juga terbukti berhasil menangani kondisi khusus pas posisi *start* dan *finish* benar-bener terisolasi oleh tembok (nilai 1). Dalam kasus buntu total seperti itu, fungsi dfs bakal kelar tanpa pernah memicu kondisi `row == FR && column == FC`, sehingga hasil akhir yang dicetak di layar tetap bernilai 0 sesuai sama ekspektasi permasalahan yang diminta di soal.

## Soal 5

### Tree Traversal

#### A. Source Code

*Tabel 31. Source Code Soal 5 Modul 6*

|    |                                                                     |
|----|---------------------------------------------------------------------|
| 1  | <code>#include &lt;iostream&gt;</code>                              |
| 2  | <code>#include &lt;string&gt;</code>                                |
| 3  | <code>#include "RedBlackTree.h"</code>                              |
| 4  |                                                                     |
| 5  | <code>using namespace std;</code>                                   |
| 6  |                                                                     |
| 7  | <code>void print_preorder(const RedBlackTree::Node* node) {</code>  |
| 8  | <code>    if (node-&gt;isNil) return;</code>                        |
| 9  |                                                                     |
| 10 | <code>    cout &lt;&lt; " " &lt;&lt; node-&gt;key;</code>           |
| 11 | <code>    print_preorder(node-&gt;left);</code>                     |
| 12 | <code>    print_preorder(node-&gt;right);</code>                    |
| 13 | <code>}</code>                                                      |
| 14 |                                                                     |
| 15 | <code>void print_inorder(const RedBlackTree::Node* node) {</code>   |
| 16 | <code>    if (node-&gt;isNil) return;</code>                        |
| 17 |                                                                     |
| 18 | <code>    print_inorder(node-&gt;left);</code>                      |
| 19 | <code>    cout &lt;&lt; " " &lt;&lt; node-&gt;key;</code>           |
| 20 | <code>    print_inorder(node-&gt;right);</code>                     |
| 21 | <code>}</code>                                                      |
| 22 |                                                                     |
| 23 | <code>void print_postorder(const RedBlackTree::Node* node) {</code> |
| 24 | <code>    if (node-&gt;isNil) return;</code>                        |
| 25 |                                                                     |

```

26     print_postorder(node->left);
27     print_postorder(node->right);
28     cout << " " << node->key;
29 }
30
31 int main() {
32     int N;
33     cin >> N;
34
35     RedBlackTree rbt;
36     for (int i = 0; i < N; i++) {
37         int A;
38         cin >> A;
39
40         if (!rbt.contains(A)) {
41             rbt.insert(A);
42         }
43     }
44
45     int Q;
46     cin >> Q;
47
48     for (int i = 0; i < Q; i++) {
49         string T;
50         cin >> T;
51
52         if (rbt.empty()) {
53             cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";
54             continue;
55         }

```

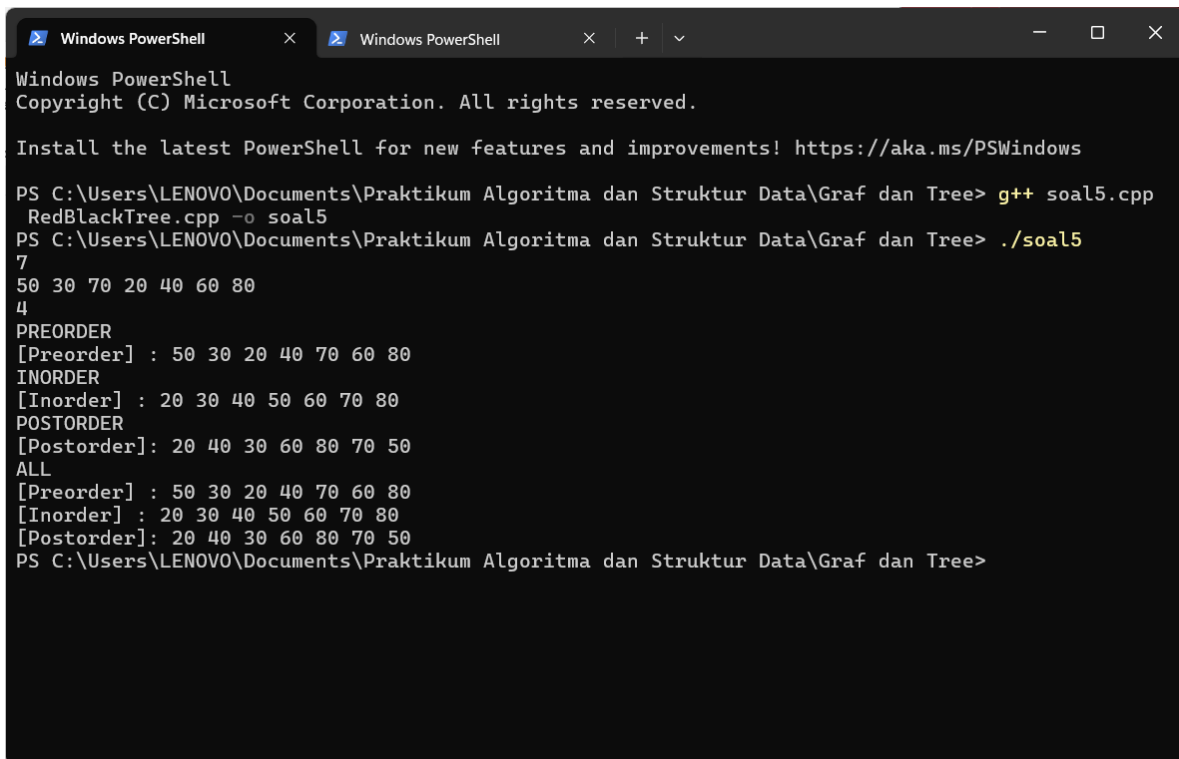


```

56
57     if (T == "PREORDER" || T == "ALL") {
58         cout << "[Preorder] :";
59         print_preorder(rbt.root());
60         cout << "\n";
61     }
62     if (T == "INORDER" || T == "ALL") {
63         cout << "[Inorder] :";
64         print_inorder(rbt.root());
65         cout << "\n";
66     }
67     if (T == "POSTORDER" || T == "ALL") {
68         cout << "[Postorder]:";
69         print_postorder(rbt.root());
70         cout << "\n";
71     }
72 }
73
74     return 0;
75 }

```

## B. Output



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> g++ soal5.cpp
RedBlackTree.cpp -o soal5
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> ./soal5
7
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder]: 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree>
```

Gambar 19. Screenshot Hasil Jawaban Soal 5 Modul 6

## C. Pembahasan

Untuk penyelesaian Soal 5, kode ini dirancang buat menangani operasi pembuatan struktur data *Red-Black Tree* (RBT) sekaligus melakukan penelusuran *traversal* di dalam pohonnya. Secara definisi, RBT itu semacam *Binary Search Tree* yang punya kemampuan menyeimbangkan dirinya sendiri biar kedalamannya tetep optimal. Proses kerjanya dibagi jadi dua fase utama. Pertama, program bakal ngebaca N bilangan dan masukin bilangannya satu per satu ke dalam pohon. Pas proses ini, diselipin juga teknik pengecekan pakai fungsi `contains` buat menangani input nilai duplikat, jadi angka yang udah ada di dalam pohon ga bakal dimasukin dua kali. Fase kedua adalah ngebaca perintah *query*. Nah, di tahap ini program dibikin interaktif, yang artinya tiap kali program kelar ngebaca satu perintah *query* (T) dari user, saat itu juga program langsung mengeksekusi cetakan `PREORDER`, `INORDER`, `POSTORDER`, atau `ALL` untuk pemanggilan fungsi rekursif. Jadi program ga perlu repot simpan

perintahnya dulu. Asumsi yang dibangun ngikutin aturan soal, di mana maksimal *query* (Q) adalah 20, dan penanda *node* kosong diasumsikan ga sekedar pake *null* biasa, tapi pake properti `isNil` bawaan dari strukturnya.

Kalau dianalisis dari segi efisiensinya, algoritma struktur data RBT ini jauh lebih stabil buat menangani data dibanding *Binary Tree* biasa.

### **Kompleksitas Waktu**

Untuk setiap proses masukin data (*insert*) dan ngecek duplikat (*contains*) ke dalam RBT, waktu yang dibutuhin adalah  $O(\log N)$ . Karena ada N data yang diinput, maka fase pertama butuh waktu  $O(N \log N)$ . Pada fase kedua, setiap eksekusi traversal bakal ngelewatin semua *node* yang ada, jadinya butuh waktu  $O(N)$ . Karena terdapat Q buah *query*, waktu untuk cetaknya adalah  $O(Q \times N)$ . Total keseluruhan kompleksitas waktunya  $O(N \log N + Q \times N)$ . Ini udah optimal dan bakal selesai di bawah *time limit* 1.0 detik.

### **Kompleksitas Ruang**

Memori utama cuma dialokasiin buat ngebangun struktur *node* di dalem pohon sebanyak N elemen. Karena di kodingan ini diterapin eksekusi *query* secara langsung dan ga butuh *array buffer* tambahan buat nyimpen string, maka kompleksitas ruangnya  $O(N)$ . Penggunaan memori ini sangat minim dan ga ngelewatin batasan *memory limit* 64 MB dari sistem.

Berdasarkan hasil eksekusi pake contoh soal, logika RBT dan mekanisme *input-output* interaktif ini terbukti akurat. Sebagai pembuktian, eksekusi *query* yang langsung dicetak sesaat setelah diinput terbukti jalan dengan mulus tanpa merusak alur pembacaan data. Terus, karena pada dasarnya RBT tetep mempertahankan sifat *Binary Search Tree*, pas program diminta ngejalanin perintah `INORDER`, output angkanya kebukti selalu tercetak urut dari yang terkecil ke yang terbesar. Logika pencegahan duplikat juga terbukti berhasil menangani angka kembar, misal kalau kita masukin angka 50 dua kali, cuma satu angka 50 yang kesimpen dan dicetak. Terakhir, program juga kebukti sukses menangani kondisi khusus, misalnya kalau kita ngasih *query* tapi pohonnya ga punya isian apa-apa, program ga bakal *crash* melainkan berhasil nyetak pesan "Tree kosong" sesuai sama yang diinstruksikan. Kesimpulannya, alur kerjanya udah sesuai persis sama ekspektasi permasalahan di soal.



## TAUTAN GIT

Berikut adalah tautan untuk semua source code yang telah dibuat.

<https://github.com/nullrym/PRAKTIKUM-ALGORITMA-DAN-STRUKTUR-DATA>